

IS 324

Database Systems II

By

Prof. Mohamed Anwar Assal

رؤية كلية الحاسبات والذكاء الاصطناعي

تسعى كلية الحاسبات والذكاء الاصطناعي لأن تكون صرحاً تعليمياً وبحثياً متميزاً في مجال علوم الحاسب وتكنولوجيا المعلومات يعمل وفقاً لمعايير الجودة العالمية بما يمكن من تحقيق المنافسة والتميز على كل من المستويات المحلية والإقليمية والعالمية.

رسالة كلية الحاسبات والذكاء الاصطناعي

تلتزم كلية الحاسبات والذكاء الاصطناعي بتقديم خدمات تعليمية أكاديمية تؤدي إلى خريج متميز قادر على المنافسة محلياً وإقليمياً وعالمياً وعلى كفاءة عالية في مجالات الحاسبات وتكنولوجيا المعلومات ويتمتع بمهارات وأخلاقيات مهنية رفيعة وقادر على التعلم الذاتي لمواكبة التطورات السريعة في مجال تخصصه وتقديم تعليم مستمر وخدمات تدريبية وإستشارية لتحقيق التنمية القومية والارتقاء بالبيئة المحلية وإجراء البحوث العلمية في مختلف مجالات علوم الحاسب وتكنولوجيا المعلومات.

Course Specification IS 324: Database Systems II

A- Affiliation:

Relevant Program: Computer Science - BSc program.

Information System - BSc program.

Department offering the program: Information System Department.

Department offering the course: Information System Department.

Date of Specification Approval: 19th February 2018

B-Basic Information:

Title: Database Systems II

Code: IS 324

Level: 3 **Semester:** Spring

Pre-requisite: Database Systems (I)

Tutorial/Practical: 2H

Lectures: 3H

Credit Hours: 3H

B- Professional Information

Relational database normalization. Database security. Database integrity. Database recovery. Database concurrency. Introduction to Distributed Database. Object Database Design using UML. Introduction to Data Warehouses. Introduction to Data Mining. Case studies: Database Applications.

1- Course Learning Objectives:

By the end of this course the students should demonstrate the knowledge and understanding of the concepts of database recovery, database concurrency, distributed databases, data warehouse and data mining.

They should be able to design database for an application using Enhanced Entity Relationship (EER) model and Object Database Design using UML.

2- Intended Learning Outcomes (ILOS)

a- Knowledge and Understanding:

On successful completion of the course, the students should demonstrate knowledge and sound understanding of:

a1- Advanced principles of database systems including database design using EER and UML class diagram, database recovery and concurrency, distributed databases, and data warehouse and data mining (A1, A13, A14).

a2- Techniques and methods of Object oriented database design to meet the special needs and semantics of real world applications (A1, A4, A5, A18, A20)

- a3-** Build of data warehouse and data mining applications using commercial software tools (A2, A3, A19).

b- Intellectual Skills:

On successful completion of the course, the students should be able to:

- b1-** Realize the principles of recovery and concurrency, distributed database, data warehouse and data mining to solve real world semantic and decision making problems (B1, B2).
- b2-** Analyze and compare different techniques and methods of object oriented database design to meet the special needs of real world semantics (B11, B14).
- b3-** Design, implement and test complete database software system for a specified application domain. (B14, B17, B18)

c –Professional and Practical Skills:

On successful completion of the course, the students should be able to:

- c1-**Deploy database design tools, and commercial database facilities to improve the design process of database systems (C1, C2, C3, C9, C19).
- c2-** Implement object oriented/decision making applications using packages of commercial database (e.g. SQL server) and data mining (e.g. WEKA) (C1, C2, C6, C14, C15).
- c3-** Test and maintain database system applications (C21).

d- General and Transferable Skills:

On successful completion of the course, the students should be able to:

- d1-** Work effectively alone and as a member of development team and involve in group discussions and in seminars (D1, D2)
- d2-** Communicate effectively and present data results orally and in written form (D6)
- d3-** Search for information in references and in internet (D3, D7).
- d4-** Practice self-learning (D7).

Course Contribution in the program ILO'S

ILO'S		program ILO'S
A	Knowledge and understanding	A1, A2, A3,A4, A5,A13, A14, A18, A19,A20
B	Intellectual skills	B1, B2, B11, B14, B17, B18
C	Professional and practical skills	C1,C2, C3, C6,C9,C14, C15, C19,C21
D	General and transferable skills	D1, D2, D3,D6, D7

3- Contents

Topic	Lecture	Tutorial/ Practical
database design using Enhanced EER modeling,	4	4
Object Database Design using UML	4	4
Database Recovery	4	2
database concurrency	4	4
distributed database management	4	2
data warehouse	4	4
data mining.	4	4
Implementation of database applications using current DBMS packages	2	6
Total	30	30

Teaching and Learning and Assessment methods:

Course IL O'S		Teaching Methods						Learning Methods		Assessment Method				
		Lecture	Presentations and Videos	Discussions & Activities	Tutorials	Problem solving	Laboratory & Experiments	Researching and Reports	Modeling and Simulation	Written Exam	Practical Exam	Quizzes	Term papers	Assignments
Knowledge	a1	√	√	√	√		√	√		√	√	√	√	√
	a2	√			√					√		√	√	√
	a3	√			√		√	√		√		√	√	√
Intellectual	b1	√			√					√		√	√	√
	b2	√			√									
	b3		√				√		√	√				√
Applied	c1	√		√		√	√			√	√	√	√	√
	c2	√	√					√		√				
	c3	√			√			√		√				
General	d1			√		√		√					√	
	d2		√					√					√	
	d3	√	√					√						√
	d4	√						√	√				√	

5- Assessment Timing and Grading:

Assessment method	Timing	Grade (Decreases)
Semester Work : seminars, quizzes assignment and reports	Bi-Weekly	20%
Mid-Term Exam	7-th Week	20%
Practical Exam	15-th Week	10%
Written Exam	16-th Week	50%
Total		100%

6- List of references

6.1 – Text Books:

- Elmasri. R., Navathe, SB. (2011) *Fundamentals of database systems*. Addison Wesley. 6th Edition.

6.2 - Course notes:

- Lecture power point slides and handed to students.

6.3- Required Books:

- C.J Date. *An Introduction to Database Systems*. Pearson Education Inc. 8th edition, 2004.
- Silberschatz, A., Korth, H.F. and Sudarshan, S. *Database System Concepts*. 5th Ed. McGraw Hill book Co.

6.4- Websites:

- <https://www.pearsonhighered.com/sign-in.html?null>

7- Facilities required for teaching and learning:

7.1 – White board and markers.

7.2 – Overhead projector (transparencies) .

7.3 – Data show.

Course Coordinator: Prof. Dr. Mohamed Anwar Assal

Head of Department: Prof. Dr. Hafez M.S Abdel-Wahab

Table of Contents

Chapter 1 Database Fundamentals	1
1.1 Database System.....	1
1.1.1 Databases and Database Users	1
1.1.2 Database Versus Traditional File Processing	1
1.2 Data Models and Database Design.....	2
1.3 Data Modeling Using the Entity-Relationship Model	5
1.3.1 Introduction.....	5
1.3.2 ER Model Concepts:.....	6
1.3.3 Implementation Models "Relational Model".....	18
1.3.4 ER to Relational Mapping	19
1.3.5 Relational Integrity Constrains.....	21
Exercises	22
Chapter 2 Normalization for Relational Databases.....	27
2.1 Normalization Theory	27
2.1.1 Normal forms (NF).....	27
2.1.2 The First Normal Form Relations.....	27
2.1.3 The Second Normal Form Relations	29
2.1.4 The Third Normal Form Relations	29
2.1.5 The Boyce-Codd Normal form (BCNF).....	32
Exercises	34
Chapter 3 The Enhanced Entity Relationship Model EER	35
3.1 Object Oriented Concepts.....	35
3.2 Specialization Concept	36
3.2.1 Constraint and Characteristics of specialization:	37
3.2.2 Specialization Hierarchies and Lattices.....	39
3.3 The concept of Category	42
3.4 EER-to-Relational Mapping.....	45
3.5 Use of UML Diagrams as an Aid to Database Design Specification	50
3.5.1 UML as a Design Specification Standard	50
3.5.2 UML for Database Application Design.....	50
3.5.3 Different Types of Diagrams in UML	51
3.5.4 A Modelling and Design Example: UNIVERSITY Database	54
Exercises	55
Chapter 4 An Introduction to SQL	56
4.1 Data Definition in SQL	57
4.1.1 CREATE TABLE in SQL	57
4.1.2 DROP SCHEMA and DROP TABLE Commands	60
4.1.3 The ALTER TABLE Command.....	60
4.2 Queries in SQL (SELECT Statement).....	61
4.2.1 Basic SQL Queries	61
4.2.2 Dealing with Ambiguous Attribute Names	63

4.2.3	Unspecified WHERE Clauses and Use of '*'	64
4.2.4	Tables as Sets in SQL	64
4.2.5	Set Operations.....	65
4.2.6	Nested Queries and Set Comparisons.....	66
4.2.7	The EXISTS functions in SQL	68
4.2.8	The Explicit Sets and NULLS in SQL	69
4.2.9	Aggregate Functions and Grouping in SQL.....	69
4.2.10	Sub-string Comparisons, Arithmetic Operators, and Ordering.....	73
4.3	Data Manipulation Language (DML).....	74
4.3.1	The INSERT Command	74
4.3.2	The DELETE Command	75
4.3.3	The UPDATE Command.....	76
4.4	Data Transaction Language (DTL)	77
	Exercises	78
Chapter 5	Complex Queries & Active Databases	79
5.1	SQL Implementations.....	79
5.1.1	Difference between SQL and T-SQL	79
5.1.2	Difference Between T-SQL and PL-SQL	80
5.1.3	Varchar vs nVarchar	80
5.2	Joined Tables in SQL	81
5.3	More Complex SQL Retrieval Queries	83
5.3.1	Nested Queries, Tuples, and Set/Multiset Comparisons	83
5.3.2	The EXISTS and UNIQUE Functions in SQL.....	84
5.3.3	More Subqueries.....	85
5.3.4	T-SQL Well-known String Functions	87
5.4	Views in SQL	87
5.4.1	Specification of Views in SQL	88
5.4.2	Remarks on Updating Views.....	89
5.5	Specifying Indexes in SQL.....	90
5.6	Specifying Constraints in SQL.....	91
5.7	Specifying Constraints as Assertions	92
5.8	Stored Procedures.....	93
5.9	SQL/PSM: Extending SQL	95
5.10	Triggers in SQL.....	97
5.10.1	Why Use Triggers?.....	98
5.10.2	Trigger Syntax	99
5.10.3	Trigger Examples.....	99
	Exercises	107
Chapter 6	Database Security.....	108
6.1	Types of Security.....	108
6.1.1	Threats to Databases	108
6.1.2	Control Measures.....	109
6.1.3	Database Security and the DBA	110
6.1.4	Access Control, User Accounts, and Database Audits.....	110
6.2	Discretionary Access Control.....	111
6.2.1	Privileges, Roles, Schema and Database Users:.....	111
6.2.2	Granting and Revoking of Privileges	112
6.2.3	Granting Object Privileges	114



Exercises	115
Chapter 7 Database Recovery & Concurrency	116
7.1 Transaction Processing Systems	116
7.1.1 Single-User versus Multi-User Systems.....	116
7.1.2 Basic Database Access Operations.....	117
7.1.3 Why Concurrency Control is Needed?	117
7.1.4 Why Recovery Is Needed?	118
7.1.5 Types of Failures:	119
7.1.6 Transaction States & Operations	119
7.1.7 The System Log.....	120
7.2 Characterizing Schedules Based on Recoverability.....	123
7.3 Characterizing Schedules Based ON Serializability	124
7.3.1 Conflict – Serializable Schedules	125
7.3.2 Algorithm for testing conflict Serializability of schedules:	126
7.4 Recovery Concepts.....	128
7.4.1 Categorization of recovery algorithms:	128
7.4.2 Caching (Buffering) of disk blocks	129
7.4.3 Transaction rollback	130
7.4.4 Checkpoint.....	132
7.5 The ARIES Recovery Algorithm	132
Exercises	135
Chapter 8 Distributed Databases.....	136
8.1 Reasons for Distribution and DDBMS functions:.....	136
8.1.1 The advantage of DDBSs as follows:.....	136
8.1.2 Architecture of a DDES.....	137
8.1.3 Application Processors and Data Processors.....	137
8.2 Data Fragmentation, Replication, and Allocation Techniques	138
8.2.1 Data Fragmentation	138
8.2.2 Data Replication and Allocation.....	139
8.3 Types of Distributed Database System	142
8.4 Distributed Recovery.....	143
Exercises	144
Chapter 9 Data warehouse & Data Mining	145
9.1 What is a data warehouse?	145
9.1.1 Data Warehouse.....	145
9.2 OLTP & OLAP.....	146
9.3 Data Warehouse.....	148
9.3.1 OLAP Operations	150
9.4 Multiple Dimension Data Model	153
9.4.1 Schemas for multidimensional Databases.	157
Exercises	163
References	164

Chapter 1 Database Fundamentals

1.1 Database System

1.1.1 Databases and Database Users

A Database:

It is a collection of related data. By data, we mean known facts that can be recorded and that have implicit meaning (e. g the names, telephone numbers, and addresses of all the people you know).

A database has the following implicit properties:

- A database is a logically coherent collection of data with inherent meaning.
- A database is designed, built, and populated with data for a specific purpose. It has an intended group of users and some preconceived applications, which these users are interested.
- A database represents some aspects of real world, sometimes called the mini world. Changes to the mini world are reflected in the database.

A Database Management System (DBMS):

The **DBMS** is a collection of programs that enables users to create and maintain a database. The **DBMS** is a general-purpose s/w system that facilitates the following processes for various applications:

- Defining a database involves specifying the types of data to be stored in the database, along with a detailed description of each type of data.
- Constructing the database is the process of storing the data itself on some storage medium that is controlled by DBMS.
- Manipulating a database include such functions as querying the database to retrieve specific data, updating the database to reflect changes in the mini world, and generating reports from the data.

1.1.2 Database Versus Traditional File Processing

The database approach has following advantages:

1. Self-contained Nature of a Database System:

A fundamental characteristic of the database approach is that the database system contains not only the database itself but also a complete definition or description of the database. This definition is stored in the system Catalog. The information stored in the Catalog is called meta-data. The Catalog is used by DBMS s/w and occasionally by the database users who need information about the database structure.



The DBMS s/w is not written for any specific database application and hence must refer to the catalog to know the structure of a specific database. In traditional file processing approach programs are constrained to work with only one specific task but DBMS can access many different databases by extracting the database definitions from the catalog.

2. Insulation between Programs and Data:

In traditional file processing, the structure of the data files is embedded in the access programs, so any changes to the structure of a file may require changing all programs that access this file. By contrast, the DBMS access programs are written independently of any specific data files.

3. Data Abstraction:

A DBMS provides users with a conceptual representation of data that does not include many of the details of how the data is stored. A Data Model is a type of data abstraction that is used to provide this conceptual representation. Data model uses logical concepts, such as objects, their properties, and their interrelations, which may be easier for most users to understand than computer storage concepts (i.e. data models hides storage details that may not be of interest to most database users).

4. Support of Multiple Views of the Data:

A database typically has many users, each of whom may require a different perspective or views of the database which are easily supported by a multi-user DBMS.

1.2 Data Models and Database Design

A Data Model is a type of data abstraction that can be used to provide a conceptual representation for the data.

The database design has three levels (models):

1. High Level data model (external “conceptual” schema):

It is a concise description of the data requirements of the users and includes detailed descriptions of the data types, relationships, and constraints using the concepts provided by the model (entities, attributes, and relationships). Because these concepts do not include any implementation details, they are usually easier to understand and can be used to communicate with non-technical users since it expresses all users' data requirements.

2. Implementation data model:

It represents the actual implementation of the database using a commercial DBMS. The current commercial DBMS are relational, network and hierarchical. A data model-mapping step is needed to transform the conceptual schema from the high-level data model representation into the implementation data model.



3. Physical data model (Internal schema):

It is a low-level data model, which describes the physical storage structure of the database. This model describes the complete details of the data storage and the access paths for the database.

Figure (1.1) shows a simplified database system environment & **Figure (1.2)** shows an example of a database that stores student records and their grades.

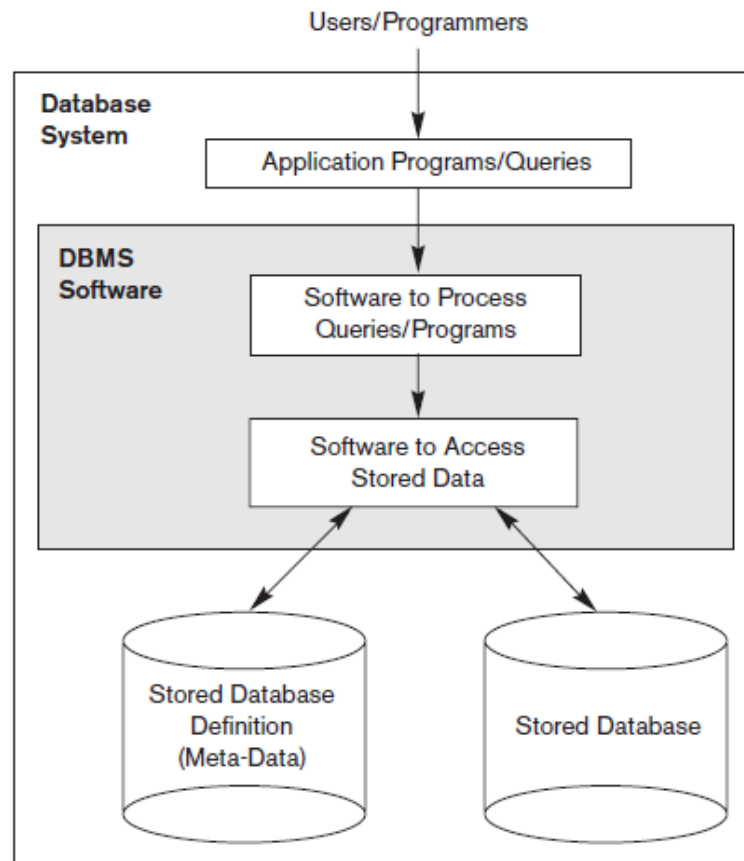


Figure 1.1A Simplified Database



STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2 A database that stores student and course information.



1.3 Data Modeling Using the Entity-Relationship Model

1.3.1 Introduction

The simplified description of the database design process is shown in the following figure:

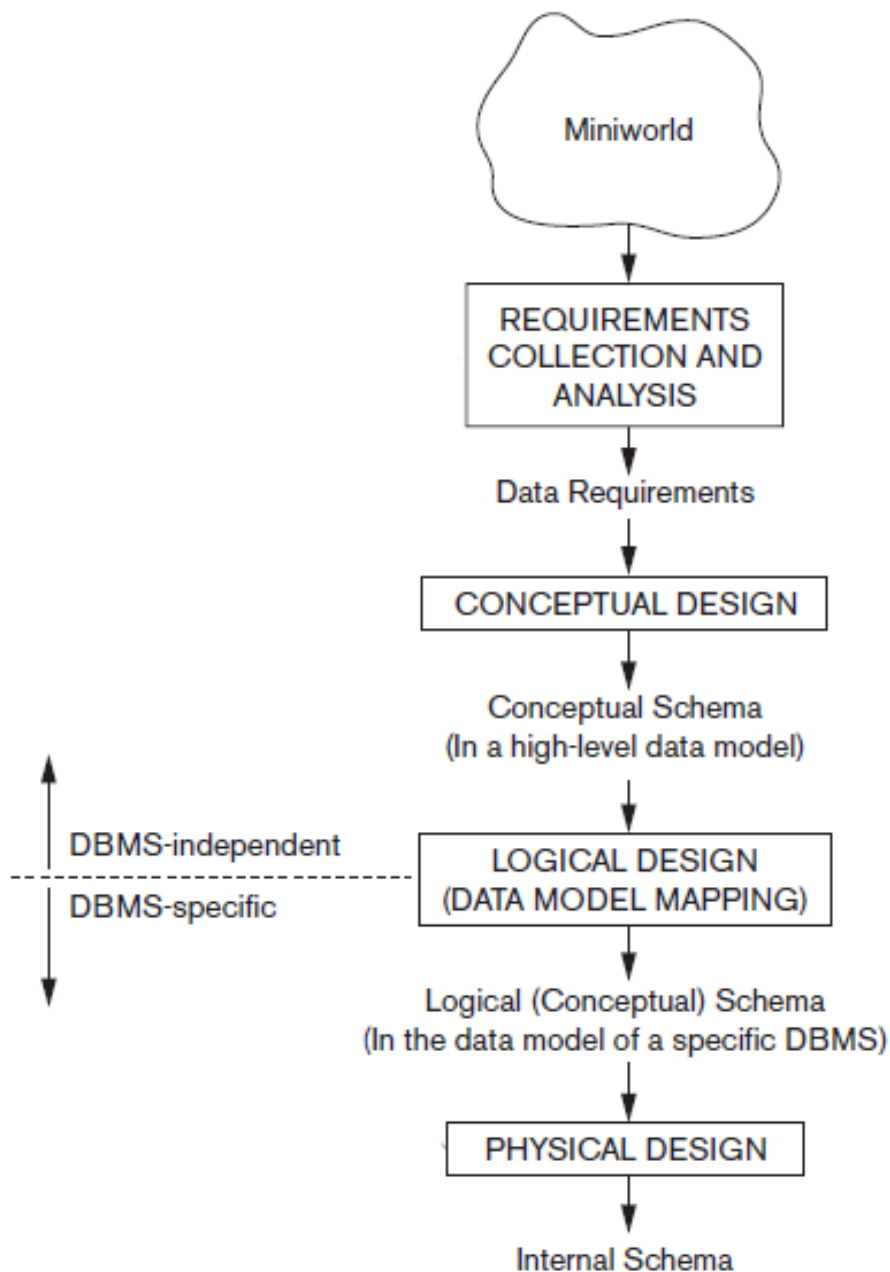


Figure 1.3 The database design process



The database design process can be explained through a sequence of steps as follows:

1. Requirement collection and analysis.

During this step, the database designers interview prospective database users to understand and document their requirements. These requirements should be abstracted, after removing any redundancy, from the point of view of the database designers and outcome of this step is the database requirements.

2. The creation of conceptual schema for the database.

The conceptual schema is a concise description of the database requirement of the users and includes detailed descriptions of the database, relationships, and constraints.

The result of this step is the representation of the database using a high-level conceptual schema. This high-level data model is called the Entity Relationship model (ER-model) or the database External schema.

3. The actual Implementation of the database using a commercial DBMS.

In this step, an implementation data model is to be built for the database using certain available DBMS.

In this step the high-level data model is transformed to an implementation data model. This transformation is performed using a mapping procedure which depends on the chosen DBMS to build the database system. Most current commercial DBMSs are called Relational DBMSs or RDBMSs. (The other types of the DBMSs are Network DBMSs and hierarchical DBMSs). So, the result of this step is the building of the database implementation model.

4. The physical database design phase.

The objective of this step is the choice of the internal storage structures and file organizations for the database system. A specific database design process is restricted to choose the most appropriate structures for the database files from among the options offered by the DBMS in order to achieve the following:

- Suitable response time
- Efficient space utilization

1.3.2 ER Model Concepts:

We now present the concepts of ER Model. There are three main tools to build an ER Model of database system:

- Entity/ Entity Type
- Attribute
- Relationships



Entity

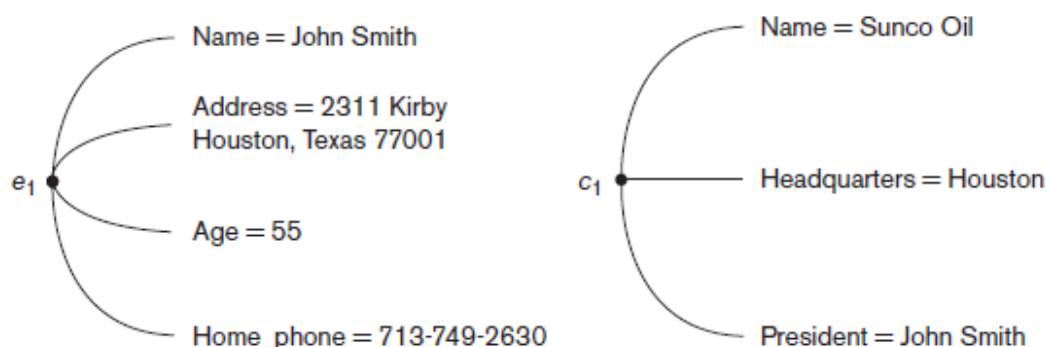
It is a thing in the real world with an independent existence. The entity existence has two forms:

- Physical existence (a person, car, house, or employee).
- Conceptual existence (company, job, department, or project).

Attribute & Attribute Types:

Attribute: It is a particular property for an entity to describe that entity.

The following figure shows two entities and the value of their attributes to demonstrate the attribute definition:



Attribute types can be considered through three different points of view:

- 1 Simple (atomic) versus Composite
- 2 Single valued versus multi-valued
- 3 Stored versus Derived.

1 Simple versus composite attributes

- **Simple attribute:** It is an attribute that is not divisible into more basic components (basic attributes) e.g., Identification number or the sex of the person. This type of attribute is called also atomic attribute.
- **Composite Attribute:** It is an attribute that can be divided into a set of basic component attribute (Simple attributes). e.g., Name attribute which can be divided into first-name, middle-name, and last name. Also, the address attribute that can be divided into street address, city, and state.

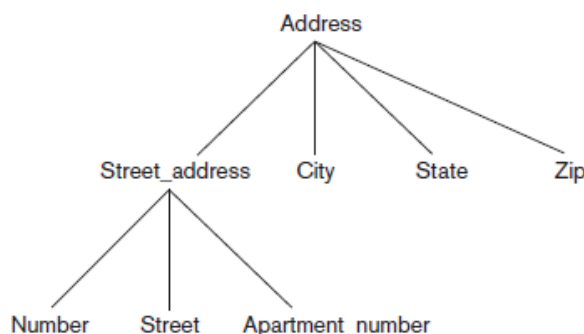


Figure 1.4 A hierarchy of composite attributes



Remark: In a particular database, if we are going always to refer to the composite attribute as a whole (i.e., as one unit), there is no need to subdivide it into component attributes.

2 Single valued versus multi-valued attribute

- **Single valued attribute:** It is an attribute which has a single value for a particular entity (name, birth date).
- **Multi-valued attribute:** It is an attribute which has a set of values for a particular entity (e.g., college degree attribute for a person or his hobby attribute).

3 Stored versus Derived attribute

- **Derived attribute:** It is an attribute which has a value for particular entity that can be calculated (derived) from the value of another attribute for the same entity or a related entity.
- **Stored attribute:** attribute values that cannot be derived.

For Example:

- Age attribute can be derived from the birthdate, so age attribute is called derived attribute and birth date is called a stored attribute.
- An attribute "number-of-employees" of department entity can be calculated by counting the number of employees working for that department (two related entities: employees, department).

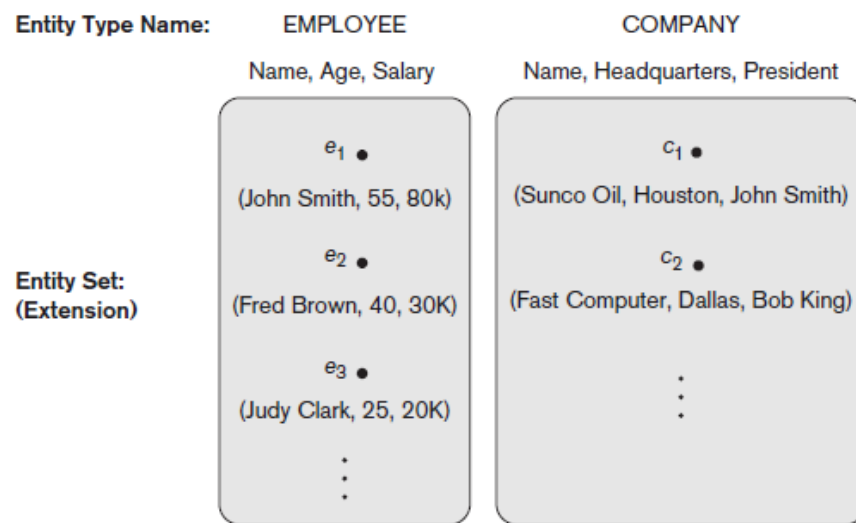
Remarks:

- An attribute of certain entity has a value called a NULL value i.e., this attribute has not any applicable value for that entity (e.g. the apartment number attribute of an address applies only to addresses that are in apartment buildings and no other types of residences such as single-family homes).
- There are set of related definitions to both the entities and attributes:



Entity Type

Group of entities, which are described by the same set of attributes, defines an entity type. The following figure shows two entity types and some of their member entities:



Entity Type Features:

The entity type has the following features:

- The entity type description which is called entity type schema (structure). The entity type schema specifies a common structure shared by individual entities type schema (intension) should not be changed.
- The extension of an entity type is the set of individual entity instances at a particular moment in time (extension may be changed often by adding, deleting, or updating an entity of that entity type).

Key Attribute of an Entity Type:

It is the attribute of an entity type, which has a value that is distinct for each individual entity of that entity type (e.g., the ID number of a person).

Sometimes, several attributes together may form the key attribute (e.g., car number, city code number) i.e., the combination of the values of a set of attributes can only be distinct for each individual entity.

Value Set:

It is the range of values (domain of values) that can be assigned to an attribute of an entity type (e.g., age attribute of an employee in a company ranges only from 16 to 50 years in a database).



Relationships

Before going any further, we should discuss the following example:

The **COMPANY DATABASE** keeps track of a company's employees, departments, and projects. Suppose that after the requirements collection and analysis phase, the database designers stated the following description of the "mini-world": the part of the company to be represented in the database:

- The company is organized into departments. Each department has a name, a number, and an employee who manages the department. We keep track of the start date when that employee started managing the department. A department may have several locations.
- A department controls several projects, each of which has a name, a number, and a single location.
- We store each employee's name, social security number, address, salary, sex, and birth date. An employee is assigned to one department but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee.
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's name, sex, birth date, and relationship to the employee.

The preliminary design of entity types (Rough Schema) for that example is as follows:

DEPARTMENT

Name, Number, {locations}, Manager, ManagerStartDate

PROJECT

Name, Number, Locations, ControllingDepartment

EMPLOYEE

Name (FName, MName, LName), SSN, Sex, Address, Salary, Birthdate, Department, Supervisor, {WorksOn (Project, Hour)}

DEPENDENT

Employee, DependentName, Sex, Birthdate, Relationship

Remarks:

- In the previous rough schema for a company database, the fact that an employee can work in several projects, is represented as a multi-valued composite attribute of employee entity type called Works-On with simple component attributes (project, hours).
- Alternatively, we can represent the previously mentioned fact, as a multi-valued composite attribute for the entity type project called Workers with simple component (employee, hour).



In fact, whenever an attribute of one entity type refers to another entity types, a relationship exists. In the previous example we can detect 6 relationships:

- The attribute Manager of a department entity type refers to the employee who manages that department.
- The attribute Controlling-Department of project entity type refers to the department responsible about that project.
- The attribute Supervisor of the entity type employee refers to an employee of the same entity type (so we have here a relation called recursive).
- The attribute Works-On of the entity type employee refers the projects in which an employee Works-On.
- The attribute Department of the entity type employee refers to the department in which the employee works.
- The attribute Employee in the entity type dependent refers to the employee entity type to determine the information about employee who has a relationship with that dependent.

- **Relationship Type:**

Relationship type R among n entity types $E_1, E_2, \dots, E_n$ is a set of associations among entities from these types. In formally, each relationship instance r_i in R is an association of entities, where the association includes exactly one entity from each participating entity types. Such relationship instance represents the fact that entities participating in are related together in some way in the corresponding mini-world situation.

- **Degree of a Relationship Type:**

The degree of a relationship type is the number of participating entity type in that relationship (e.g., the works for relationship type is of degree two). A relationship type of degree two is called binary and of degree three is called ternary relationship.

In general, a ternary relationship type represents more information than three binary relationship types (see the following figure).



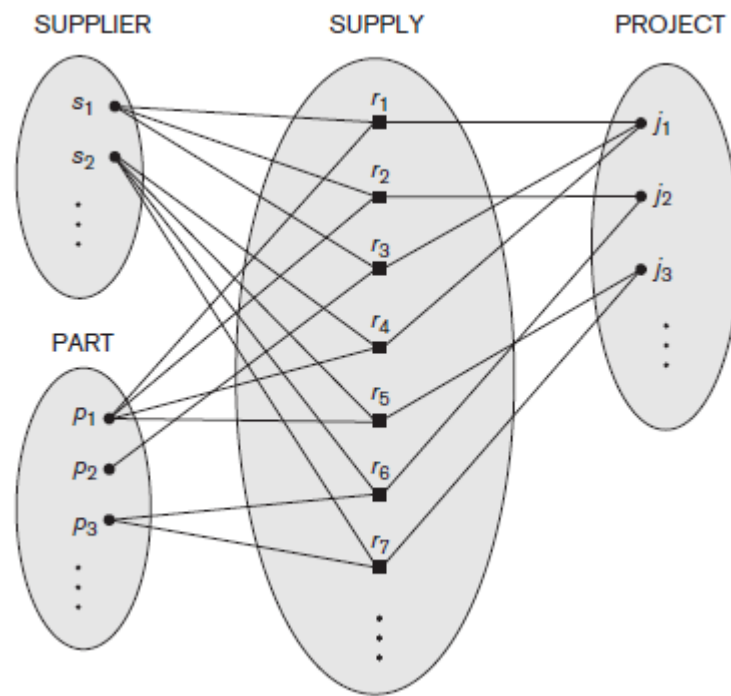


Figure 1.5 SUPPLY ternary relationship

For example, consider the three binary relationship types *CAN_SUPPLY*, *USES*, and *SUPPLIES*. Suppose that *CAN_SUPPLY*, between *SUPPLIER* and *PART*, includes an instance (s, p) whenever supplier s can supply part p (to any project); *USES*, between *PROJECT* and *PART*, includes an instance (j, p) whenever project j uses part p; and *SUPPLIES*, between *SUPPLIER* and *PROJECT*, includes an instance (s, j) whenever supplier s supplies some part to project j.

The existence of three relationship instance (s, p), (j, p), and (s, j) in *CAN-SUPPLY*, *USES*, and *SUPPLIES*, respectively, does not necessarily imply that an instance (s, j, p) exists in the ternary relationship *SUPPLY*. This has been called the **connection trap**.

- **Role Names & Recursive Relationships:**

Each entity type that participates in a relationship type plays a particular role in the relationship. The role name signifies the role that a participating entity from the entity type plays in each relationship instance.

For example, in the *Works_For* relationship type, the employee plays the role of worker and department plays the role of employer.

In some cases, the same entity type participates more than once in a relationship type (Recursive Relationship). The figure of the supervision relationship type relates an employee to a supervisor where both employee and supervisor entities are members of the same entity type (employee entity type) which plays the following 2 roles:

- a. Role of supervisor
- b. Role of supervisee



- **Constraints on Relationship Type:**

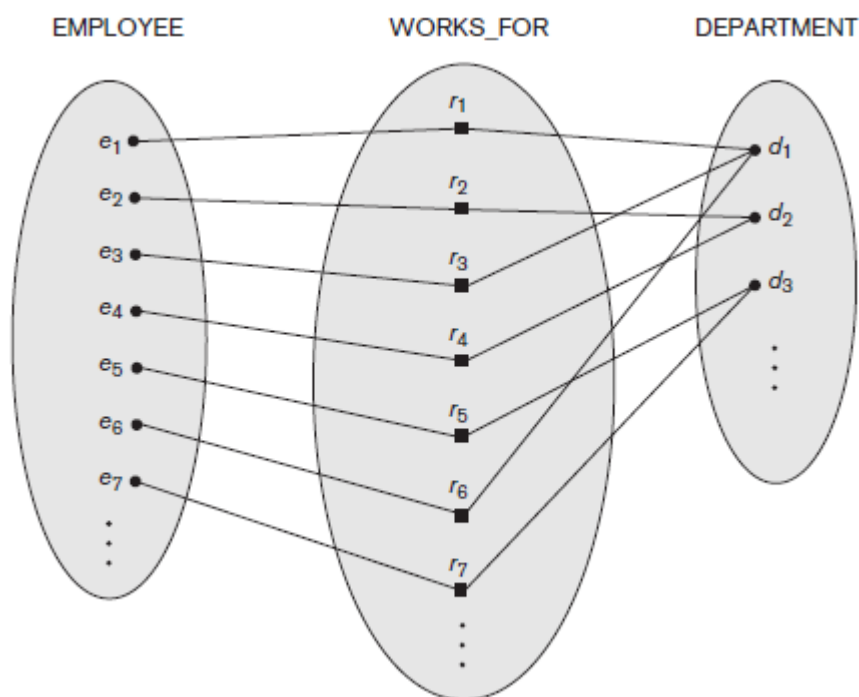
The constraints are determined from the mini-world situation that the relationships represent. The two main types of relationship constraints are:

- a. Cardinality ratio
- b. Participation

a. The cardinality ratio constraint

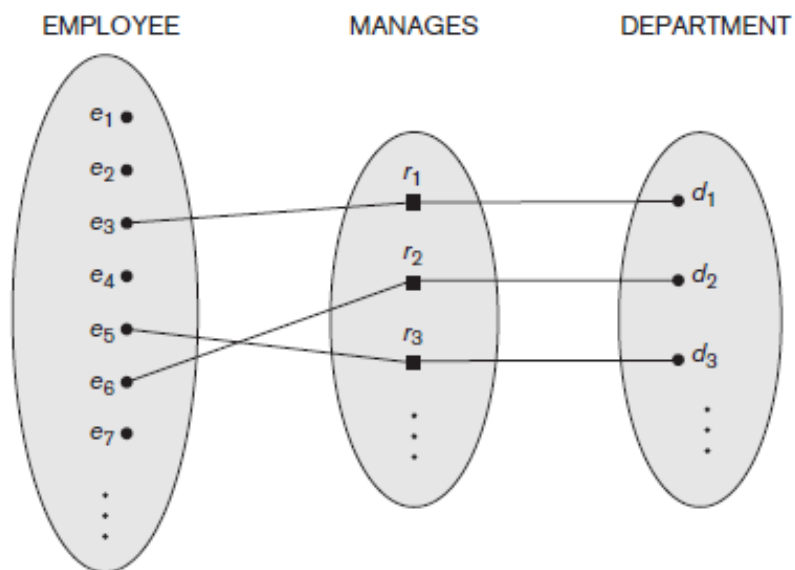
It specifies the number of relationship instance that an entity can participate in:

- **Works-For** relationship (department-employee) is **1:N** as shown in the following figure:

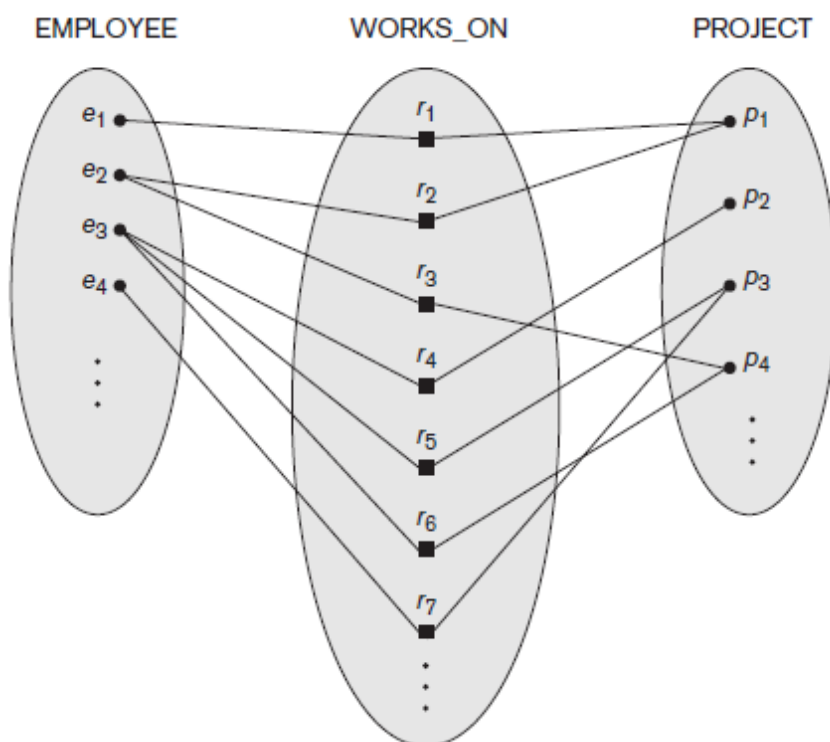


- **Manages** relationship (employee-department) is **1:1** as shown in the following figure:

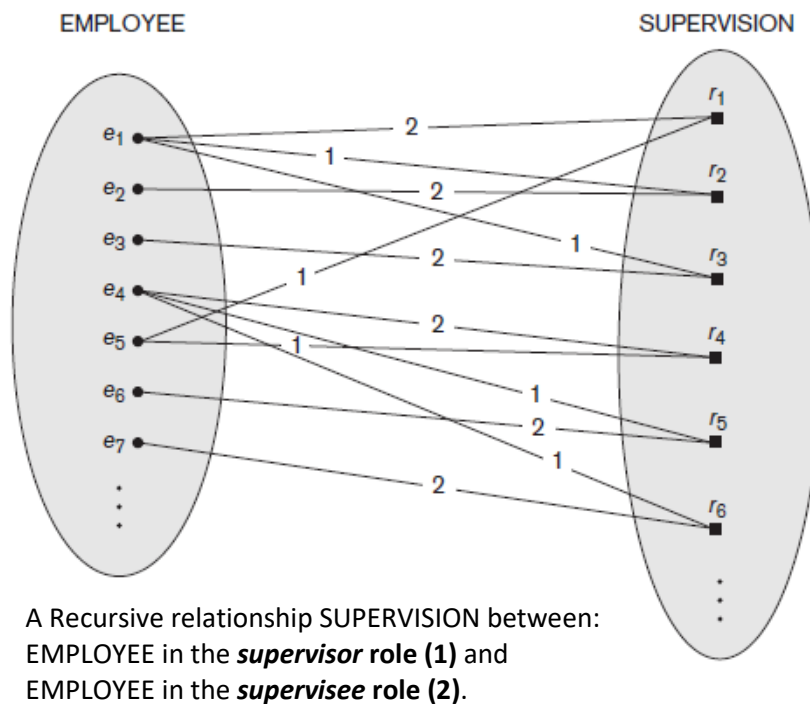




- **Works-ON** relationship (employee-project) is **M:N** as shown in the following figure:



- **Supervision** relationship is (supervisor [employee], supervisee [employee]) is 1:N.



Similarly:

- **Controls** relationship (department, project) is 1:N.
- **Dependents-Of** relationship is 1:N.

b. The Participation constraint

It specifies whether the existence of an entity depends on its being related to another entity via the relationship type. There are two types of participation:

- Total participation (existence dependency)
- Partial participation

Examples:

- In the **Works-For** relationship, the participation of the employee is a total participation because any employee entity must be related to a department (no employee exists without being related to department). Also, department has a total participation **Works-For** relationship.
- In the **Manages** relationship, we do not expect that every employee to manage a department, so it is a partial participation. Department has a total participation in the **Manages** relationship.

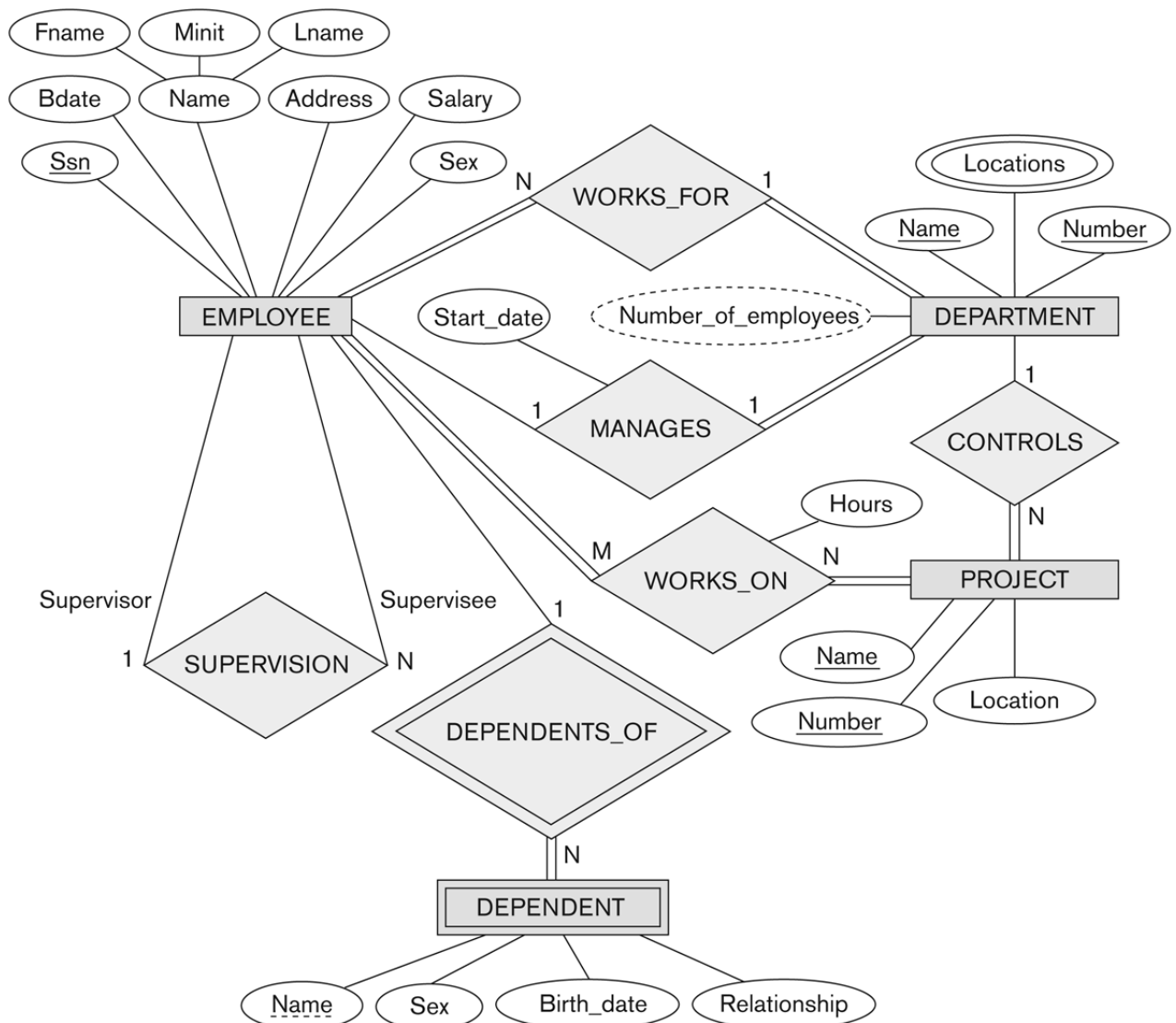


Remark: Here, we can define the **Weak Entity Type**:

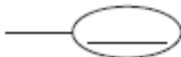
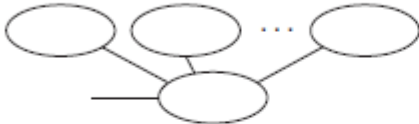
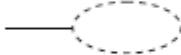
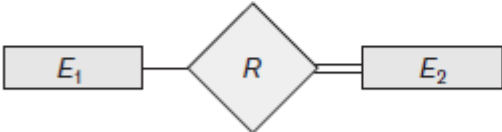
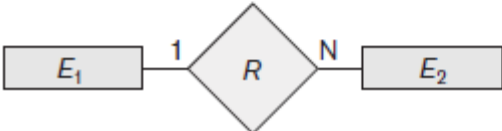
Some entity types may not have any key attributes of their own. This implies that we are not able to distinguish between some entities because the combinations of values of their attributes can be identical. Such entity types are called **weak entity types**.

Entities belonging to a weak entity type are identified by being related to specific entities from another entity type in combination with some of their attribute values. We call this other entity type the identifying owner and call the relationship type, that relates a weak entity type to its owner, the identifying relationship. A weak entity types always have a total participation (existence dependent) respect to its identifying relationship because a weak entity cannot be identified without an owner entity.

From the previous discussion, The Entity Relationship Model of the Company Database is drawn in the following figure:



The summary of ER diagram notations:

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for $E_1 : E_2$ in R



1.3.3 Implementation Models "Relational Model"

Relational Model introduced by Codd in 1970, which is a conceptual data model. This model represents the data in a database as a collection of relations. Each relation is represented by a table. The rows of this tables are called **n-tuples** (instances), where n is the degree of the relation, and the columns are called attributes. These attributes must be atomic i.e., it cannot be further subdivided (not composite).

The no. of attributes determines the degree of the relation. Each table has a heading & body. the heading is the name of attributes, and the body is the set of tuples. **Properties of a Relation are:**

- We do not have duplicate tuples.
 - Attributes are unordered.
 - Tuples are unordered.
 - All attributes' values are atomic.
- **A relational database schema S** is a set of relations $\{R_1, R_2, \dots, R_n\}$ and a set of integrity constraints (IC) represented by the attribute types (included in the table heading).
 - **The degree of a relation** is the number of atomic attributes participating in that relation.
 - **Key Attributes** for a Relation:
 - Primary Key
 - Candidate Key
 - Foreign Key

- **Candidate Key:**

It is a key candidate to be primary key. Note that a relation is defined as a set of distinct tuples i.e. the attributes of a relation represent a composite key (primary key). A relation may have more than one candidate key we choose one that has small number of attributes.

One of the candidate keys is chosen to be primary key and the others are called alternate keys.

How to Choose a Candidate Key?

In a relation R $\{A_1, A_2, \dots, A_n\}$ relation of degree n the set of attributes $\{A_1, A_2, \dots, A_k\}$ is said to be a Candidate Key of a relation R if and only if they satisfy the following :

- **Uniqueness:** It means no tuples of R have the same value of $\{A_1, A_2, \dots, A_k\}$.
- **Minimality:** None of $\{A_1, A_2, \dots, A_k\}$ can be discarded from the key without destroying the uniqueness property.
- **Foreign Key:** In a relation R2 is any attributes (may be attribute combinations) whose values are required to match those of the primary key in another relation R1.



1.3.4 ER to Relational Mapping

Using the Entity-Relationship Model (ER), the production of Relational Schema (Implementation Model) can be performed through the execution of the following mapping rules:

STEP 1:

- For each regular entity type E in the ER, we create a relation R that includes all the single attributes of E. For a composite attribute we include only its simple attributes.
- We choose one of the key attributes of E as a primary key for R.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>
-------	----------------

STEP 2:

- For each weak entity type W with owner entity type E, we create a relation R & include all simple attributes (or simple components of composite attributes in W) as attributes of R.
- Include as a foreign key, the primary key attributes of the owner entity E.
- The primary key of R is the combination of the primary key of the owner & the partial key of the weak entity type.

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

STEP 3:

- For each binary **1:1** relationship type R in ER schema, we identify the relations S&T that corresponds to the entity types participating in R.
- We choose the relation S (S is chosen because it has a total participation in R) and include in it the primary key of T as foreign key.
- We include all the simple attributes and the simple components of any composite attributes, of the binary **1:1** relation R as attributes of S.

DEPARTMENT

FK

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------



STEP 4:

- For each binary **1:N** relation type R, we identify the relation S that represents the participating entity type at the N Side of the relationship R.
- We include as foreign key in S the primary key of T that represent the other entity type participating in R.
- We include all the simple attributes and the simple components of any composite attributes, of the binary **1:N** relation R as attributes of S.

EMPLOYEE								FK	
Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno

PROJECT			FK	
Pname	<u>Pnumber</u>	Plocation	Dnum	

STEP 5:

- For each binary **M:N** relationship type R, we create a new relation S to represent R. we include in it the primary keys of the participating entity types in the relation R as a foreign key.
- We include all the simple attributes and the simple components of any composite attributes, of the binary **M:N** relation R as attributes of S.

WORKS_ON		
<u>Essn</u>	<u>Pno</u>	Hours

STEP 6:

- For each multi-valued attribute, A, we create a new relationship R. The primary key attribute of the entity type K, that has A as an attribute, is included as foreign key in R.
- The primary key of R is then the combination of A & K. if the multi-valued is composite; we include its simple components.

DEPT_LOCATIONS	
<u>Dnumber</u>	<u>Dlocation</u>

STEP 7:

- For each ternary relationship R, we create a new relation S to represent R. We include (as **FKs**) the **PKs** of the relations, that represent the entities participating in R into S.
- We include all the simple attributes and the simple components of any composite attributes, of the ternary relation R as attributes of S.
- The **PK** of S is the combination of all **FK**



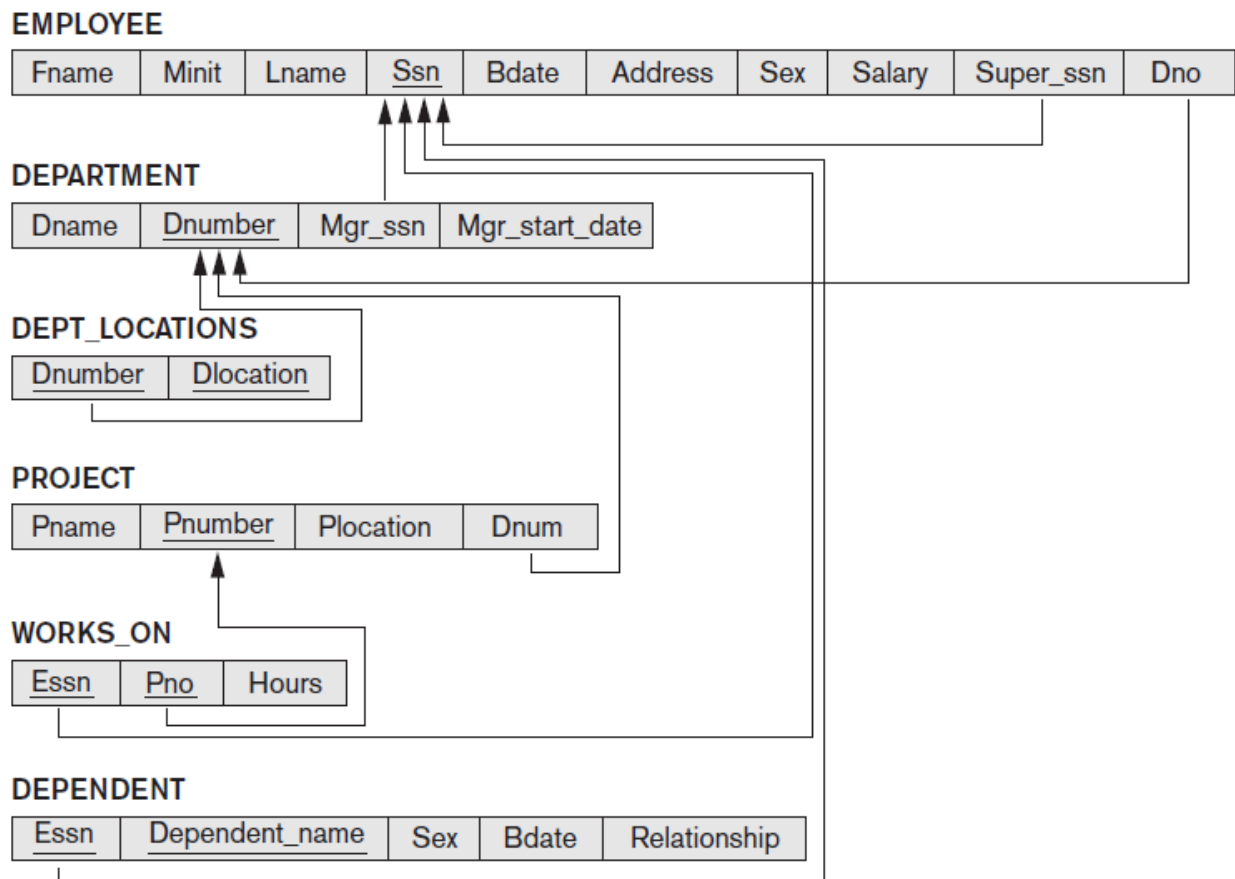


Figure 1.6 The COMPANY relational database schema

1.3.5 Relational Integrity Constrains

So, far, the draw of the relational DB schema, has been explained. Now, we fill this DB with the instances. These instances should satisfy some rules or constrains, called Integrity constrains., in order to ensure that the DB is correct and consistent.

These integrities constrain are:

- Specific: specified for a specific DB.
- General: They are constrains, that are applicable for any relational DB.

General Constrains:

- **Entity Integrity Constrains:** Specifies that no attributes participating in a PK of a relation R can accept a NULL value.
- **Referential Integrity Constrains:** In a relation R2, If the FK is matching the PK of a relation R1, then every value of FK is either:
 - a. be equal to the value of the PK in some tuple of R1 or,
 - b. Wholly NULL (i.e., ever attribute participating in FK is a NULL)

These conditions are applied to only those relations of partial participations. But in total participation between two entity types, the NULL is not allowed.



Exercises

Design and draw the appropriate ER schema diagram for the given data requirements. Specify key attributes of each entity type and the structural constraints on each relationship type. Give the relational database schema that corresponds to the given data requirements. Specify all keys, entity, and referential constraints.

Student Name:

Student ID:

Exercise 1

A database is decided for a large construction enterprise to support its resources management. Such an enterprise has a set of Departments and Projects. Each department has a set of Equipment. Equipment's are assigned to projects for a specific period of time.

- No equipment is assigned to more than one project at a time.
- An equipment could be idle for a given period of time.
- The database is to contain the following information:
 - Department data together, with data concerning the equipment of each department.
 - Project's data together with necessary data of all equipment assigned to each project (start and finish dates of assigning an equipment to a project).
 - For each department: department number (unique), name (unique), and budget.
 - For each equipment: equipment number (unique), name, store location.
 - For each project: project number (unique), name, location, project start date and project finish date.



Student Name:

Student ID:

Exercise 2

A database is to contain information concerning sales representatives, sales areas, and products. Each representative is responsible for sales in only one area. Each area may have one or more responsible representatives. Each representative is responsible for sales of one or more products, and each product has one or more responsible representatives. A sales commission is assigned to each representative. The commission rate is determined according to the sales area, in other words, for each area there is a certain fixed commission rate for all responsible representatives of that area. The database is to contain the following information:

- For each representative: representative number (unique), representative name (unique), and sales commission.
- For each sales area: area number (unique), and location.
- For each product: product number (unique), product name and quantity sold by each representative in each area.



Student Name:

Student ID:

Exercise 3

A database is decided for scheduling the coming football tournament of World Cup in France. The participating teams are divided into groups each of 8 teams. The designed database must contain data about:

- Teams described by country, coach, and the score of these team in each match.
- Groups described by group number, teams in that groups and the group supervisor name.
- Matches described by the teams, referee name, court number, court city and date.
- Players described by teams, player number, player name, player position in the team, number of goals for each player in each match.



Student Name:

Student ID:

Exercise 4

A database is to contain information about the internal education system of a large industrial company. The company in question maintains an education department whose function is to run several training courses for the employees of the company; each course is offered at several different locations within the organization, and the database contains details both of offerings already given and of offerings scheduled to be given in future. The database contains the following information:

- For each course: course number (unique), course title, details of all immediate prerequisite course, and details of all offerings.
- For each offering, of a given course; offering number, date, location, details of the teacher, and details of all students.
- For each teacher of a given offering; employee number (unique), and name.
- For each student of a given offering; employee number (unique), name, and grade.

The following constraints apply:

- For each course, all offerings of that course are taught by only one teacher.
- A teacher may teach more than one course type.



Student Name:

Student ID:

Exercise 5

Consider the following set of requirements for EGYPTAIR database which contains information about the airline flight system of that national company:

- Each FLIGHT is identified by a flight number, weekdays, and date. Each flight consists of one or more flight legs. A flight leg is a nonstop portion of a flight.
- Each FLIGHT LEG has number (1, 2, 3, etc.), departure airport, scheduled departure time (day-hour-min), arrival airport, scheduled arrival time (day-hour-min), and airplane. The leg number is unique for flight legs of the same flight.
- Each AIRPORT has code (unique), name, and city.
- Each AIRPLANE TYPE has type name (unique), company, and maximum number of seats. Each airplane type can land only on certain airports. There is a set of airplanes for each airplane type.
- Each AIRPLANE has ID number (unique for airplanes of the same type), and technical status (in service, out of service, in maintenance, ... etc.).



Chapter 2 Normalization for Relational Databases

2.1 Normalization Theory

Normalization theory is a set of Concepts and rules that can help in the process of data base design, to ensure the correctness of that design.

2.1.1 Normal forms (NF)

A relation is said to be in a normal form if and only if it satisfies a specific set of constraints:

- Normalized Relation in the (1st NF).
- Normalized Relation in the (2nd NF).
- Normalized Relation in the (3rd NF).
- Boyce-Codd Normal form (BCNF).

2.1.2 The First Normal Form Relations

First normal form (1NF) states that a relation is said to be in the 1st if and only if all attribute values are atomic.

In other words, **1NF** disallows relations within relations or relations as attribute values within tuples. The only attribute values permitted by **1NF** are single atomic (or indivisible) values.

Full Function Dependence (FFD):

An attribute Y is Full Function Dependent on attribute X if and only if Y is dependent on X and not dependent on any proper subset of X.

Example:

If (SSN, PNO) -----> Hours **and**

SSN -----> Hours

then Hours is not FFD on (SSN, PNO) because Hours dependent on SSN which is a subset of (SSN, PNO).

Consider the DEPARTMENT relation schema shown in Figure 14.1, whose primary key is *Dnumber*, and suppose that we extend it by including the *Dlocations* attribute as shown in Figure 2.1 (a). We assume that each department can have a number of locations. The DEPARTMENT schema and a sample relation state are shown in Figure 14.9.

As we can see, this is not in **1NF** because *Dlocations* is not an atomic attribute, as illustrated by the first tuple in Figure 2.1 (b). There are two ways we can look at the *Dlocations* attribute:



- The domain of *Dlocations* contains atomic values, but some tuples can have a Set of these values. In this case, *Dlocations* is not functionally dependent on the primary key *Dnumber*.
- The domain of *Dlocations* contains sets of values and hence is non-atomic. In this case, $Dnumber \rightarrow Dlocations$ because each set is considered a single member of the attribute domain.

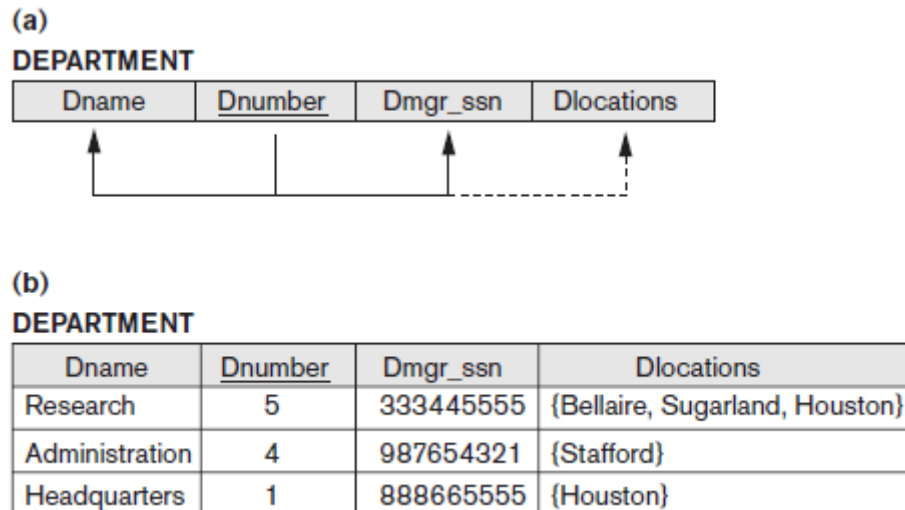


Figure 2.1 DEPARTMENT schema

In either case, the DEPARTMENT relation in Figure 2.1 is not in **1NF**; in fact, it does not even qualify as a relation.

There are three main techniques to achieve first normal form for such a relation:

1. Remove the attribute *Dlocations* that violates **1NF** and place it in a separate relation DEPT_LOCATIONS along with the primary key *Dnumber* of DEPARTMENT. The primary key of this newly formed relation is the combination $\{Dnumber, Dlocation\}$, as shown in Figure 14.2. A distinct tuple in DEPT_LOCATIONS exist for each location of a department. This decomposes the **non-1NF** relation into two **1NF** relations.

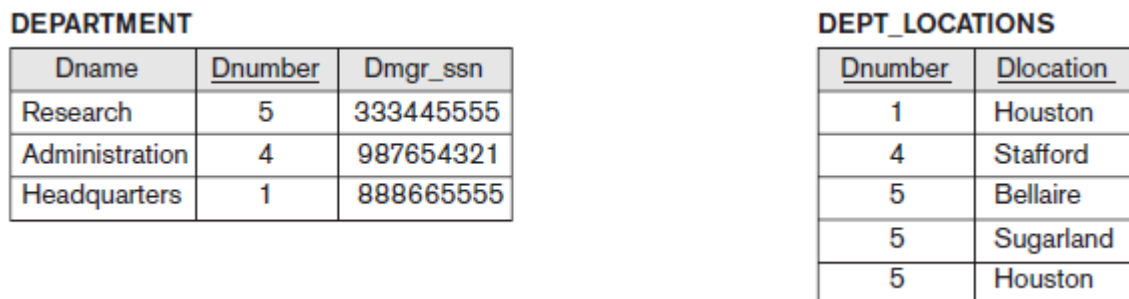


Figure 2.2 1NF 1st Technique

2. Expand the key so that there will be a separate tuple in the original DEPARTMENT relation for each location of a DEPARTMENT, as shown in Figure 14.9(c). In this case, the primary key becomes the combination $\{Dnumber, Dlocation\}$.



This solution has the disadvantage of introducing redundancy in the relation and hence is rarely adopted.

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 2.3 1NF 2nd Technique

3. If a maximum number of values is known for the attribute—for example, if it is known that at most three locations can exist for a department—replace the *Dlocations* attribute by three atomic attributes: *Dlocation1*, *Dlocation2*, and *Dlocation3*.

This solution has the disadvantage of introducing NULL values if most departments have fewer than three locations. Querying on this attribute becomes more difficult. For all these reasons, it is best to avoid this alternative.

Of the three solutions above, the first is generally considered best because it does not suffer from redundancy, and it is completely general; it places no maximum limit on the number of values.

2.1.3 The Second Normal Form Relations

A Relation R is said to be in the (2nd NF) if and only if all non-key attributes are FFD on the Primary Key of the relation.

Remark: Any relation with a simple Primary Key is in the (2nd NF).

2.1.4 The Third Normal Form Relations

A relation is said to be in the (3rd NF) if and only if:

- All non-key attributes are FFD on the Primary Key.
- All non-key attributes are Mutually Independent.

Example:

In the Relation Employee (SSN, Name, Sex, Dno, Salary) SSN is the Primary Key and all employees in one department have the same salary.

The First Constraint:

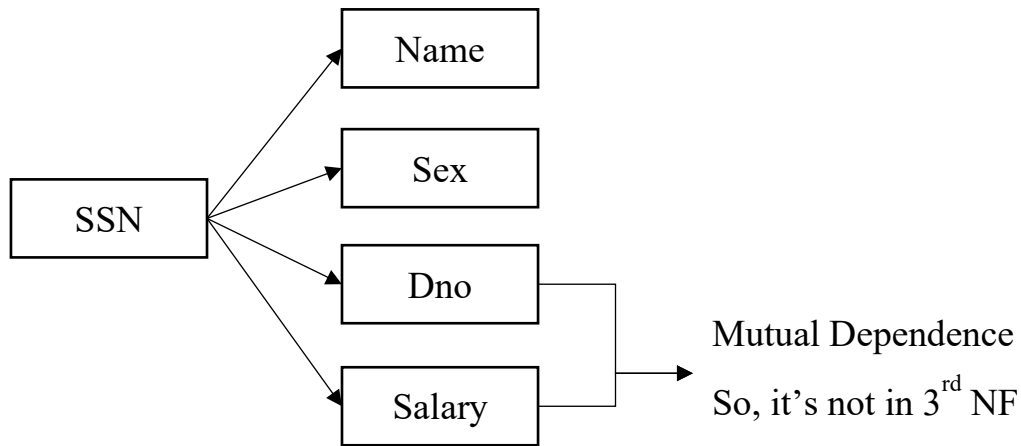
- SSN $\rightarrow$ Name
- SSN $\rightarrow$ Sex



- $SSN \rightarrow Dno$
- $SSN \rightarrow Salary$

The Second Constraint:

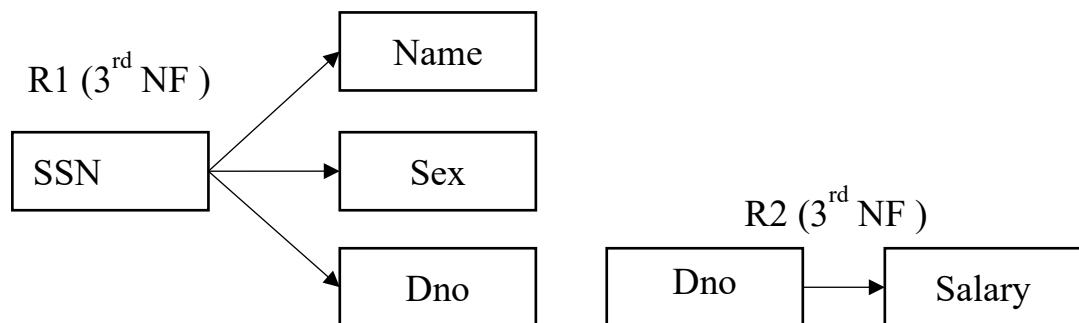
- $Dno \rightarrow Salary$



This relation is in the (2nd NF) but not in the (3rd NF). The solution is to divide the relation into two relations without losing information as follows:

The Employee (SSN, Name, Sex, Dno, Salary) can be broken into:

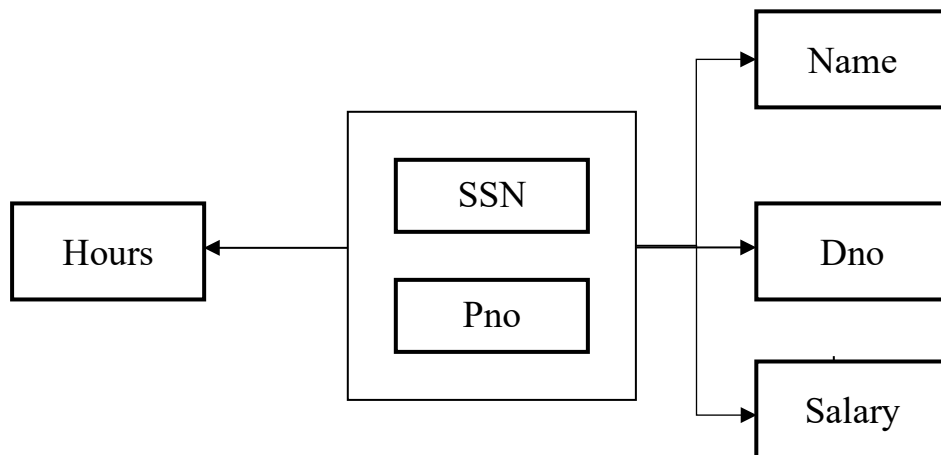
- The 3rd NF Relation R1 (SSN, Name, Sex, Dno) &
- The 3rd NF Relation R2 (Dno, Salary).



Example:

The relation R Employee (SSN, Name, Sex, Dno, Salary, Hours) (SSN,Pno) is the Primary Key (because of the added Hours attribute) and all employees in one department have the same salary.



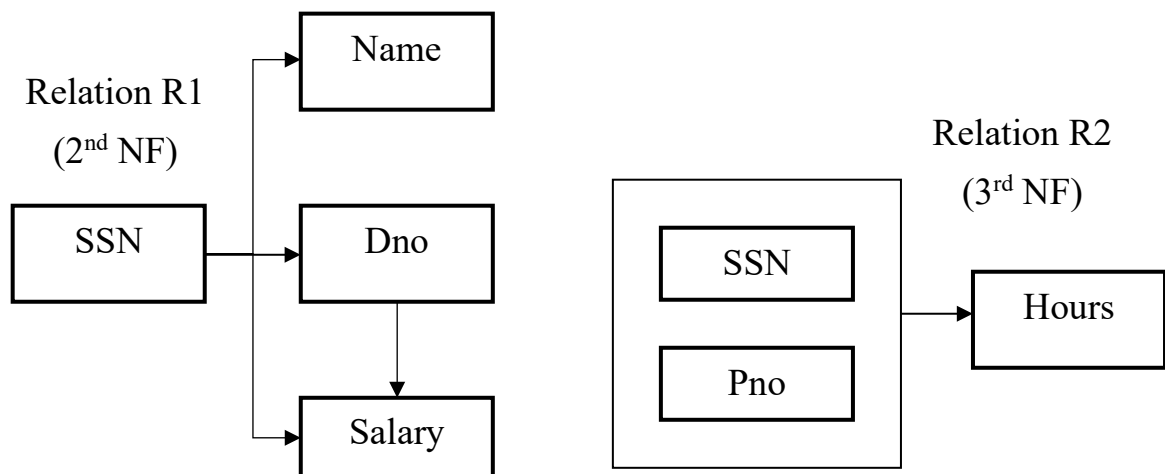
The relation graph:

This relation is not in the 2nd NF & respectively not in the 3rd NF

Solution:

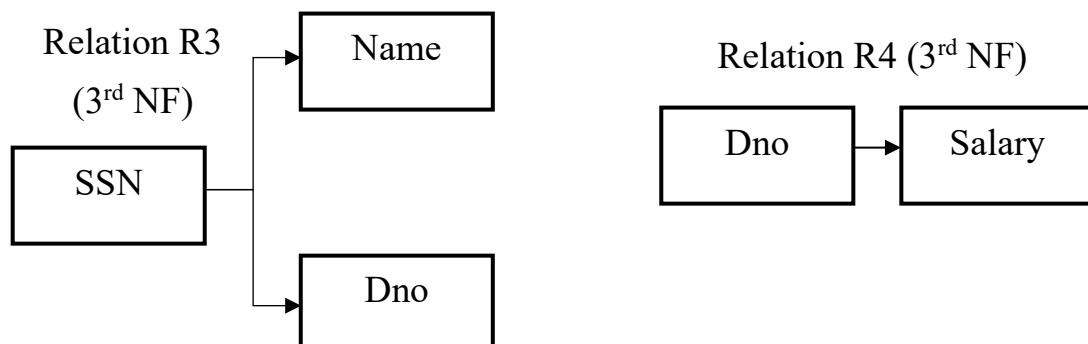
We divide the above un-normalized relation into normalized relations as follows:

- R1 (SSn, Name, Dno, Salary) in the 2nd NF
- R2 (SSN, Pno, hours) in the 3rd NF.



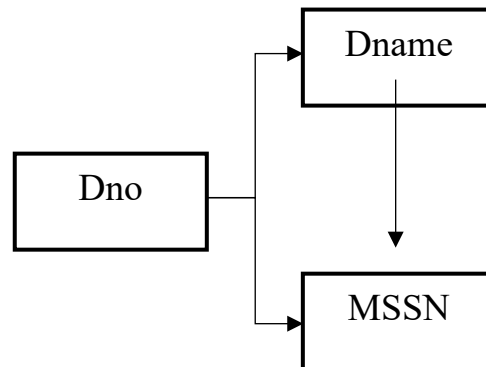
The relations R1 must be divided again into:

- R3 (SSn, Name, Dno) in the 3rd NF.
- R4 (Dno, Salary) in the 3rd NF.

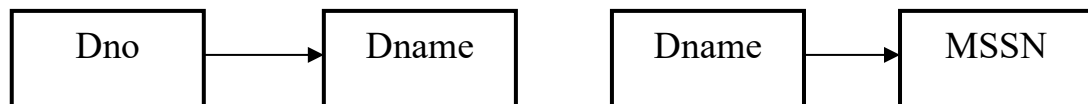


Example:

Suppose the Relation R (Dno, Dname, MSSN). Both Dno and Dname are unique.



This relation is not in the 3rd NF but it is normalized since the division of the relation will be un-normalized:



2.1.5 The Boyce-Codd Normal form (BCNF)

The 3rd NF is not suitable to check the correctness of the design if the relation has more than one candidate key which can be used as a primary key. The BCNF is suitable to check the correctness of the design for any relation regardless the number of the candidate keys.

Determinant: Any attribute (or attribute combinations) on which some other attribute is fully function dependent is a Determinant.

Example:

The relation R (SSN, Name, Dno, salary, Pno, Hours) mentioned before has three determinants:

1. (SSN, Pno)
2. Dno
3. SSN

Definition of the BCNF:

A relation R is said to be in BCNF if and only if every Determinant is a Candidate Key.

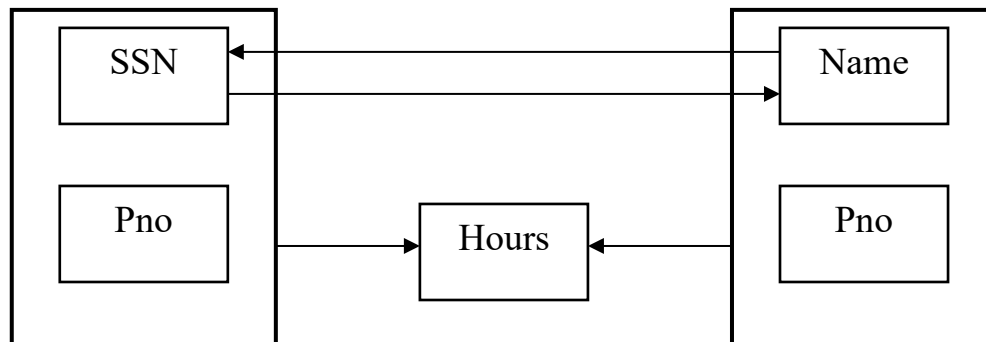
The relation R mentioned in the previous example has three determinants but only one of them is a Candidate Key, so it is not in BCNF.

Example:



The Relation WORKS (SSN, Name, Pno, Hours). Assume that:

- $SSN \rightarrow$ is unique for each employee and
- $Name \rightarrow$ is also unique for each employee



Solution:

The Candidate Keys are:

- (SNN, Pno)
- (Name, Pno)

The Determinants are:

- (SNN,Pno) $\rightarrow$ Hours
- (Name, Pno) $\rightarrow$ Hours
- SNN $\rightarrow$ Name
- Name $\rightarrow$ SNN

So, the relation WORKS is not in BCNF and it must be broken into group of relations:

1. The first suggestion is: R1 (SNN, Name) and R2 (SSN, Pno, Hours)
2. The second suggestion is: R1 (SNN, Name) and R2 (Name, Pno, Hours)



Exercises

- 1 - Define first, second, and third normal forms when only primary keys are considered. How do the general definitions of 2NF and 3NF, which consider all keys of a relation, differ from those that consider only primary keys?
- 2 - What undesirable dependencies are avoided when a relation is in 2NF?
- 3 - What undesirable dependencies are avoided when a relation is in 3NF?



Chapter 3 The Enhanced Entity Relationship Model EER

The ER modeling concepts discussed in Chapter 1 are enough for representing many database schemas for traditional database applications, which include many data-processing applications in business and industry. Since the late 1970s, however, designers of database applications have tried to design more accurate database schemas that reflect the data properties and constraints more precisely. This was particularly important for newer applications of database technology, such as databases for engineering design and manufacturing (CAD/CAM), telecommunications, complex software systems, and geographic information systems (GISs), among many other applications. These types of databases have requirements that are more complex than the more traditional applications. This led to the development of additional semantic data modeling concepts that were incorporated into conceptual data models such as the ER model.

The EER goal is achieved by incorporation of a semantic data modeling concepts into the conceptual ER Model.

These semantic concepts are:

1. **Object Oriented Concept**
 - a. Superclass & Subclass Relationship
 - b. Attribute & Relationships inheritance
2. **The concept of Specialization**
Looking for the real world from different point of views
3. **The concept of Categories**
Generation of a class which represents the union of entities of other classes.

Remark: The cooperating of the previews concepts has the following advantages:

1. Storage Saving
2. Performance Enhancement

3.1 Object Oriented Concepts

Features of the Superclass /Subclass Relationship concept on ER:

1. An entity in a subclass is related via the key attribute to its superclass entity.
2. An entity cannot exist in a database by being a member of a subclass unless it is a member in superclass.
3. An entity may be a member in many subclasses, but it is not necessary that every entity in a superclass is a member in subclass.
4. An entity that is a member of a subclass inherits all the attributes of its superclass & also inherits its relationships as well.
5. A member entity of the subclass represents the same real-world entity in the related superclass but in distinct specific role.



3.2 Specialization Concept

Specialization is the process of defining a *set of subclasses* of an entity type; this entity type is called the **superclass** of the specialization. The set of subclasses that forms a specialization is defined based on some distinguishing characteristic of the entities in the superclass.

For example, the set of subclasses {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of the superclass EMPLOYEE that distinguishes among employee entities based on the job type of each employee. We may have several specializations of the same entity type based on different distinguishing characteristics. For example, another specialization of the EMPLOYEE entity type may yield the set of subclasses {SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}; this specialization distinguishes among employees based on the method of pay.

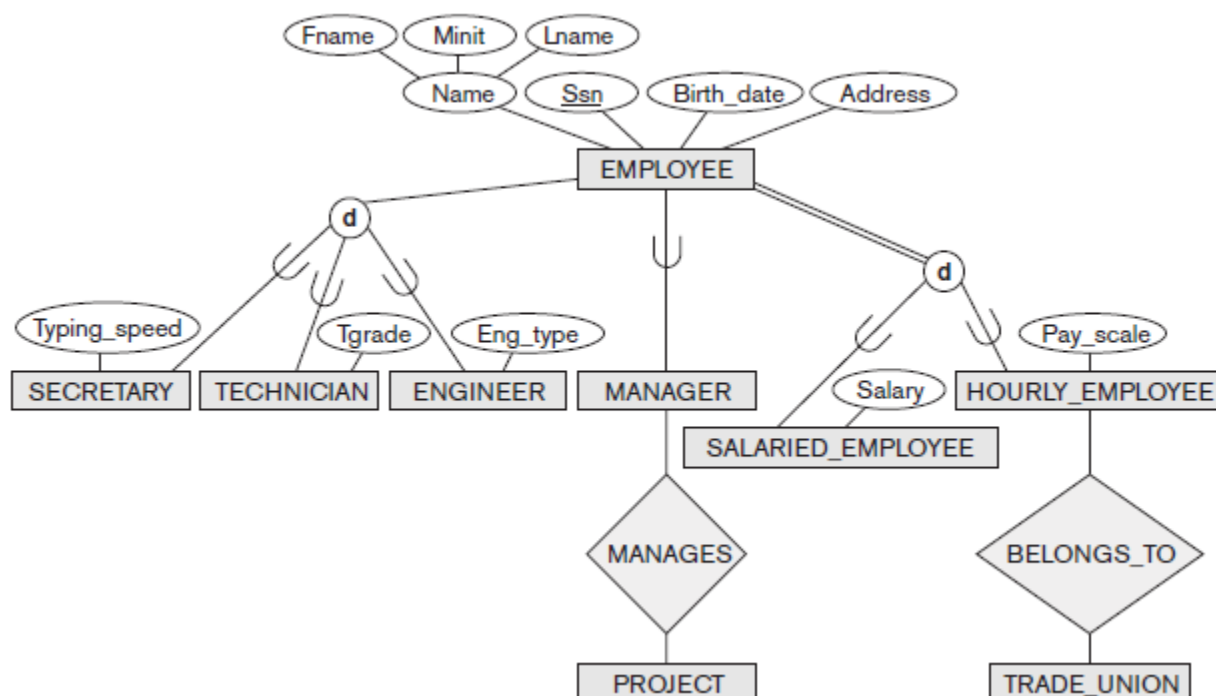


Figure 3.1 EER diagram notation to represent subclasses and specialization.

Remarks: In the previous example in *figure 3.1*.

1. We can determine three Specifications based on the following characteristics:
 - Job Type
 - Rank
 - Method of Pay
2. The subclasses that define a specialization are attached by lines to a circle which is connected to superclass.
3. The subset symbol on each line connecting a subclass to the circle indicates the direction of the superclass/subclass relationships.
4. Attributes that apply only to entities of a particular subclass are attached to the rectangle representing those subclass & these attributes are called Specific Attributes (or Local Attribute) *ex: Typing_Speed* of SECRETARY
5. Subclass can participate in a specific relationship types *ex: The relationship named Belongs_to* in previous example.

3.2.1 Constraint and Characteristics of specialization:

1. Definition Constraints:

a. Predicate Defined Specialization

It is process of defining a condition to determine exactly the entities that will become members of each subclass by placing a condition on the value of some attribute of the superclass, which is called **Defining Attribute** of the related subclass.

b. User Defined Specialization

When we have not any condition in order to determine membership in a subclass hence membership is specified individually for each entity by the user and not by any condition that can be evaluated automatically.

2. Disjoints Constraints:

a. Disjoint Specialization

This means that an entity can be a member of at most one subclass of a specialization (Figure 3.2)



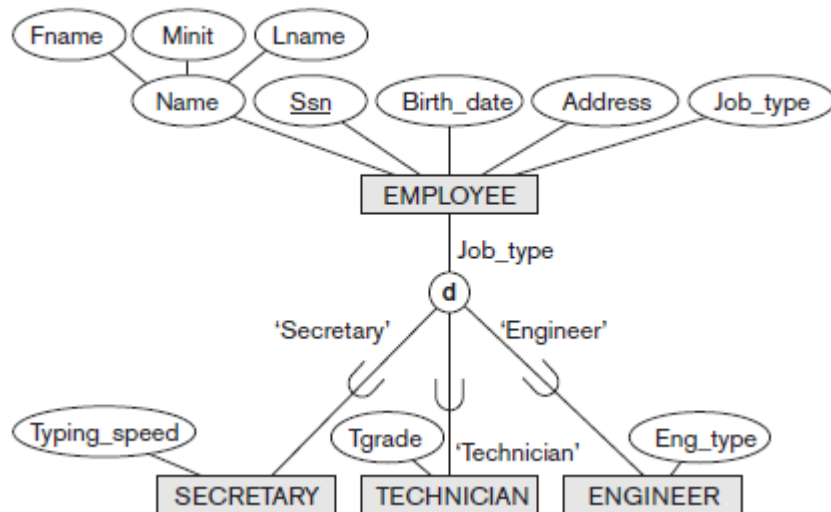


Figure 3.2

b. Overlapped specialization:

This means that an entity can be a member in any number of subclasses of specialization (Figure 3.3)

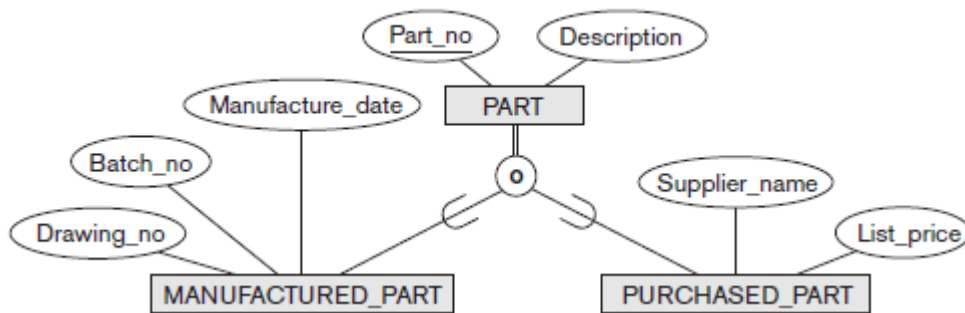


Figure 3.3

3. Participation Constraints:

a. Total Participation Specialization

It specifies that every entity in a superclass must be a member of at least one subclass in the specialization.

b. Partial Participation Specialization

It allows an entity in superclass not to belong to any of its subclasses in the specialization.



3.2.2 Specialization Hierarchies and Lattices

A subclass itself may have further subclasses specified on it, forming a hierarchy or a lattice of specializations (Figure 3.4).

1. Specialization Hierarchy (Tree Inheritance)

It is the constraint that every subclass participates as a subclass in only subclass/class relationship.

2. Specialization Lattice (Multiple Inheritance)

It is the constraint that a subclass can be a subclass in more than one class/subclass relation.

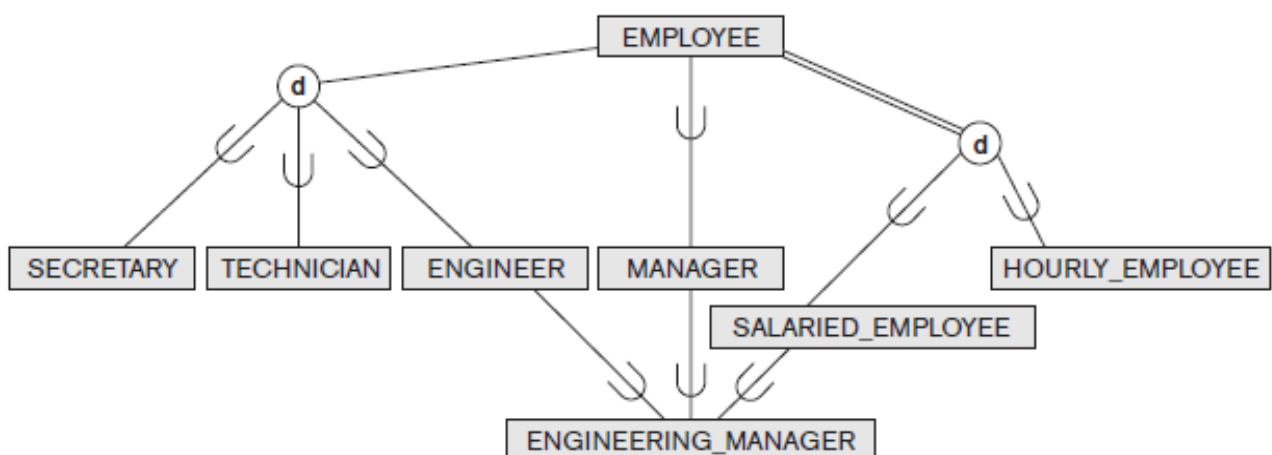


Figure 3.4

Remarks:

- In specialization with lattice or hierarchy inheritance, a subclass inherits the attributes not only of its direct superclass but also of all its predecessor superclasses all the way to the root of the hierarchy or lattice.
- **Leaf Node Class:** it is a class that has no subclasses of its own.
- **Shared Subclass:** it is a subclass with more than one superclass and its entities represent a subset of the intersection of the entities of its superclasses. This means that an entity of the shared subclass must exist as an entity in all its superclasses.
- **For the Example in Figure 3.4** the shared subclass ENGINEERING_MANAGER means that an engineering manager must be an engineer, manager and salaried_employee.

Example:

Figure 3.5 shows another specialization lattice of more than one level. This part of a conceptual schema for a UNIVERSITY database. Notice that this arrangement would have been a hierarchy except for the STUDENT ASSISTANT subclass, which is a subclass in two distinct class/ subclass relationships.

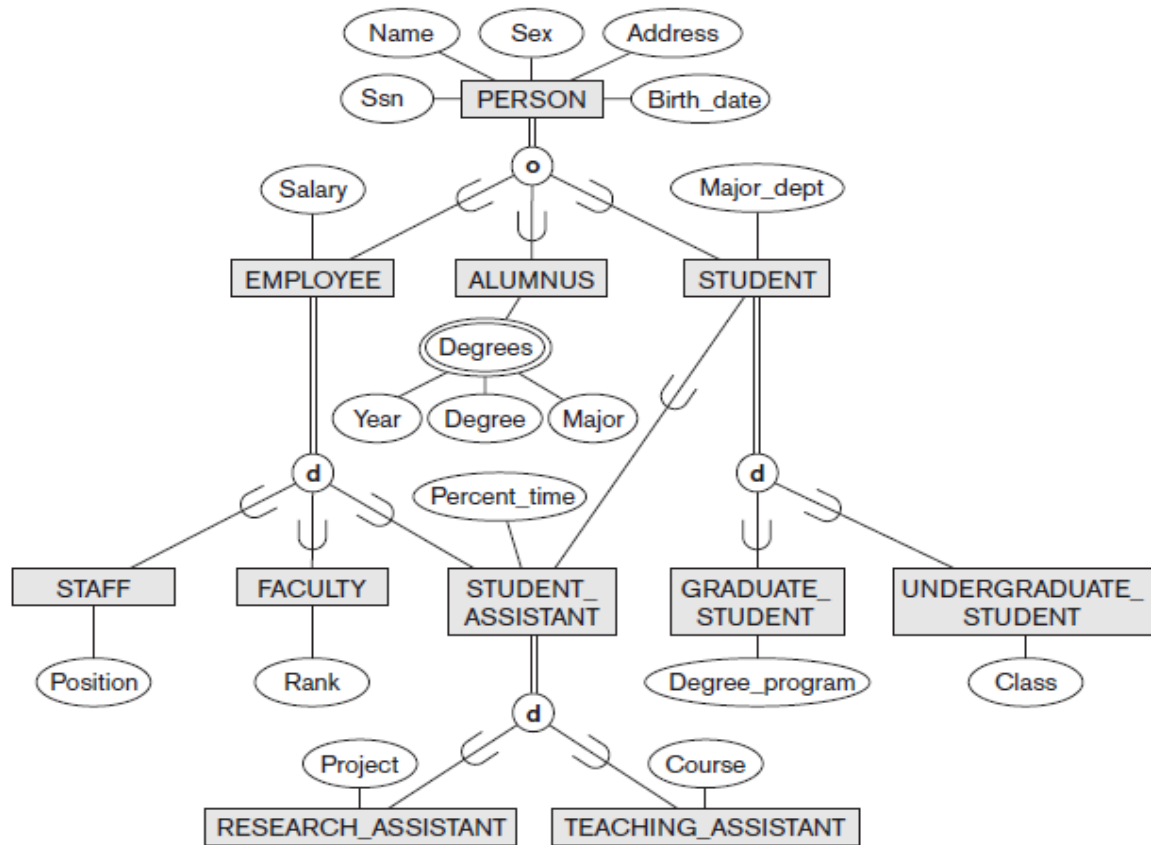


Figure 3.5

The requirements for the part of the UNIVERSITY database shown in (Figure 3.5) are the following:

1. The database keeps track of three types of people: employees, alumni, and students. A person can belong to one, two, or all three of these types. Each person has a name, SSN, sex, address, and birth data.
2. Every employee has a salary, and there are three types of employees; faculty, staff, and student assistant. Each employee belongs to exactly one of these types. For each alumnus, a record of the degree or degrees that he or she earned at the university is kept, including the name of the degree, the year granted, and the major department. Each student has a major department.
3. Each faculty has a rank, whereas each staff member has a staff position. Student assistants are classified further as either research assistants or teaching assistants, and the percent of time that they work is recorded in the database. Research assistants have their research project stored, whereas teaching assistants have the current course they work on.



4. Students are further classified as either graduate or undergraduate with the specific attribute degree program (M.S, Ph.D., M.B.A, and so on) and class (freshman, sophomore, and so on), respectively.

In figure 3.5, all person entities represented in the database are members of the PERSON entity type, which is specialized into the subclasses {EMPLOYEE, ALUMNUS, STUDENT}. This specialization is over lapping; for example, an alumnus may also be an employee and may also be a student pursuing an advanced degree. The subclass STUDENT is the superclass for the specialization {GRADUTE_STUDENT, UNDERGRADUTE_STUDENT}, while EMPLOYEE is the superclass for the specialization {STUDENT_ASSISTANT, FACULTY, STAFF}. Notice that STUDENT_ASSISTANT is also a subclass of STUDENT. Finally, STUDENT_ASSISTANT is the superclass for the specialization into {RESEARCH_ASSISTANT, TEACHING_ASSISTANT}.

In such a specialization lattice or hierarchy, a subclass inherits the attributes not only of its direct superclass, but also of all its predecessor super classes all the way to the root of the hierarchy or lattice.

For example, an entity in GRADUATE_STUDENT inherits all the attributes of that entity as a STUDENT and as a PERSON. Notice that an entity may exist in several leaf nodes of the hierarchy, where a leaf node is a class that has no subclasses of its own. For example, a member of GRADUATE_STUDENT may also be a member of RESEARCH_ASSISTANT.

A subclass with more than one superclass is called **Shared Subclass**, such as ENGINEERING_MANAGER in figure 3.4. This leads to the concept known as **Multiple Inheritance**, where the shared subclass ENGINEERING_MANAGER directly inherits attributes and relationships from multiple classes. Notice that the existence of at least one shared class leads to a lattice (and hence to multiple inheritance); if no shared subclass existed, we would have a hierarchy rather than a lattice.

An important rule related to multiple inheritances can be illustrated by the example of the shared subclass STUDENT_ASSISTANT in figure 3.5, which inherits attributes from both EMPLOYEE and STUDENT. Here, both EMPLOYEE and STUDENT inherit the same attributes from PERSON. The rule states that if an attribute (or relationship) originating in the same superclass (PERSON) is inherited more than once via different paths (EMPLOYEE and STUDENT) in the lattice, than it should be included only once in the shared subclass (STUDENT_ASSISTANT). Hence, the attributes of PERSON are inherited only once in the STUDENT_ASSISTANT subclass of figure (5).

If we do not allow for overlapping to occur in specialization (by considering all possible combinations of classes that may have some entity belong to all these classes) there will be no shared subclasses.



In the example the person may be {E, A, S} and to prevent overlapping It would be necessary to create seven subclasses of a person in order to cover all possible types of entities: **E,A, S, E-A, E-S, A-S and E-A-S** which will lead to more complexity.

3.3 The concept of Category

Category is a union type represented by a subclass that contains a collection of real-world entities (objects) which are a subset of the union of entity types.

Example (UNION TYPES)

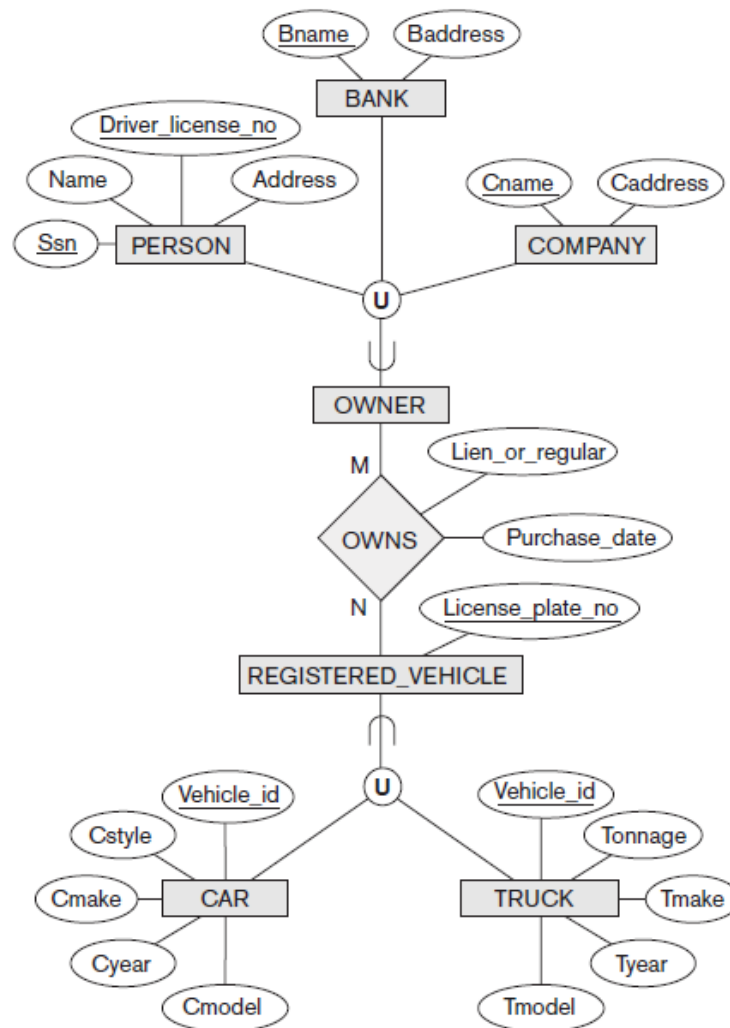


Figure 3.6

Remarks:

- Attribute inheritance works more selectively in the case of category. For example, in figure 3.6 each owner entity inherits the attributes of a company, a person or a bank depending on the superclass to which the entity belongs.
- **On the other hand**, a shared subclass entity such as `ENGINEERING_MANAGER` figure 3.4 inherits all the attributes of its super classes: (`SALARIED_EMPLOYEE`, `ENGINEER`, and `MANAGER`).

The UNIVERSITY Database Example:

For our example database application, consider a UNIVERSITY database that keeps track of students and their majors, transcripts, and registration as well as of the university's course offerings. The database also keeps track of the sponsored research projects of faculty and graduate students. This schema is shown in figure 3.7.

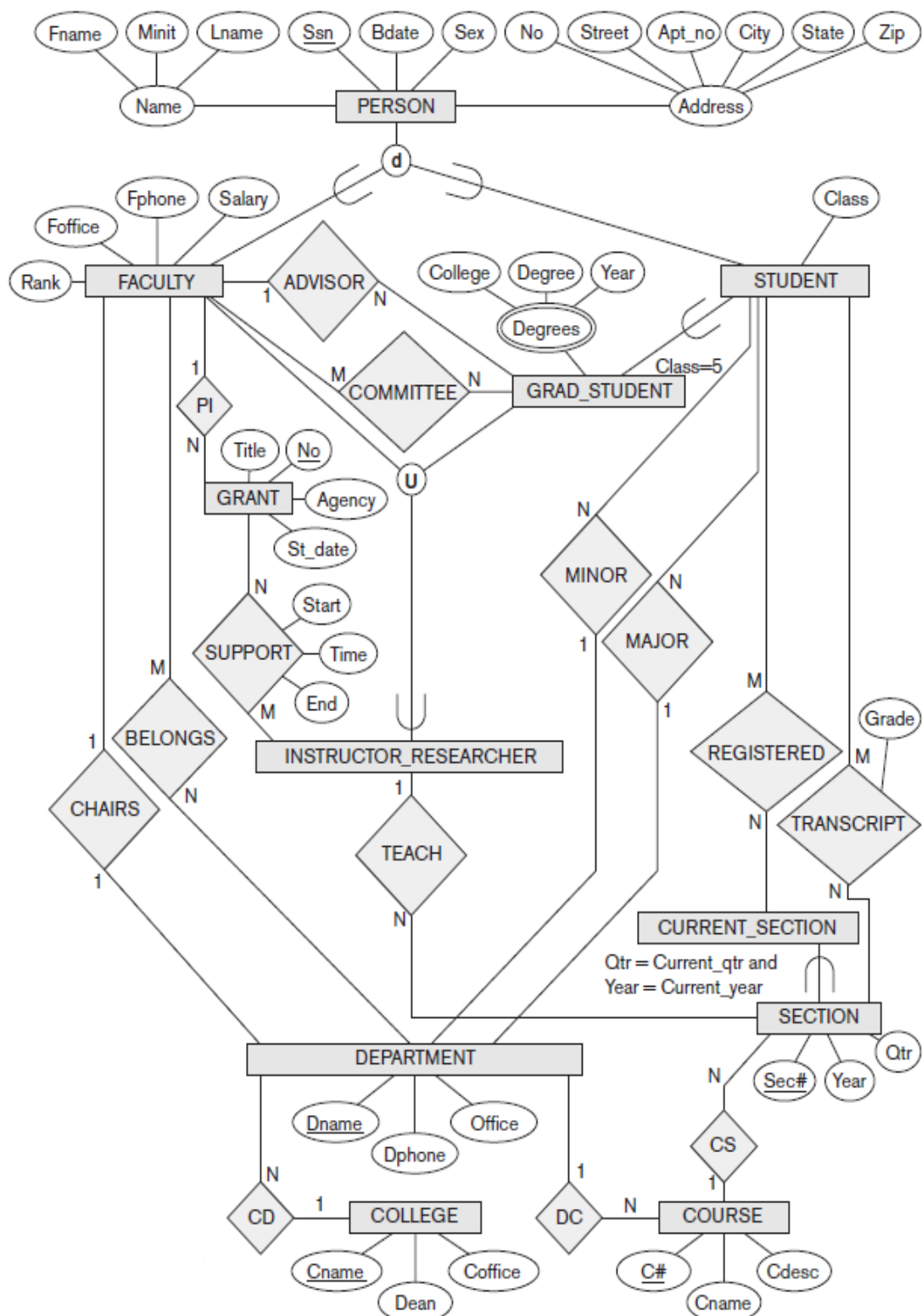


Figure 3.7



A discussion of the requirements that led to this schema as follows:

For each person, the database maintains information on the person's Name [Name], social security number [Ssn], address [Address], sex [Sex], and birth date [Bdate]. To subclass of the *PERSON* entity type are identified: *FACULTY* and *STUDENT*. Specific attributes of *FACULTY* are rank [Rank] (assistant, associate, adjunct, research, visiting, and so on), office [Foffice], office phone [Fphone], and salary [Salary].

All faculty members are related to the academic department(s) with which they are affiliated [BELONGS] (a faculty member can be associated with several departments, so the relationship is M: N). A specific attribute of *STUDENT* is [class] (freshman = 1, sophomore = 2, ..., graduate student = 5).

Each *STUDENT* is also related to his or her major and minor department, if known ([MAJOR] and [MINOR]), to the course sections he or she is currently attending [REGISTERED], and to the courses.

completed [TRANSCRIPT]. Each *TRANSCRIPT* instance includes the grade the student received [Grade] in the course section.

GRAD_STUDENT is a subclass of *STUDENT*, with the defining predicate Class = 5. For each graduate student, we keep a list of previous degrees in a composite, multi-valued attribute [Degrees]. We also relate the graduate student to a faculty advisor [ADVISOR] and to a thesis committee [COMMITTEE], if one exists.

An academic department has the attributes name [Dname], telephone [Dphone], and office number [Office] and is related to the faculty member who is its chairperson [CHAIRS] and to the college to which it belongs [CD]. Each college has attributes collage name [Cname], office number [Coffice], and the name of its dean [Dean].

A course has attributes course number [C#], course name [Cname], and course description [Cdesc]. Several sections of each course are offered, with each section having the attributes section number [Sec#] and the year and quarter in which the section was offered ([Year] and [Qtr]). Section numbers uniquely identify each section.

The sections being offered during the current quarter are in a subclass *CURRENT_SECTION* of *SECTION*, with the defining predicate Qtr = Current_qtr and Year = Current_year. Each section is related to the instructor who taught or is teaching it ([TEACH]) if that instructor is in the database.

The category *INSTRUCTOR_RESEARCHER* is a subset of the union of *FACULTY* and *GRAD_STUDENT* and includes all faculties, as well as graduate students who are supported by teaching or research. Finally, the entity type *GRANT* keeps track of research grants and contracts awarded to the university.

Each grant has attributes grant title [Title], grant number [No], the awarding agency [Agency], and the starting data [St_data]. A grant is related to one principal investigator [PI] and to all researchers it supports [SUPPORT]. Each instance of support has as attributes the starting date of support [Start], the ending date of the support (if known) [End], and the percentage of time being spent on the project [Time] by the researcher being supported.



3.4 EER-to-Relational Mapping

Here we are going to add further step to the ER-to-Relational mapping algorithm (**Seven Steps**) to handle the mapping of specialization. This step will have 4-main options and conditions under which we can determine the suitable option. We use $\text{Attrs}(R)$ to denote the attributes of relation R and $\text{PK}(R)$ to denote the primary key of R .

First, we describe the mapping formally, then we illustrate it by examples.

Step 8: Options for mapping Specialization:

Convert each specialization with m subclasses $\{S_1, S_2, \dots, S_m\}$ and superclass C , where the attributes of C are $\{k, a_1, \dots, a_n\}$ and K is the primary key, into relation schemas using one of the following option.

A. **Option 8A** Multiple relations Superclass and Subclasses:

Create a relation L for C with attributes $(L) = \{k, a_1, \dots, a_n\}$ and $\text{PK}(L) = k$. Create a relation L_i for each subclass S_i , $1 \leq i \leq m$, with the attributes $(L_i) = \{k\} \cup \{\text{attributes of } S_i\}$ and $\text{PK}(L_i) = k$. This option works for any specialization (total or partial, disjoint or overlapping).

B. **Option 8B** Multiple Relations-Subclass relation Only:

Create a relation L_i for each subclass S_i , $1 \leq i \leq m$ with the Attributes $(L_i) = \{\text{attributes of } S_i\} \cup \{k, a_1, \dots, a_n\}$ and $\text{PK}(L_i) = k$. This option only works for a specialization whose subclasses are total (**Why?**). If the specialization is overlapping; an entity may be duplicated in several relations. (If the specialization is disjoint & total it will be optimal mapping).

C. **Option 8C** Single relation with one type Attribute:

Create a single relation L with attributes $(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t\}$ and $\text{PK}(L) = k$. The attribute t is called a type (or discriminating) attribute that indicates the subclass to which each tuple belongs, if any. This Option works only for a specialization whose subclasses are disjoint and has the potential for generating many Null values if many specific attributes exist in a subclass.

D. **Option 8D:** Single relation with multiple type attributes:

Create a single relation schema L with Attributes $(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t_1, t_2, \dots, t_m\}$ and $\text{PK}(L) = k$.

Each t_i , $1 \leq i \leq m$, is a Boolean type attribute indicating whether a tuple belongs to subclass S_i . This option works for specialization whose subclasses are overlapping or disjoint.



Examples:

1. Mapping the EER Schema in the following figure using option 8A

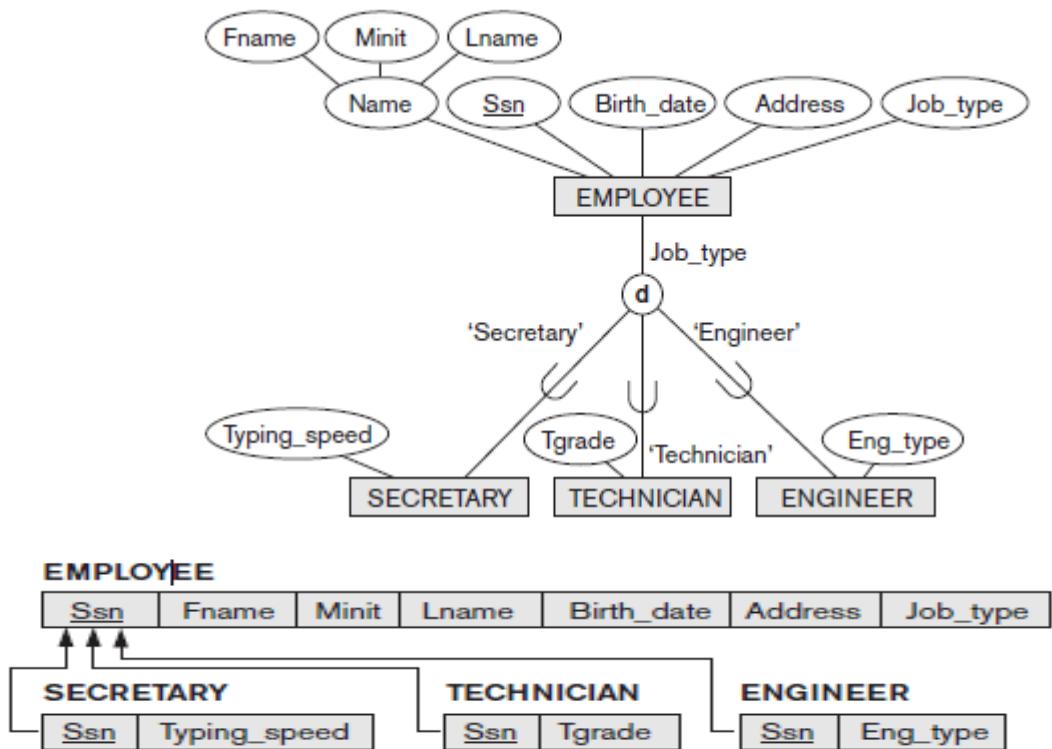


Figure 3.8 Mapping the EER schema using option 8A

2. Mapping the following EER Schema using option 8B

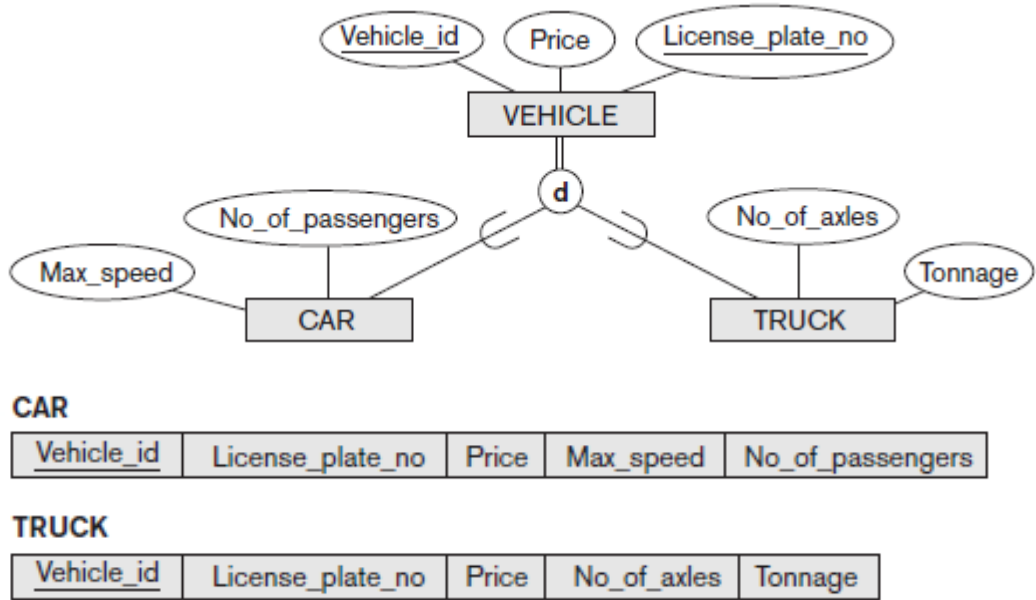


Figure 3.9 Mapping (Car & Truck) the EER schema using option 8B.



3. Mapping the EER Schema in the following figure using option 8C

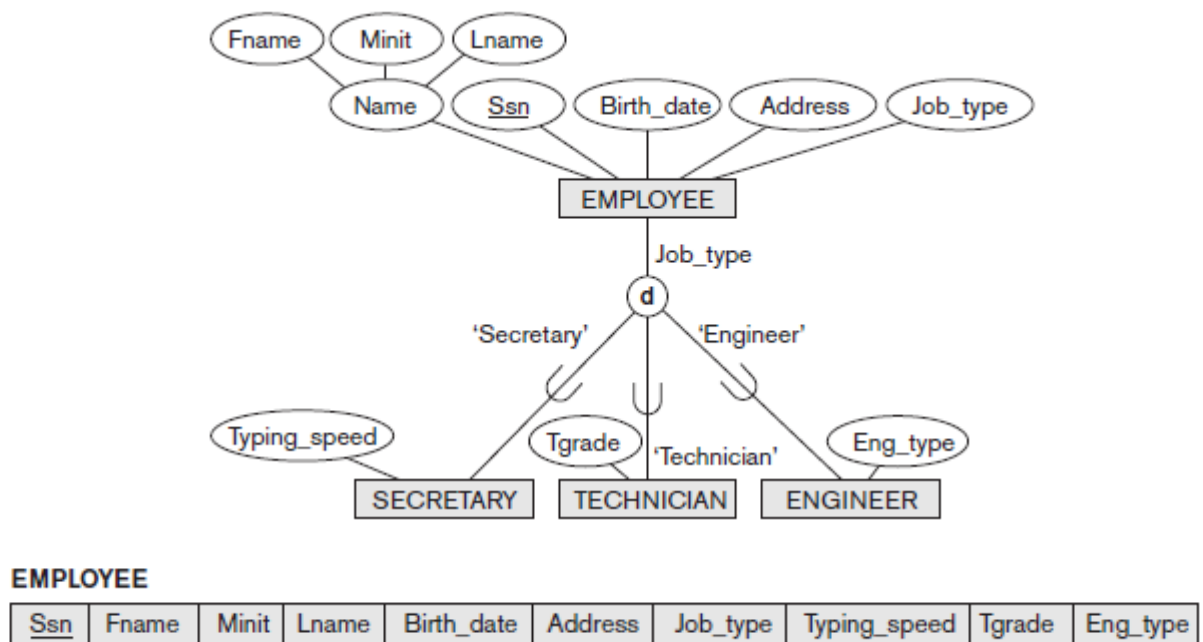


Figure 3.10 Mapping the EER schema in Figure 3.2 using

4. Mapping the EER Schema in the following figure using option 8D

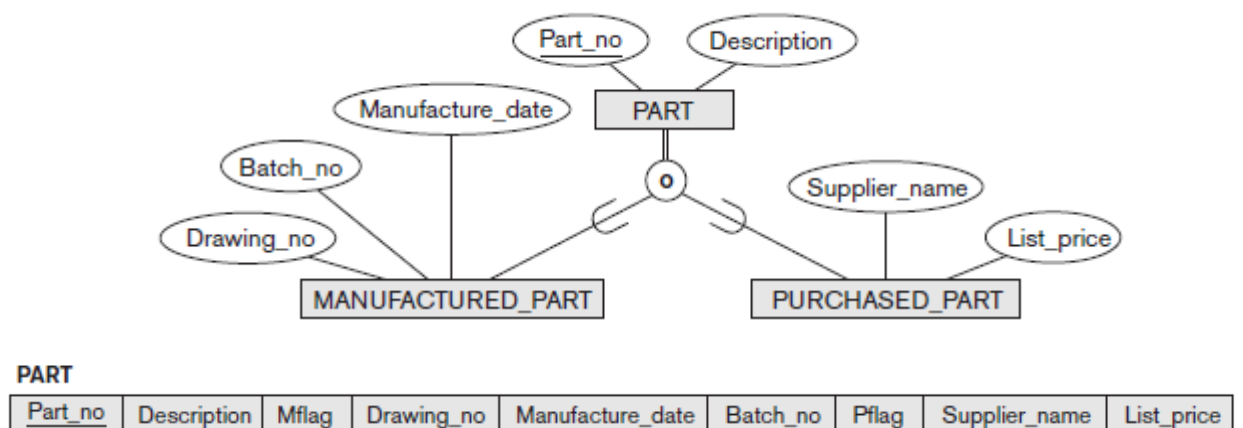


Figure 3.11 Mapping using option 8D with Boolean type fields



Remarks:

1. Option 8A works for any constraints on specialization (total, partial, disjoint, overlap).
2. Option 8B works for only when both the disjoint and total constraints hold.
3. Option 8c & 8D (Single Relations) are Not recommended if many specific attributes (local att.) are defined for the subclasses. **Why?**
4. Option 8C & 8D (Single Relations) are recommended if few specific attributes (local att.) are defined for the subclasses. **Why?**
5. Option 8C is used to handle disjoint subclasses by including a single **type attribute** (discriminating) to indicate the subclass to which each tuple belongs. If specialization is partial the **type Attribute** can have a Null value in tuples that do not belong to any subclass.
6. Option 8D is designed to handle overlapping subclasses by including m boolean type field where m is Number of subclasses of the specialization.
7. Figure 3.12 show the mapping for Figure 3.5 in which:
 - Option 8A is used for Person / {Employee, Alumnus, Student},
 - Option 8C for Employee / {Staff, Faculty, Student_Assistant}, and
 - Option 8D for Student_Assistant/Research_Assistant, Teaching_Assistant}, Student/Student_Assistant and Student/{Graduate_Student, UnderGraduate_Student}.

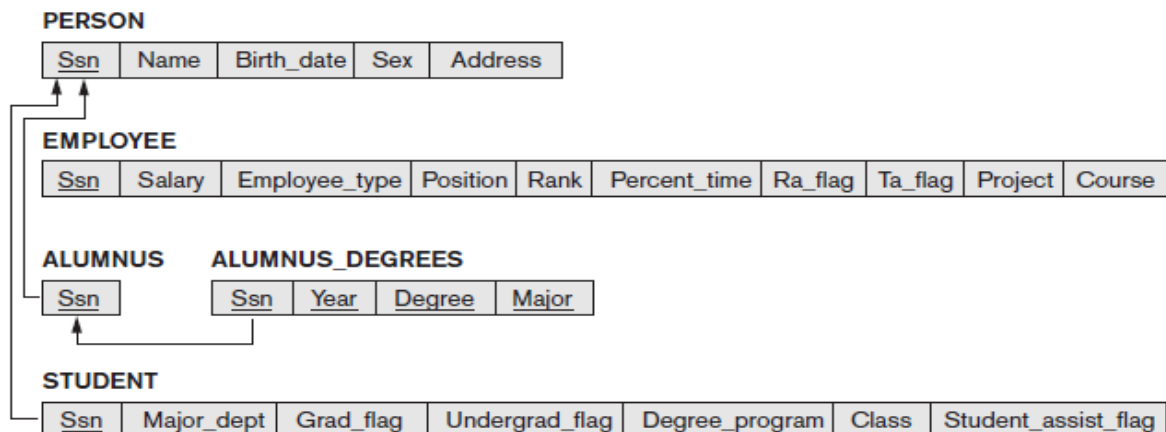


Figure 3.12



Step 9: Mapping of Categories (union type)

All faculty members are related to the academic department(s) with which they are affiliated [BELONGS] (a faculty member can be associated with several departments, so the relationship is M: N).

For mapping a category whose super classes have different keys, it is customary to specify a new key attribute called surrogate key.

Surrogate key: it is a new key attribute created for a relation which corresponds to a category defining super classes that have different keys.

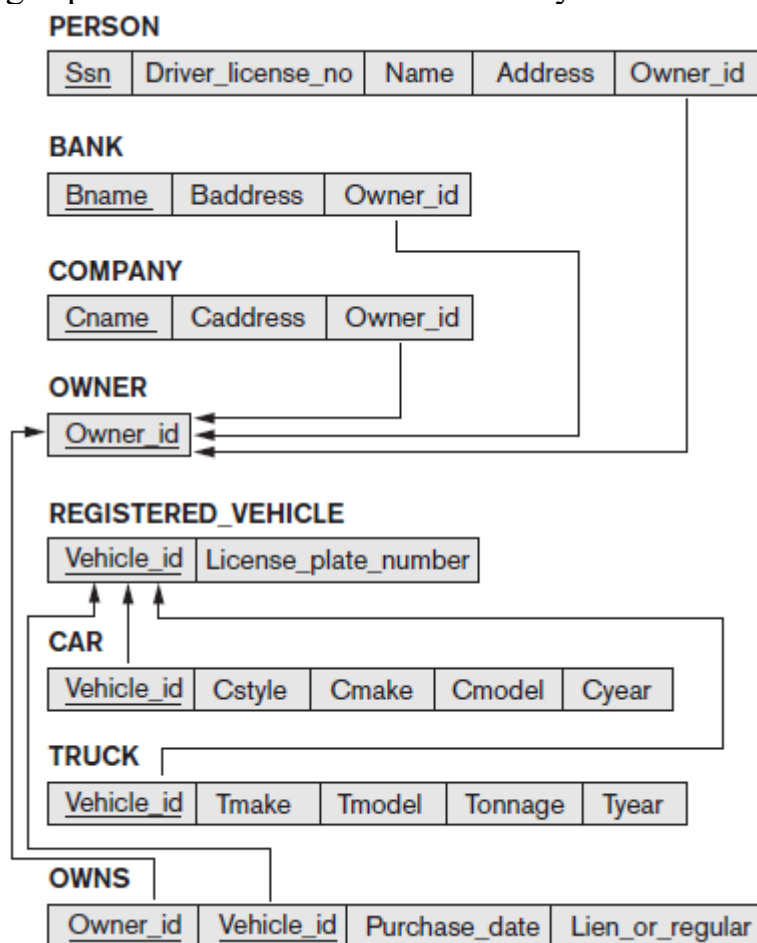


Figure 3.13

Remark:

The surrogate key is primary key of the category relation, and it is also included as a foreign key in the relations which corresponding to super classes of the category.



3.5 Use of UML Diagrams as an Aid to Database Design Specification

3.5.1 UML as a Design Specification Standard

There is a need of some standard approach to cover the entire spectrum of requirements analysis, modelling, design, implementation, and deployment of databases and their applications. One approach that is receiving wide attention and that is also proposed as a standard by the Object Management Group (OMG) is the Unified Modelling Language (UML) approach. It provides a mechanism in the form of diagrammatic notation and associated language syntax to cover the entire life cycle.

Presently, UML can be used by software developers, data modelers, database designers, and so on to define the detailed specification of an application. They also use it to specify the environment consisting of users, software, communications, and hardware to implement and deploy the application.

UML combines commonly accepted concepts from many object-oriented (O-O) methods and methodologies. It is generic and is language-independent and platform-independent. Software architects can model any type of application, running on any operating system, programming language, or network, in UML. That has made the approach very widely applicable. Tools like Rational Rose are currently popular for drawing UML diagrams—they enable software developers to develop clear and easy-to-understand models for specifying, visualizing, constructing, and documenting components of software systems. Since the scope of UML extends to software and application development at large, we will not cover all aspects of UML here. Our goal is to show some relevant UML notations that are commonly used in the requirements collection and analysis phase of database design, as well as the conceptual design phase (see Phases 1 and 2 in Figure 2.1). A detailed application development methodology using UML is outside the scope of this book and may be found in various textbooks devoted to object-oriented design, software engineering, and UML.

UML has many types of diagrams. **Class diagrams**, which can represent the end result of conceptual database design. To arrive at the class diagrams, the application requirements may be gathered and specified using **use case diagrams**, **sequence diagrams**, and **statechart diagrams**.

In the rest of this section we introduce the different types of UML diagrams briefly to give the reader an idea of the scope of UML. Then we describe a small sample application to illustrate the use of some of these diagrams and show how they lead to the eventual class diagram as the final conceptual database design. The diagrams presented in this section pertain to the standard UML notation and have been drawn using Rational Rose.

3.5.2 UML for Database Application Design

UML was developed as a software engineering methodology. Most software systems have sizable database components. The database community has started embracing UML, and now some database designers and developers are using UML for data modeling as



well as for subsequent phases of database design. The advantage of UML is that even though its concepts are based on object-oriented techniques, the resulting models of structure and behavior can be used to design relational, object-oriented, or object-relational databases.

One of the major contributions of the UML approach has been to bring the traditional database modelers, analysts, and designers together with the software application developers. In Figure 2.1 we showed the phases of database design and implementation and how they apply to these two groups. UML also allows us to do behavioural, functional, and dynamic modelling by introducing various types of diagrams.

This results in a more complete specification/description of the overall database application. In the following sections we summarize the different types of UML diagrams and then give an example of the use case, sequence, and state chart diagrams in a sample application.

3.5.3 Different Types of Diagrams in UML

UML defines nine types of diagrams divided into these two categories:

- **Structural Diagrams.** These describe the structural or static relationships among schema objects, data objects, and software components. They include class diagrams, object diagrams, component diagrams, and deployment diagrams.
- **Behavioural Diagrams.** Their purpose is to describe the behavioural or dynamic relationships among components. They include use case diagrams, sequence diagrams, collaboration diagrams, state chart diagrams, and activity diagrams.

We introduce the nine types briefly below. The **structural diagrams** include:

- A. Class Diagrams.** Class diagrams capture the static structure of the system and act as foundation for other models. They show classes, interfaces, collaborations, dependencies, generalizations, associations, and other relationships. Class diagrams are a very useful way to model the conceptual database schema.
- B. Object Diagrams.** Object diagrams show a set of individual objects and their relationships and are sometimes referred to as *instance diagrams*. They give a static view of a system at a particular time and are normally used to test class diagrams for accuracy.
- C. Component Diagrams.** Component diagrams illustrate the organizations and dependencies among software components. A component diagram typically consists of components, interfaces, and dependency relationships. A component may be a source code component, a runtime component, or an executable



component. It is a physical building block in the system and is represented as a rectangle with two small rectangles or tabs overlaid on its left side. An **interface** is a group of operations used or created by a component and is usually represented by a small circle. Dependency relationship is used to model the relationship between two components and is represented by a dotted arrow pointing from a component to the component it depends on. For databases, component diagrams stand for stored data such as tablespaces or partitions. Interfaces refer to applications that use the stored data.

- D. **Deployment Diagrams.** Deployment diagrams represent the distribution of components (executables, libraries, tables, files) across the hardware topology. They depict the physical resources in a system, including nodes, components, and connections, and are basically used to show the configuration of runtime processing elements (the nodes) and the software processes that reside on them (the threads).

Next, we briefly describe the various types of **behavioral diagrams** and expand on those that are of particular interest.

- E. **Use Case Diagrams.** Use case diagrams are used to model the functional interactions between users and the system. A **scenario** is a sequence of steps describing an interaction between a user and a system. A **use case** is a set of scenarios that have a common goal. The use case diagram was introduced by Jacobson⁷ to visualize use cases. A **use case diagram** shows actors interacting with use cases and can be understood easily without the knowledge of any notation. An individual use case is shown as an oval and stands for a specific task performed by the system. An **actor**, shown with a stick person symbol, represents an external user, which may be a human user, a representative group of users, a certain role of a person in the organization, or anything external to the system (see Figure 2.7). The use case diagram shows possible interactions of the system (in our case, a database system) and describes as use cases the specific tasks the system performs. Since they do not specify any implementation detail and are supposed to be easy to understand, they are used as a vehicle for communicating between the end users and developers to help in easier user validation at an early stage. Test plans can also be described using use case diagrams. Figure 2.7 shows the use case diagram notation. They **include** relationship is used to factor out some common behaviour from two or more of the original use cases—it is a form of reuse. For example, in a university



environment shown in Figure 2.8, the use cases *Register for course* and *Enter grades* in which the actor's student and professor are involved, include a common use case called *Validate user*. If a use case incorporates two or more significantly different scenarios, based on circumstances or varying conditions, the **extend** relationship is used to show the subcases attached to the base case.

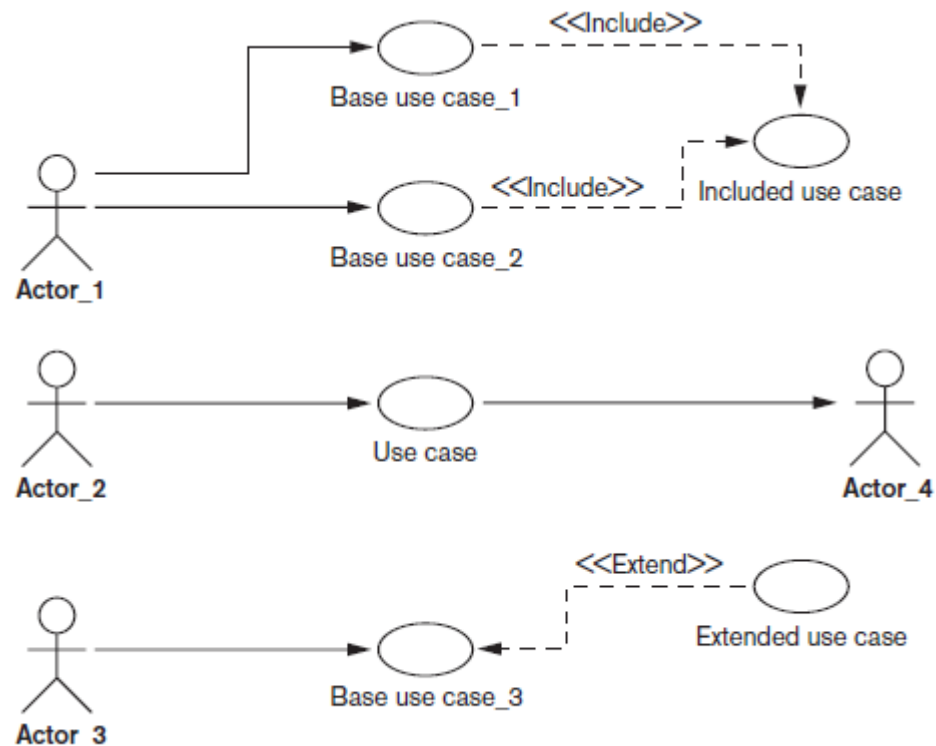


Figure 3.14

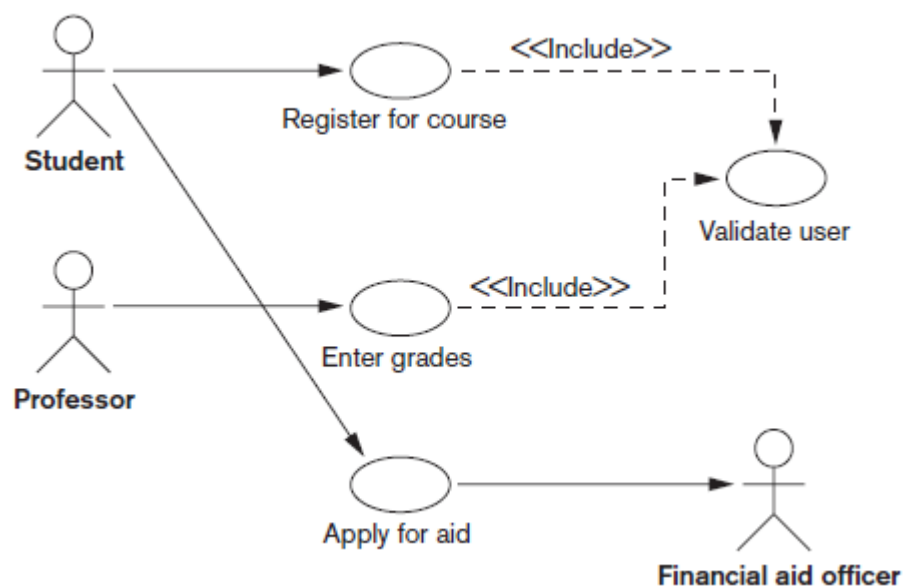


Figure 3.15

3.5.4 A Modelling and Design Example: UNIVERSITY Database

the designer should make sure that the class diagram will account for all the specifications that have been given in the form of the use cases, statechart, and sequence diagrams. The class diagram in Figure 2.9 shows a possible class diagram for this application, with the structural relationships and the operations within the classes. These classes will need to be implemented to develop the UNIVERSITY database, and together with the operations they will implement the complete class schedule/enrollment/aid application. Only some of the attributes and methods (operations) are shown in Figure 2.9. It is likely that these class diagrams will be modified as more details are specified and more functions evolve in the UNIVERSITY application.

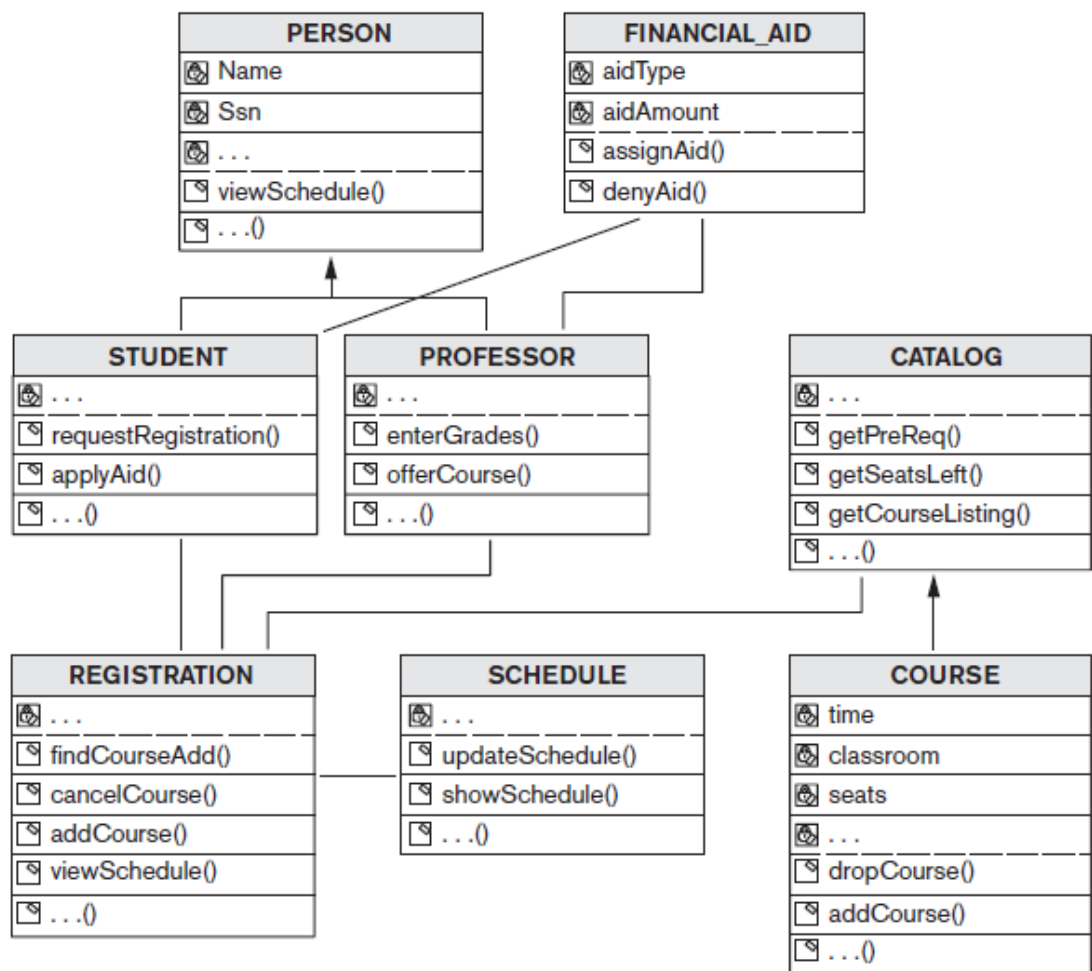


Figure 3.16



Exercises

Student Name:

Student ID:

The following narrative describes a simplified version of the organization of Olympic facilities planned for the summer Olympics. **Draw an EER diagram that shows the entity types, attributes, relationships, and specializations for this application.** State any assumptions you make.

- The Olympic facilities are divided into sports complexes. Sports complexes are divided into one-sport and multisport types.
- Multisport complexes have areas of the complex designated for each sport with a location indicator (e.g., center, NE corner, and so on).
- A complex has a location, chief organizing individual, total occupied area, and so on.
- Each complex holds a series of events (e.g., the track stadium may hold many different races).
- For each event there is a planned date, duration, number of participants, number of officials, and so on.
- A roster of all officials will be maintained together with the list of events each official will be involved in.
- Different equipment is needed for the events (e.g., goal posts, poles, parallel bars) as well as for maintenance.
- The two types of facilities (one-sport and multisport) will have different types of information.
- For each type, the number of facilities needed is kept, together with an approximate budget.



Chapter 4 An Introduction to SQL

The name SQL is presently expanded as Structured Query Language. Originally, SQL was called SEQUEL (Structured English QUery Language) and was designed and implemented at IBM Research as the interface for an experimental relational database system called SYSTEM R. SQL is now the standard language for commercial relational DBMSs. The standardization of SQL is a joint effort by the American National Standards Institute (ANSI) and the International Standards Organization (ISO), and the first SQL standard is called SQL-86 or SQL1. A revised and much expanded standard called SQL-92 (also referred to as SQL2) was subsequently developed.

To design an efficient query in the implementation phase, you must learn all about the SQL (Structured Query Language).

SQL has many commands available and they can be grouped as follows:

1. Data Definition Language (DDL)

- CREATE
- ALTER
- DROP
- RENAME

2. SELECT Statement

The select statement has the general form:

```
SELECT < field-list >  
FROM < table-list>  
WHERE < condition>
```

3. Data Manipulation Language (DML)

- INSERT
- UPDATE
- DELETE

4. Data Control Language (DCL)

- GRANT
- REVOKE

5. Data Transaction Language (DTL)

- COMMIT
- ROLLBACK
- SAVE POINT



4.1 Data Definition in SQL

The SQL commands for data definition are CREATE, ALTER, DROP and RENAME. An SQL schema is identified by a schema name and includes an authorization identifier to indicate the user or account who owns the schema, as well as descriptors for each element in the schema.

A schema is created via the CREATE SCHEMA statement, which can include all the schema elements' definitions. For example, the following statement creates a schema called COMPANY, owned by the user with authorization identifier JSMITH:

```
CREATE SCHEMA COMPANY AUTHORIZATION JSMITH;
```

4.1.1 CREATE TABLE in SQL

The CREATE TABLE command is used in SQL for:

- Table Creation
- SQL Data Types' definition
- SQL Constraints

The **CREATE TABLE** command is used to specify a new table (relation) by giving it a name and specifying its attributes and constraints. The attributes are specified first; and each attribute is giving a name, a data type to specify its domain of values, and possibly some constraints. The key, entity integrity, and referential integrity constraints are then specified. Figure (2) shows sample data definition statements in SQL for the relational database schema shown in Figure 4.1.

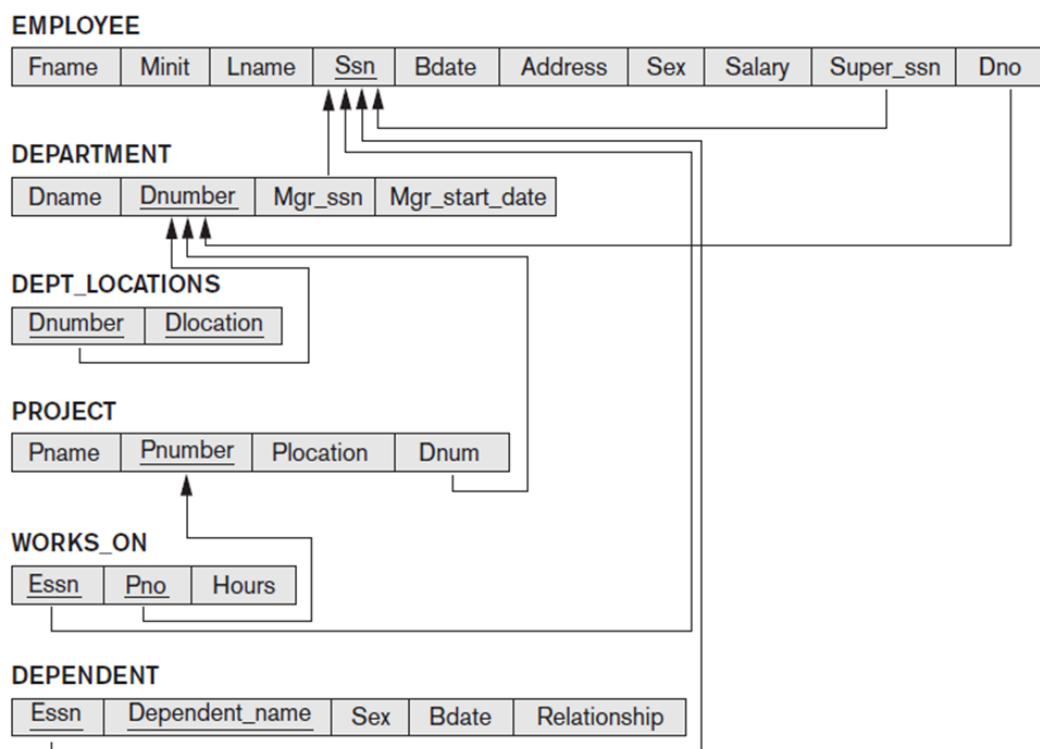


Figure 4.1



```
CREATE TABLE EMPLOYEE
(
    FNAME          VARCHAR(15)    NOT NULL,
    MINIT          CHAR,
    LNAME          VARCHAR (15)    NOT NULL,
    SSN            CHAR (9)        NOT NULL,
    BDATE          DATE
    ADDRESS        VARCHAR (30),
    SEX            CHAR,
    SALARY         DECIMAL (10,2)
    SUPERSSN       CHAR (9),
    DNO            INT             NOT NULL,
    PRIMARY KEY (SSN),
    FOREIGN KEY (SUPERSSN) REFERENCES EMPLOYEE (SSN),
    FOREIGN KEY (DNO) REFERENCES DEPARTMENT (DNUMBER) );
```

```
CREATE TABLE DEPARTMENT
( DNAME          VARCHAR(15)    NOT NULL,
  DNUMBER        INT            NOT NULL,
  MGRSSN        CHAR (9)        NOT NULL,
  MGRSTARTDATE  DATE,
  PRIMARY KEY (DNUMBER),
  UNIQUE (DNAME),
  FOREIGN KEY (MGRSSN) REFERENCES EMPLOYEE (SSN) );
```

```
CREATE TABLE DEPT_LOCATIONS
( DNUMBER        INT            NOT NULL,
  DLOCATION       VARCHAR (15)    NOT NULL,
  PRIMARY KEY (DNUMBER, DLOCATION)
  FOREIGN KEY (DNUMBER)REFERENCESDEPARTMENT(DNUMBER) );
```

```
CREATE TABLE PROJECT
( PNAME          VARCHAR(15)    NOT NULL,
  PNUMBER        INT            NOT NULL,
  PLOCATION       VARCHAR (15),
  DNUM           INT            NOT NULL,
  PRIMARY KEY (PNUMBER),
  UNIQUE (PNAME),
  FOREIGN KEY (DNUM) REFERENCES DEPARTMENT(DNUMBER) );
```



```
CREATE TABLE WORKS_ON
( ESSN          CHAR(9)          NOT NULL,
  PNO           INT              NOT NULL,
  HOURS         DECIMAL (3,1)    NOT NULL,
  PRIMARY KEY (ESSN, PNO),
  FOREIGN KEY (ESSN) REFERENCES EMPLOYEE(SSN) ,
  FOREIGN KEY (PNO) REFERENCES PROJECT(PNUMBER) );
```

```
CREATE TABLE DEPENDENT
( ESSN          CHAR(9)          NOT NULL,
  DEPENDENT_NAME VARCHAR (15)    NOT NULL,
  SEX           CHAR,
  BDATE        DATE,
  RELATIONSHIP  VARCHAR (8),
  PRIMARY KEY (ESSN, DEPENDENT_NAME)
  FOREIGN KEY (ESSN) REFERENCES EMPLOYEE(SSN) );
```

The schema designer can specify the action to be taken if a referential integrity constraint is violated upon deletion of a referenced tuple or upon modification of a referenced primary key value, by attaching a referential triggered action clause to any foreign key constraint. The options include ***SET NULL***, ***CASCADE***, and ***SET DEFAULT***.

An option must be qualified with either ***ON DELETE*** or ***ON UPDATE***.

```
CREATE TABLE EMPLOYEE
(
  ...,
  DNO      INT NOT NULL DEFAULT 1,
  CONSTRAINT EMPPK
  PRIMARY KEY (SSN),
  CONSTRAINT EMPSUPERFK
  FOREIGN KEY (SUPERSSN) REFERENCES EMPLOYEE (SSN)
    ON DELETE SET NULL    ON UPDATE CASCADE,
  CONSTRAINT EMPDEPTFK
  FOREIGN KEY (DNO) REFERENCES DEPARTMENT (DNUMBER)
    ON DELETE SET DEFAULT  ON UPDATE CASCADE);
```



```
CREATE TABLE DEPARTMENT
(
    ...,
    MGRSSN    CHAR(9) NOT NULL DEFAULT "888665555" .
    ...,
CONSTRAINT DEPTPK
PRIMARY KEY (DNUMBER),
CONSTRAINT DEPTSK
UNIQUE (DNAME),
CONSTRAINT DEPTMGRFK
FOREIGN KEY (MGRSSN) REFERENCES EMPLOYEE(SSN)
ON DELETE SET DEFAULT ON UPDATE CASCADE);
```

```
CREATE TABLE DEPT_LOCATIONS
(
    ...,
PRIMARY KEY (DNUMBER, DLOCATION),
FOREIGN KEY (DNUMBER) REFERENCES DEPARTMENT(DNUMBER)
ON DELETE CASCADE ON UPDATE CASCADE);
```

4.1.2 DROP SCHEMA and DROP TABLE Commands

If a whole schema is not needed any more, the ***DROP SCHEMA*** command can be used:

```
DROP SCHEMA COMPANY CASCADE;
```

If the **RESRICT** option is chosen in place of **CASCADE**, the schema is dropped only if it has no elements in it; otherwise, the **DROP** command will not be executed. We can get rid of the **DEPENDENT** relation by issuing the command:

```
DROP TABLE DEPENDENT CASCADE;
```

If the **RESTRIC**T option is chosen instead of **CASCADE**, a table is dropped only if it not referenced in any constraints (such as by foreign key definitions in another relation) or views. With the **CASCADE** option, all such constraints and views that reference the table are dropped automatically from the schema, along with the table itself.

4.1.3 The ALTER TABLE Command

The definition of a base table can be changed by using the **ALTER TABLE** command. The possible alter table actions include adding or dropping a column (attribute), changing a column definition, and adding or dropping table constraints. For example, to add an attribute for keeping track of employee's jobs to the **EMPLOYEE** base relations in the **COMPANY** schema, we can use the command:



```
ALTER TABLE COMPANY.EMPLOYEE ADD JOB VARCHAR (12);
```

If no **DEFAULT** clause is specified, the new attribute will have **NULL** in all the tuples of the relation immediately after the command is executed; hence, the **NOT NULL** constraint is not allowed in this case. To drop a column, one must choose either **CASCADE** or **RESTRICT** for drop behavior. If **CASCADE** is chosen, all constraints and views the reference the column are dropped automatically from the schema, along with the column. If **RESTRICT** is chosen, the command is successful only if no views or constraints reference the column. For example:

```
ALTER TABLE COMPANY.EMPLOYEE DROP ADDRESS CASCADE;
```

It is also possible to alter a column definition by dropping an existing default clause or by defining a new default clause. The following examples illustrate this clause:

```
ALTER TABLE COMPANY.DEPARTMENT  
ALTER MGRSSN DROP DEFAULT;
```

```
ALTER TABLE COMPANY.DEPARTMENT  
ALTER MGRSSN SET DEFAULT "333445555";
```

4.2 Queries in SQL (SELECT Statement)

SQL has one basic statement for retrieving information from a database: the SELECT statement. There are many options and flavors to the SELECT statement in SQL so we will introduce its feature gradually. Also, we will use example queries specified on the schema of the Company Problem demonstrated in Figure (1).

4.2.1 Basic SQL Queries

The basic form of the SELECT statement, is formed of the three clauses SELECT, FROM, and WHERE and has the following form:

```
SELECT < attribute list >  
FROM < table list >  
WHERE < condition >
```

where:

- < attribute list > is a list of attribute names whose values are to be retrieved by the query.
- < table list > is a list of the table names required to process the query.
- < condition > is a conditional (Boolean) expression that identifies the tuples (record data) to be retrieved by the query.



Example Query 1

Retrieve the birthdate and address of the employee whose name is 'Jone B.Smith'.

```
SELECT BDATE, ADDRESS
FROM EMPLOYEE
WHERE FNAME='Jone' AND MINIT='B' AND LNAME='Smith';
```

Example Query 2

Retrieve the name and address of all employees who work for the '*Research*' department

```
SELECT FNAME, MINIT, LNAME, ADDRESS
FROM EMPLOYEE, DEPARTMENT
WHERE DNAME='Research' AND DNUMBER=DNO;
```

In this query, we have two condition types:

- **Selection Condition** (DNAME = 'Research') selects tuples (records) from one table.
- **Join Condition** (DNUMBER=DNO), which joins two tables (EMPLOYEE, DEPARTMENT).

Remark:

any number of select and join conditions may be specified in a single SQL query.

Example Query 3

For every project located in 'Stafford': list the project number, the controlling department, and the department manager's last name, address and birthdate.

```
SELECT PNUMBER, DNUM, LNAME, ADDRESS, BDATE
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE PLOCATION='Stafford' AND DNUM =DNUMBER AND MGRSSN=SSN;
```

The join condition DNUM =DNUMBER relates a project to its controlling department, where the join condition MGRSSN=SSN relates the controlling department to the employee who manages that department.



4.2.2 Dealing with Ambiguous Attribute Names

CASE 1

In SQL we can use the same name for two (or more) attributes as long as the attributes are in different tables. If this the case, and a query refers to two or more attributes with the same name, we need to qualify the attribute name with the table name to prevent ambiguity.

Prefixing the table name to the attribute name and separating the two by a period perform the qualification.

To illustrate this, suppose that the attributes **DNO** and **LNAME** of the **EMPLOYEE** table were called **DNUMBER** and **NAME** respectively and at the same time the attribute **DNAME** of the **DEPARTMENT** table was called **NAME**. In fact, these changes create ambiguity. So, Query 1 will be rewritten as follows:

```
SELECT FNAME, MINIT, EMPLOYEE.NAME, ADDRESS
FROM EMPLOYEE, DEPARTMENT
WHERE DEPARTMENT.NAME='Research' AND DEPARTMENT.DNUMBER =
EMPLOYEE.DNUMBER;
```

Another Solution:

This solution is based on the use aliases i.e. creation of copies from a table in a query. So, Query 1 will be rewritten as follows:

```
SELECT E.FNAME, E.MINIT, E.NAME, E.ADDRESS
FROM EMPLOYEE E, DEPARTMENT D
WHERE D.NAME='Research' AND D.DNUMBER = E.DNUMBER;
```

Remark:

E is a copy from EMPLOYEE table and D is a copy from DEPARTMENT table

CASE 2

Ambiguity also arises in the case of queries that refer to the same table twice (recursive relations) as in the following example:

Example Query 4

For each employee, retrieve the employee's first and last name and first and last name of his or her immediate supervisor.

```
SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME
FROM EMPLOYEE E S
WHERE E.SUPERSSN=S.SSN;
```



4.2.3 Unspecified WHERE Clauses and Use of '*'

A missing where clause indicates no condition on selection criteria as in the following query:

Example Query 5

For each employee, retrieve all the employee's SSN

```
SELECT SSN  
FROM EMPLOYEE;
```

To retrieve all the attribute values of the selected tuples (record), we do not have to explicitly list the attribute names but we just specify an asterisk (*). So, we can write the following query:

Example Query 6

Retrieve all the attribute values of EMPLOYEE records (tuples) who work in department number 5

```
SELECT *  
FROM EMPLOYEE  
WHERE DNO = 5;
```

To retrieve all the attributes of an EMPLOYEE and the attributes of the DEPARTMENT he works in for every employee work in the 'Research' department, we can write the following query:

Example Query 7

Retrieve all the attributes of all employees who work for the '*Research*' department

```
SELECT *  
FROM EMPLOYEE, DEPARTMENT  
WHERE DNAME='Research' AND DNUMBER=DNO;
```

4.2.4 Tables as Sets in SQL

SQL does not automatically eliminate duplicate tuples in the results of queries for the following reasons:

- Duplicate elimination is an expensive operation from the execution time point of view.
- User may want to see duplicate tuples in the result of a query.
- When an aggregate function (see the following sections) is applied to tuples, in most cases we do not want to eliminate duplicates.



In SQL, to eliminate duplicate tuples from the result of a query we use the key word **DISTINCT** in the select clause. For example, Query 8 retrieves the salary of every employee; if several of employees have the same salary that salary value will appear as many times as possible in the result of the query. IF we are interested only in distinct salary values, we want each value to appear once, regardless of how many of the employees earn that salary as demonstrated in Query 9.

Example Query 8

Retrieve the salaries of all employees.

```
SELECT SALARY
FROM EMPLOYEE;
```

Example Query 9

Retrieve the salary set of employees.

```
SELECT DISTINCT SALARY
FROM EMPLOYEE;
```

4.2.5 Set Operations

SQL has directly incorporated some of the set operations of the relational algebra. There is a set union operations (**UNION**), and in some versions of SQL there are also set difference (**MINUS**) and set intersection (**INTERSECT**) operations.

The resulting relations of these set operations are sets of tuples; that is, duplicate tuples are eliminated from the result. Because the set operations apply only to union compatible relations, we must make sure that the two relations on which we apply the operation have the same attributes and the attributes appear in the same order in both relations. The next example illustrates the use of **UNION**.

Example Query 10:

Make a list of all project names for projects that involve an employee whose last name is 'Smith' as a worker or as a manager.

```
(SELECT PNAME
FROM EMPLOYEE, DEPARTMENT, PROJECT
WHERE LNAME = 'Smith' AND SSN = MGRSSN AND DNUM = DNUMBER)
UNION
(SELECT PNAME
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE LNAME = 'Smith' AND SSN = ESSN AND PNO = PNUMBER)
```



Remark:

Notice that if several employees have the last name 'Smith', the project names involving any of them would be retrieved but applying the UNION operation on the two SELECT queries eliminates duplicated project names.

4.2.6 Nested Queries and Set Comparisons

Some queries require that existing values in the database be fetched and then used in a comparison condition. Such queries can be conveniently formulated in nested queries, which are complete SELECT queries within the WHERE clause of another query. That another query is called the outer query. Query 9 can be rephrased to use nested queries as follows:

Example Query 11

Make a list of all project names for projects that involve an employee whose last name is 'Smith' as a worker or as a manager.

```
SELECT PNAME
FROM PROJECT
WHERE PNUMBER IN ( SELECT PNUMBER
                    FROM EMPLOYEE, DEPARTMENT, PROJECT
                    WHERE LNAME = 'Smith' AND SSN = MGRSSN
                    AND DNUM = DNUMBER)
OR
PNUMBER IN ( SELECT PNO
            FROM EMPLOYEE, WORKS_ON
            WHERE LNAME = 'Smith' AND SSN = ESSN )
```

Remarks:

- Notice that the outer query selects a project tuple if the PNUMBER value of that tuple is in the result of either nested query. The comparison operator **IN** compares the value of the PNUMBER with the values selected by the inner query and evaluates to TRUE if PNUMBER value matches one of these selected values.
- The outer query is uncorrelated with either of the inner queries. In nested queries potential ambiguity of attributes, may arise as in the following example:



Example Query 12

Retrieve the name of each employee who has a dependent with the same first name and same sex as the employee.

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE E
WHERE E.SSN IN ( SELECT ESSN
                  FROM DEPENDENT
                  WHERE ESSN = E.SSN AND
                        E.FNAME = DEPENDENT_NAME AND SEX = E.SEX )
```

Remarks:

- a) In this example, the outer query is correlated with the inner queries. two queries are said to be correlated, whenever a condition in the **WHERE** clause of a nested query references some attribute of a relation declared in the outer query. We can understand a correlated query better by considering that the nested query is evaluated once for each tuple in the outer query.

For example, we can think of Query 12 as follows:

For each **EMPLOYEE** tuple, evaluate the nested query, which retrieves the ESSN values for all **DEPENDENT** tuples with the same social security number, sex, and name as the **EMPLOYEE** tuple; if the **SSN** value of the **EMPLOYEE** tuple is in the result of the nested query, then select that **EMPLOYEE** tuple.

- b) Notice that we need **SSN = ESSN** condition in the **WHERE** clause of the nested query; without this condition, we would select employees whose first name and sex match those of any dependent.

Example Query 13

This example is concerned with the **CONTAINS** comparison operator which compares two sets. This operator returns true if one set contains all values in the other set.

For example, retrieve the name of each employee who works on all the projects controlled by department number 5:

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE ( (SELECT PNO FROM WORKS_ON WHERE SSN = ESSN)
        CONTAINS
        (SELECT PNUMBER FROM PROJECT WHERE DNUM = 5) )
```

Remarks:

- a) **CONTAINS** is not part of the standard SQL, but it is supported in SQL Server & Oracle.



- b) For each employee tuple, the first nested query, which is correlated with the outer query, retrieves the project numbers on which the employee works; if these contains all projects retrieved by the second uncorrelated nested query the employee tuple is selected.

4.2.7 The EXISTS functions in SQL

The **EXISTS** function in SQL check whether the result of a correlated nested query is empty or not. We illustrate the use of **EXISTS** and the **NOT EXISTS** with some examples. Query 12 can be reformulated using the **EXISTS** function as follows:

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE E
WHERE EXISTS ( SELECT *
                FROM DEPENDENT
                WHERE ESSN = E.SSN AND
                      E.FNAME = DEPENDENT_NAME AND SEX = E.SEX)
```

We can think in this query as follows:

For each **EMPLOYEE** tuple, evaluate the nested query, which retrieves all **DEPENDENT** tuples with the same social security number, sex, and name as the **EMPLOYEE** tuple; if at least one tuple **EXISTS** in the result of the nested query, then select that **EMPLOYEE** tuple. The **NOT EXISTS** function returns true if there are no tuples in the result of the nested query.

Example Query 14

Retrieve the name of each employee who has no dependent related to him.

```
(SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE NOT EXISTS (SELECT * FROM DEPENDENT WHERE SSN = ESSN )
```

Example Query 15

List the names of managers who have at least one dependent.

```
(SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE EXISTS (SELECT *
                FROM DEPENDENT
                WHERE SSN = ESSN
                AND
                EXISTS (SELECT *
                        FROM DEPARTMENT
                        WHERE SSN = MGRSSN )
```



Remarks:

- a) The two nested queries are correlated to the outer query i.e. for each employee tuple the nested queries are executed.
- b) If at least one tuple of the first nested query and at least one tuple of the second nested query exists, we select the employee tuple.

4.2.8 The Explicit Sets and NULLS in SQL

We have seen several queries with a nested query in the **WHERE** clause. It is also possible to use an explicit set of values in the **WHERE** clause rather than a nested query. Such set is enclosed in parentheses in SQL.

Example Query 16

Retrieve the social security numbers of all employees who work on project number 1,2 or 3

```
SELECT DISTINCT ESSN
FROM WORKS_ON
WHERE PNO IN (1, 2, 3);
```

SQL allows queries that check if a value is **NULL** (any field has a **NULL** value, means that the value of this field is missing or undefined or not applicable). SQL uses the **IS** and **IS NOT** key words to check if a field has or has not a **NULL** value, rather than using = or <> to compare an attribute to **NULL**. That is because SQL considers each **NULL** value distinct from other **NULL** values. It follows that when a join condition operation is specified on tuples with a **NULL** value for the join attributes are not included in the result.

Example Query 17

Retrieve the names of all employees who do not have supervisors.

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE SUPERSSN IS NULL;
```

4.2.9 Aggregate Functions and Grouping in SQL

Grouping and aggregation are required in many database applications. So, **SQL** has features that incorporate these concepts. In **SQL** there many aggregate functions like **COUNT**, **SUM**, **MAX**, **MIN**, and **AVG**.

These functions can be used in the **SELECT** clause only or in a **HAVING** clause that we will introduce below.



Example Query 18

List the sum of the salaries of all employees, the maximum salary, the minimum salary and the average salary.

```
SELECT SUM(SALARY), MAX(SALARY), MIN(SALARY), AVG(SALARY)
FROM EMPLOYEE
```

If we want to limit the number of tuples on which the aggregate function is to be applied, we must write a condition in the SELECT clause.

The following query calculates the aggregate function values for the employees of the 'Research' department only:

```
SELECT SUM(SALARY), MAX(SALARY), MIN(SALARY), AVG(SALARY)
FROM EMPLOYEE, DEPARTMENT
WHERE DNO=DNUMBER AND DNAME='Research'
```

Example Query 19

Retrieve the total number of employees in the company.

```
SELECT COUNT (*)
FROM EMPLOYEE
```

Example Query 20

Retrieve the total number of employees in the 'Research' department.

```
SELECT COUNT (*)
FROM EMPLOYEE, DEPARTMENT
WHERE DNO=DNUMBER AND DNAME='Research';
```

Here the asterisk (*) refers to the number of rows (tuples), so **COUNT (*)** returns the number of rows in the result of the query. We may also use the COUNT function to count values in a column rather than tuples. For example, to count the number of distinct salary values in the database, we write query19.

Example Query 21

```
SELECT COUNT (DISTINCT SALARY)
FROM EMPLOYEE
```

Notice that if we write **COUNT(SALARY)** instead of **COUNT (DISTINCT SALARY)** in query 21, we get the same result as **COUNT (*)** because duplicates would not be eliminated. The above examples show how functions are applied to the result of a query in some cases, we may need to use functions to select particular tuples. In such cases, we must specify a correlated nested query with the desired function and use that nested query in the **WHERE** clause of the outer query.



Example Query 22

Retrieve the names of all employees who have two or more dependents.

```
SELECT LNAME, FNAME
FROM EMPLOYEE
WHERE (    SELECT COUNT (*)
          FROM DEPENDANT
          WHERE SSN = ESSN ) >= 2
```

The correlated nested query counts the number of dependents that each employee has; if this is greater than or equal to 2, the employee tuple is selected. In many cases we want to apply the aggregate functions to subgroups of tuples in a relation in which the tuple grouping is performed according to the value of an attribute or a set of attributes. *SQL* has **GROUP BY** clause for this purpose. The **GROUP BY** clause specifies the grouping attributes, which must also appear in the **SELECT** clause, so that the value of applying each function on the group of tuples appears along with the value of the grouping attribute(s).

Example Query 23

For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
SELECT DNO, COUNT (*), AVG(SALARY)
FROM EMPLOYEE
GROUP BY DNO
```

In query 23 the employee tuples are divided into groups according to the department number **DNO**. The **COUNT** (and the **AVG** are applied to each group of tuples separately.

Example Query 24

For each project retrieve the project number, project name and number of employees who work on that project.

```
SELECT PNUMBER, PNAME, COUNT (*)
FROM PROJECT, WORKS_ON
WHERE PNUMBER=PNO
GROUP BY PNUMBER, PNAME
```

Query 24 shows how we can use a join condition in conjunction with **GROUP BY**. In this case, the grouping and functions are applied after the joining of the two relations. Sometimes we want to retrieve the values of these functions for only those groups that satisfy certain conditions.

For example, suppose we want to modify query 24 so that only projects with more than two employees appear in the result. *SQL* provides a **HAVING** clause, which can



appear only in conjunction with **GROUP BY** clause. The **HAVING** clause provides condition on the group of tuples associated with each value of the grouping attributes.

Example Query 25

For each project (which has more than 2 employees works on it) retrieve the project number, project name and number of employees who work on that project.

```
SELECT PNUMBER, PNAME, COUNT (*)
FROM PROJECT, WORKS_ON
WHERE PNUMBER=PNO
GROUP BY PNUMBER, PNAME
HAVING COUNT (*) > 2
```

Example Query 26

For each project retrieve the project number, project name and number of employees from department 5 who work on that project.

```
SELECT PNUMBER, PNAME, COUNT (*)
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE PNUMBER=PNO AND ESSN=SSN AND DNO=5
GROUP BY PNUMBER, PNAME
```

In this query the tuples in each group are restricted to those satisfying the condition of the **WHERE** clause.

Example Query 27

Suppose that we want to count the total number of employees with salaries greater than \$40,000 who work in each department, but only for those departments where more than five employees work. Suppose we write the following query:

```
SELECT DNAME, COUNT (*)
FROM DEPARTMENT, EMPLOYEE
WHERE DNUMBER=DNO AND SALARY > 40000
GROUP BY DNAME
HAVING COUNT (*) > 5
```

This is incorrect because it will select only departments that have more than five employees who each earn more than \$40,000.

The rule is that the **WHERE** clause is executed first to select individual tuples; the **HAVING** clause is applied later to select individual groups of tuples. Hence, the tuples are already restricted to employees earning more than \$40,000 before the function in the **HAVING** clause is applied. One way to write the query correctly is to use a nested query as shown in the following query:



```

SELECT DNAME, COUNT (*)
FROM DEPARTMENT, EMPLOYEE
WHERE DNUMBER=DNO AND SALARY > 40000 AND
      DNO IN (SELECT DNO
              FROM EMPLOYEE
              GROUP BY DNO HAVING COUNT (*) > 5)
GROUP BY DNAME

```

4.2.10 Sub-string Comparisons, Arithmetic Operators, and Ordering

SQL allows comparison conditions on only parts of a character string, using the **LIKE** comparison operator. Partial strings are specified using two reserved characters; '%' (or '*' in some implementations) replaces an arbitrary number of characters and '_' replaces a single arbitrary character. For example, to retrieve all employees whose address is in Houston, Texas, we can use query 28:

Example Query 28

Retrieve the names of all employees' tuples whose address value includes the sub-string 'Houston, TX'.

```

SELECT LNAME, FNAME
FROM EMPLOYEE
WHERE ADDRESS LIKE '%Houston, TX%'

```

Example Query 29

Retrieve the names of all employees' tuples who were born during 1950's.

```

SELECT LNAME, FNAME
FROM EMPLOYEE
WHERE BDATE LIKE '_____5_'

```

Here, '5' must be the eighth character of the string (according to our date format), so we use the value '_____5_', with each underscore as a placeholder for an arbitrary character. Also, **SQL** allows the use of arithmetic in queries. The standard arithmetic operators '+', '-', '*', and '/' can be applied to numeric values in a query result.

Example Query 30

Suppose we want to see the effect of giving all employees who work on the 'ProductX' project a 10% raise.

```

SELECT LNAME, FNAME, 1.1 * SALARY
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE SSN=ESSN AND PNO=PNUMBER AND PNAME='ProductX'

```



Finally, **SQL** is a practical language meant for generating listings and reports, it allows the user to order the tuples in the result of a query by the values of one or more attributes using the **ORDER BY** clause.

Example Query 31

Suppose we want to retrieve a list of employees and the projects each works in, but we want the list ordered by the employee's department and within each department ordered alphabetically by names.

```
SELECT DNAME, LNAME, FNAME, PNAME
FROM DEPARTMENT, EMPLOYEE, WORKS_ON, PROJECT
WHERE DNUMBER=DNO AND SSN=ESSN AND PNO=PNUMBER
ORDER BY DNAME, LNAME, FNAME
```

The default order is in ascending order of values. We can specify the keyword **DESC** if we want a descending order of values. The keyword **ASC** can be used to specify ascending order explicitly, even though it is the default. If we want a descending order on **DNAME** and ascending order on **LNAME**, **FNAME**, the **ORDER BY** clause becomes:

```
ORDER BY DNAME DESC, LNAME ASC, FNAME ASC
```

4.3 Data Manipulation Language (DML)

In **SQL** there is three commands to modify the database:

1. **INSERT**
2. **DELETE**
3. **UPDATE**

4.3.1 The **INSERT** Command

The **INSERT** command is used to add a single tuple to a table and the values to be added should be listed in the same order as the corresponding attributes were specified in the **CREATE TABLE** command.

Example 1

```
INSERT INTO EMPLOYEE
```

VALUES ('Richard','k','Marrini', '653298765', '30-DEC-52', '98 Oak Forest, Katy, TX', 'M', 37000, '763487961', 4)

The second form of the **INSERT** statement allows specifying explicit attribute names that correspond to the values in the **INSERT** command. In this case, attributes with **NULL** values can be left out.



Example 2

```
INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
VALUES ('Richard', 'Marrini', '653298765')
```

SQL will reject an **INSERT** command in which an attribute declared to be **NOT NULL** does not have a value.

Example 3

```
INSERT INTO EMPLOYEE (FNAME, LNAME, DNO)
VALUES ('Richard', 'Marrini', 2)
```

The above **INSERT** command will be rejected because the **SSN** value can not be **NULL**. Finally, a variation of the **INSERT** command allows us to insert multiple tuples in a table. This is used mainly in conjunction with creating a table and loading it with the result of a query.

Example 4

```
CREATE TABLE DEPTS-INFO ( DEPT-NAME VARCHAR(15), NO_OF_EMPS INTEGER,
TOTAL_SAL INTEGER);
```

```
INSERT INTO DEPTS_INFO (DEPT_NAME, NO_OF_EMPS, TOTAL_SAL)
SELECT DNAME, COUNT(*), SUM(SALARY)
FROM DEPARTMENT, EMPLOYEE
WHERE DNUMBER=DNO
GROUP BY DNAME;
```

Note that the **DEPTS_INFO** table may not be up to date; that is, if we update either the **DEPARTMENT** or **EMPLOYEE** tables after issuing the previous **INSERT** command, the information in **DEPTS_INFO** becomes outdated. To keep such a table up to date, we must create a **VIEW**, which will be explained later.

4.3.2 The DELETE Command

The **DELETE** COMMAND removes tuples from a table. Tuples are deleted from only one table at a time. A missing **WHERE** clause specifies that all tuples in the table are to be deleted; however, the table remains in the database as an empty table.

Example 1

Delete all employee tuples with last name 'Brown':

```
DELETE FROM EMPLOYEE
WHERE LNAME = 'Brown'
```



Example 2

Delete the employee tuple with SSN = '123456789' :

```
DELETE FROM EMPLOYEE
WHERE SSN='123456789'
```

Example 3

Delete all employee tuples who work in the 'Research' department:

```
DELETE FROM EMPLOYEE
WHERE DNO IN ( SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME ='Research')
```

Example 4

Delete all employee tuples in the EMPLOYEE table:

```
DELETE FROM EMPLOYEE
```

4.3.3 The UPDATE Command

The **UPDATE** Command is used to modify attribute values of one or more selected tuples from a single table. An additional **SET** clause specifies the attributes to be modified and their new values.

Example 1

Change the location and controlling department number of project number 10 to be 'Bellaire' and 5 respectively.

```
UPDATE PROJECT
SET PLOCATION = 'Bellaire', DNUM = 5
WHERE PNUMBER = 10
```

Example 2

Give all employees in the 'Research' department a 10% raise in salary.

```
UPDATE EMPLOYEE
SET SALARY = SALARY * 1.1
WHERE DNO IN (SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME ='Research')
```



4.4 Data Transaction Language (DTL)

The DTL refers to the capability of relational DBMS to perform Transactions. Transactions are units of work that must be done in a logical order and successfully as group or not at all. Transactions have a beginning and end if anything goes wrong during transactions the entire unit of work can be cancelled if desired.

To handle Transactions there are three commands:

- **Rollback:** it fails transactions until last commit.
- **Commit:** it performs transaction.
- **Save Point:** it creates a label that both **Commit**, and **Rollback** are done until it.

```
UPDATE EMPLOYEE SET ENAME = 'Hany'  
WHERE ENAME = 'Ahmed';
```

```
SAVEPOINT pp;
```

```
SELECT ENAME FROM EMPLOYEE
```

```
UPDATE EMPLOYEE SET ENAME = 'Aly'  
WHERE ENAME = 'Hany'
```

```
ROLLBACK TO pp;
```

```
SELECT ENAME FROM EMPLOYEE
```

```
ROLLBACK;
```

```
SELECT ENAME FROM EMPLOYEE
```

Remark:

The result is that the field **ENAME** has the value 'Ahmed'.



Exercises

Student Name:

Student ID:

In SQL, specify the following queries on the COMPANY database.

- a) Retrieve the names of all employees who work in the department that has the employee with the highest salary among all employees.
- b) Retrieve the names of all employees whose supervisor's supervisor has '888665555' for Ssn.
- c) Retrieve the names of employees who make at least \$10,000 more than the employee who is paid the least in the company.
- d) A view that has the department name, manager name, and manager salary for every department
- e) A view that has the employee's name, supervisor name, and employee salary for each employee who works in the 'Research' department.
- f) A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project.
- g) A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project *with more than one employee working on it*.



Chapter 5 Complex Queries & Active Databases

This chapter describes more advanced features of the SQL language for relational databases. In Chapter 4, we described some basic types of retrieval queries in SQL. Because of the generality and expressive power of the language, there are many additional features that allow users to specify more complex retrievals from the database.

5.1 SQL Implementations

There are many SQL Implementations, the most popular Implementations are:

- **Standard SQL** is the standard language to query a database.
- **PL SQL** basically stands for "Procedural Language extensions to SQL." This is the extension of Structured Query Language (SQL) that is used in Oracle.
- **T-SQL** basically stands for "Transact-SQL." This is the extension of Structured Query Language (SQL) that is used in Microsoft.

5.1.1 Difference between SQL and T-SQL

SQL	T-SQL
SQL is a programming language which focuses on managing relational databases.	T-SQL is a procedural extension used by SQL Server.
This is used for controlling and manipulating data where large amounts of information are stored about products, clients, etc.	T-SQL has some features that are not available in SQL. Like procedural programming elements and a local variable to provide more flexible control of how the application flows.
SQL queries submitted individually to the database server.	T-SQL writes a program in such a way that all commands are submitted to the server in a single go
The syntax was formalized for many commands; some of these are SELECT, INSERT, UPDATE, DELETE, CREATE, and DROP.	It also includes special functions like the converted date () and some other functions which are not part of the regular SQL.



5.1.2 Difference Between T-SQL and PL-SQL

T-SQL	PL-SQL
• T-SQL is a Microsoft product. • Full Form of TL SQL is Transact Structure Query language. • T-SQL gives a high degree of control to programmers. • T-SQL performs best with Microsoft SQL server • It is easy and simple to understand. • T-SQL allows inserting multiples rows into a table using the BULK INSERT statement. • SELECT INTO statement used in T-SQL • In T-SQL NOT EXISTS clause used along with SELECT statements.	• PL-SQL is developed by Oracle. • Full Form of PL SQL is Procedural Language Structural Query Language. • It is a natural programming language that blends easily with the SQL • PL-SQL performs best with Oracle database server. • PL-SQL is complex to understand. • PL/SQL supports oops concepts like data encapsulation, function overloading, and information hiding. • INSERT INTO statement must be used in PL/SQL • In PL/SQL, there is a MINUS operator, which could be used with SELECT statements

5.1.3 Varchar vs nVarchar

nvarchar stores UNICODE data. If you have requirements to store UNICODE or multilingual data, nvarchar is the choice. Varchar stores ASCII data and should be your data type of choice for normal use. Regarding memory usage, nvarchar uses 2 Bytes per character, whereas varchar uses 1 Byte.

JOIN-ing a VARCHAR to NVARCHAR has a considerable performance hit. Some experts recommend nvarchar always because: since all modern operating systems and development platforms use Unicode internally, using nvarchar rather than varchar, will avoid encoding conversions every time you read from or write to the database.



5.2 Joined Tables in SQL

The concept of a **joined table** (or **joined relation**) was incorporated into SQL to permit users to specify a table resulting from a join operation *in the FROM clause* of a query. This construct may be easier to comprehend than mixing all the select and join conditions in the WHERE clause. For example, consider Query 1, which retrieves the name and address of every employee who works for the ‘Research’ department. It may be easier to specify the join of the EMPLOYEE and DEPARTMENT relations in the WHERE clause, and then to select the desired tuples and attributes. This can be written in SQL as in Query 1A:

Query 1A:

```
SELECT Fname, Lname, Address
FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE Dname = 'Research';
```

The FROM clause in Query 1A contains a single *joined table*. The attributes of such a table are all the attributes of the first table, EMPLOYEE, followed by all the attributes of the second table, DEPARTMENT. The concept of a joined table also allows the user to specify different types of joins, such as NATURAL JOIN and various types of OUTER JOIN. In a **NATURAL JOIN** on two relations R and S , no join condition is specified; an implicit *EQUIJOIN condition* for *each pair of attributes with the same name* from R and S is created. Each such pair of attributes is included *only once* in the resulting relation

If the names of the join attributes are not the same in the base relations, it is possible to rename the attributes so that they match, and then to apply NATURAL JOIN. In this case, the AS construct can be used to rename a relation and all its attributes in the FROM clause. This is illustrated in Query 1B, where the DEPARTMENT relation is renamed as DEPT and its attributes are renamed as Dname, Dno (to match the name of the desired join attribute Dno in the EMPLOYEE table), Mssn, and Msdate. The implied join condition for this NATURAL JOIN is EMPLOYEE.Dno = DEPT.Dno, because this is the only pair of attributes with the same name after renaming:

Query 1B:

```
SELECT Fname, Lname, Address
FROM (EMPLOYEE NATURAL JOIN
      (DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))
WHERE Dname = 'Research';
```

The default type of join in a joined table is called an **inner join**, where a tuple is included in the result only if a matching tuple exists in the other relation.



For example, in Query 2A, only employees who *have a supervisor* are included in the result; an EMPLOYEE tuple whose value for Super_ssn is NULL is excluded.

Query 2A:

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM (EMPLOYEE AS E INNER JOIN EMPLOYEE AS S
ON E.Super_ssn = S.Ssn);
```

If the user requires that all employees be included, a different type of join called **OUTER JOIN** must be used explicitly. There are several variations of OUTER JOIN, as we shall see. In the SQL standard, this is handled by explicitly specifying the keyword OUTER JOIN in a joined table, as illustrated in Query 2B:

Query 2B:

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
ON E.Super_ssn = S.Ssn);
```

In SQL, the options available for specifying joined tables include INNER JOIN (only pairs of tuples that match the join condition are retrieved, same as JOIN), LEFT OUTER JOIN (every tuple in the left table must appear in the result; if it does not have a matching tuple, it is padded with NULL values for the attributes of the right table), RIGHT OUTER JOIN (every tuple in the right table must appear in the result; if it does not have a matching tuple, it is padded with NULL values for the attributes of the left table), and FULL OUTER JOIN. In the latter three options, the keyword OUTER may be omitted. If the join attributes have the same name, one can also specify the natural join variation of outer joins by using the keyword NATURAL before the operation (for example, NATURAL LEFT OUTER JOIN). The keyword CROSS JOIN is used to specify the CARTESIAN PRODUCT operation, although this should be used only with the utmost care because it generates all possible tuple combinations.

It is also possible to *nest* join specifications; that is, one of the tables in a join may itself be a joined table. This allows the specification of the join of three or more tables as a single joined table, which is called a **multiway join**. For example, Query 3 is a different way of specifying Query 3 from Chapter 2 using the concept of a joined table:

Query 3:

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM ((PROJECT JOIN DEPARTMENT ON Dnum = Dnumber)
JOIN EMPLOYEE ON Mgr_ssn = Ssn)
WHERE Plocation = 'Stafford';
```



Not all SQL implementations have implemented the new syntax of joined tables. In some systems, a different syntax was used to specify outer joins by using the comparison operators $+$, $=$, $=$, and $+$ for left, right, and full outer join, respectively, when specifying the join condition. For example, this syntax is available in Oracle. To specify the left outer join in Query 2B using this syntax, we could write the query 2C as follows:

Query 2C:

```
SELECT E.Lname, S.Lname
FROM EMPLOYEE E, EMPLOYEE S
WHERE E.Super_ssn + = S.Ssn;
```

5.3 More Complex SQL Retrieval Queries

there are many additional features that allow users to specify more complex retrievals from the database. Such as:

- Nested Queries
- Join Types
- Aggregate Functions
- Grouping

5.3.1 Nested Queries, Tuples, and Set/Multiset Comparisons

Some queries require that existing values in the database be fetched and then used in a comparison condition. Such queries can be conveniently formulated by using nested queries, which are complete select-from-where blocks within another **SQL** query. That another query is called the outer query. These nested queries can also appear in the **WHERE** clause or the **FROM** clause or the **SELECT** clause or other **SQL** clauses as needed.

SQL allows the use of tuples of values in comparisons by placing them within parentheses. To illustrate this, consider the following query:

Query 4

Select the Essns of all employees who work the same (project, hours) combination on some project that employee 'John Smith' (whose Ssn = '123456789') works on.

```
SELECT DISTINCT Essn
FROM WORKS_ON
WHERE (Pno, Hours) IN ( SELECT Pno, Hours
                        FROM WORKS_ON
                        WHERE Essn = '123456789');
```



In this example, the **IN** operator compares the subtuple of values in parentheses (Pno, Hours) within each tuple in **WORKS_ON** with the set of type-compatible tuples produced by the nested query.

In addition to the **IN** operator, several other comparison operators can be used to compare a single value *v* (typically an attribute name) to a set or multiset *V* (typically a nested query). The **= ANY** (or **= SOME**) operator returns **TRUE** if the value *v* is equal to some value in the set *V* and is hence equivalent to **IN**. The two keywords **ANY** and **SOME** have the same effect. Other operators that can be combined with **ANY** (or **SOME**) include **>**, **>=**, **<**, **<=**, and **<>**. The keyword **ALL** can also be combined with each of these operators.

Query 5

Returns the names of employees whose salary is greater than the salary of all the employees in department 5.

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ALL ( SELECT Salary FROM EMPLOYEE WHERE Dno = 5 );
```

In general, we can have several levels of nested queries. We can once again be faced with possible ambiguity among attribute names if attributes of the same name exist—one in a relation in the **FROM** clause of the outer query, and another in a relation in the **FROM** clause of the nested query. The rule is that a reference to an unqualified attribute refers to the relation declared in the innermost nested query.

5.3.2 The EXISTS and UNIQUE Functions in SQL

EXISTS and **UNIQUE** are Boolean functions that return *TRUE* or *FALSE*; hence, they can be used in a **WHERE** clause condition. The **EXISTS** function in **SQL** is used to check whether the result of a nested query is empty (contains no tuples) or not. The result of **EXISTS** is a Boolean value *TRUE* if the nested query result contains at least one tuple, or *FALSE* if the nested query result contains no tuples.

The **SQL** function **UNIQUE(Q)** Returns *TRUE* if there are no duplicate tuples in the result of query *Q*.

Query 6

Retrieve the name of each employee who works on all the projects controlled by department number 5 can be written using **EXISTS** and **NOT EXISTS** in **SQL** systems.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS ( ( SELECT Pnumber FROM PROJECT WHERE Dnum = 5)
EXCEPT ( SELECT Pno FROM WORKS_ON
WHERE Ssn = Essn) );
```



the first subquery (which is not correlated with the outer query) selects all projects controlled by department 5, and the second subquery (which is correlated) selects all projects that the particular employee being considered works on.

If the set difference of the first subquery result MINUS (EXCEPT) the second subquery result is empty, it means that the employee works on all the projects and is therefore selected.

5.3.3 More Subqueries

Query 7

Subquery can be used in **SELECT**.

```
SELECT SalesOrderNumber, SubTotal, OrderDate, ( SELECT SUM(OrderQty)
                                                FROM SalesOrderDetail
                                                WHERE SalesOrderID = 43659 ) AS TotalQuantity
FROM SalesOrderHeader
WHERE SalesOrderID = 43659;
```

Query 8

Subquery can be used in **FROM**.

```
SELECT sub.*
FROM ( SELECT *
      FROM sf_crime_incidents_2014_01
      WHERE day_of_week = 'Friday' ) sub
WHERE sub.resolution = 'NONE'
```

Subqueries can be used to aggregate in multiple stages. For example, what if you wanted to figure out how many incidents get reported on each day of the week?

Better yet, what if you wanted to know how many incidents happen, on average, on a Friday in December? In January? There are two steps to this process: counting the number of incidents each day (inner query), then determining the monthly average (outer query):



Query 9

```
SELECT LEFT(sub.date, 2) AS cleaned_month, sub.day_of_week,
       AVG(sub.incidents) AS average_incidents
FROM ( SELECT day_of_week, date, COUNT(incidnt_num) AS incidents
      FROM sf_crime_incidents
      GROUP BY day_of_week, date ) sub
GROUP BY cleaned_month, day_of_week
ORDER BY cleaned_month, day_of_week
```

Query 9A

```
SELECT cleaned_month, day_of_week, average_incidents
FROM (SELECT LEFT(sub.date, 2) AS cleaned_month, sub.day_of_week,
       AVG(sub.incidents) AS average_incidents
      FROM ( SELECT day_of_week, date, COUNT(incidnt_num) AS incidents
            FROM sf_crime_incidents
            GROUP BY day_of_week, date ) sub) sub1
GROUP BY cleaned_month, day_of_week
ORDER BY cleaned_month, day_of_week
```

Query 10

Subqueries can be used in conditional logic. You can use subqueries in conditional logic (in conjunction with WHERE, JOIN/ON).

The following query returns all of the entries from the earliest date in a sales dataset:

```
SELECT p.BusinessEntityID, p.FirstName, p.LastName, s.SalesQuota
FROM   Person p INNER JOIN SalesPerson s
      ON p.BusinessEntityID = s.BusinessEntityID
WHERE  s.SalesQuota IS NOT NULL AND s.SalesQuota > ( SELECT AVG(SalesQuota)
                                                    FROM SalesPerson );
```

Query 11

It's common to join a subquery that hits the same table as the outer query rather than filtering in the WHERE clause.

```
SELECT p.ProductID, p.Name AS ProductName, p.ProductSubcategoryID AS
       SubcategoryID, ps.Name AS SubcategoryName
FROM   Production.Product p INNER JOIN (SELECT ProductSubcategoryID, Name
                                       FROM Production.ProductSubcategory
                                       WHERE Name LIKE '%bikes%' ) AS ps
      ON p.ProductSubcategoryID = ps.ProductSubcategoryID;
```



5.3.4 T-SQL Well-known String Functions

The following table summarize the most well-known string functions.

Functions	Description
CHARINDEX (substring, string, start)	searches for a substring in a string, and returns the position
PATINDEX (%pattern%, string)	returns the position of a pattern in a string
CONCAT (string1, ..., stringN)	adds two or more strings together
LEN (string)	returns the length of a string
REPLACE (string, old_str, new_str)	replaces all occurrences of a substring within a string, with a new substring
REVERSE (string)	reverses a string and returns the result
LEFT (string, number_of_chars)	extracts a number of characters from a string (starting from left)
RIGHT (string, number_of_chars)	extracts a number of characters from a string (starting from right)
SUBSTRING (string, start, length)	extracts some characters from a string
UPPER (text)	converts a string to upper-case
LOWER (text)	converts a string to lower-case

5.4 Views in SQL

A view in SQL terminology is a single table that is derived from other tables. These other tables could be base tables or previously defined views. A view is considered as a virtual table in contrast to base tables whose tuples are stored in the database.

As an example, we may frequently issue queries that retrieve the employee name and project names that the employee works on. Rather than having to specify the join of the *EMPLOYEE*, *WORKS_ON*, and *PROJECT* tables every time we issue that query, we can define a view that is a result of these joins and hence already includes the attributes we wish to retrieve frequently.

We can now issue queries on the view, which are specified as a single table retrieval rather than retrievals involving two joins on three tables.

Note that, a view is always up to date; if we modify the tuples in the base tables on which the view is defined, the view automatically reflects these changes. Hence, the view is not realized at the time of a view definition but rather at the time we specify a query on the view. It the responsibility of the **DBMS** and not the user to make sure that the view is up to date.



5.4.1 Specification of Views in SQL

The command to specify a view is **CREATE VIEW**. We give the view a name, a list of attribute names, and a query to specify the contents of the view. If none of the view attributes result from applying functions or arithmetic operations, we do not have to specify attribute names for the view as they will be the same as the names of the attributes of the query which forming the view.

Example 1

Create a view to retrieve the employee name, project name, and number of hours.

```
CREATE VIEW WORKS_ON_VIEW AS
  SELECT FNAME, LNAME, PNAME, HOURS
  FROM EMPLOYEE, PROJECT, WORKS_ON
  WHERE SSN=ESSN AND PNO=PNUMBER;
```

In this example we did not specify any new attribute names for the view *WORKS_ON_VIEW*. So, in this case the *WORKS_ON_VIEW* inherits the names of the view attributes from the defining tables.

Example 2

Create a view to retrieve the department name, and number of employees in that department and their total salaries:

```
CREATE VIEW DEPT_INFO ( DEPT_NAME, NO_OF_EMPS,TOTAL_SAL) AS
  SELECT DNAME, COUNT(*), SUM(SALARY)
  FROM EMPLOYEE, DEPARTMENT
  WHERE DNO=DNUMBER;
  GROUP BY DNAME;
```

In this example we gave new names for the attributes of the view. We can specify SQL queries on a view see the following example:

Example 3

Retrieve the employee names who work in the project name 'ProjectX', using the created view in Example 1:

```
SELECT FNAME,LNAME
FROM WORKS_ON_VIEW
WHERE PNAME='ProjectX';
```

If we do not need the view anymore, we can use the **DROP VIEW** command to dispose of it as in the following example:



Example 4

Drop the view WORKS_ON_VIEW

```
DROP VIEW WORKS_ON_VIEW;
```

5.4.2 Remarks on Updating Views

Updating of views is complicated and can be ambiguous. In summary, we can make the following observations:

- A view with a single defining table is updateable if the view attributes contain the primary key or some other candidate key of the base relation because this maps each virtual view tuple to a single tuple.
- Views defined on multiple tables using joins are generally not updateable. Views defined using grouping and an aggregate function are not updateable.

Examine the following examples in order to illustrate potential problems with updating a view defined on multiple tables:

Consider the WORKS_ON1 view of Example1 and suppose we issue the command to update the PNAME attribute of 'John Smith' from 'ProjectX' to 'ProjectY'. So, the view update will be as follows:

```
UPDATE WORKS_ON_VIEW
SET PNAME='ProjectY'
WHERE LNAME='Smith' AND FNAME='John' AND PNAME='ProjectX'.
```

This query can be mapped into several updates on the base tables to give the desired update on the View. There are two possible updates corresponding to the previous view update:

The First UPDATE

```
UPDATE WORKS_ON
SET PNO = ( SELECT PNUMBER FROM PROJECT WHERE PNAME='ProjectY')
WHERE ESSN = ( SELECT SSN
                FROM EMPLOYEE
                WHERE LNAME='Smith' AND FNAME='John' )
AND PNO = (SELECT PNUMBER
            FROM PROJECT
            WHERE PNAME = 'ProjectX' )
```

The Second UPDATE

```
UPDATE PROJECT
SET PNAME = 'ProjectY'
WHERE PNAME = 'ProjectX'
```



Remarks:

- a. The first update (update 1) relates 'John Smith' to the 'ProductY' project tuple in place of the 'ProductX' and it is the most likely update. However, The second update (update 2) would also give the desired update effect on the view, but it accomplishes this by changing the name of the 'ProductX' tuple in the *PROJECT* table to 'ProductY'.
- b. Some view updates may not make much sense; for example, modifying the *TOTAL_SAL* attribute of *DEPT_INFO* does not make sense because *TOTAL_SAL* is defined to be the sum of the individual employee salaries:

```
UPDATE DEPT_INFO
SET TOTAL_SAL = 100000
WHERE DNAME='Research'
```

This view update can be satisfied through many possible updates on the related base tables, and no one can guess what such an update might mean and decide how to execute it? In general, we cannot guarantee that any view can be updated.

5.5 Specifying Indexes in SQL

SQL has statement to create and drop indexes dynamically. These commands are generally considered to be part of the **SQL DDL**. An index makes accessing tuples based on conditions that involve its indexing attributes more efficient. This means that, in general, executing a query will take less time if some attributes involved in the query conditions (selection & join) were indexed. This improvement can be dramatic for queries where large tables are involved.

The CREATE INDEX command is used to specify an index. Each index is given a name, which is used to drop the index when we do not need it anymore. For example, to create an index on *LNAME* attribute of the *EMPLOYEE* base table:

```
CREATE INDEX LNAME_INDEX
ON EMPLOYEE (LNAME);
```

In general, the index is arranged in ascending order of the indexing attribute values. For example, to create an index on the combination of *FNAME* (descending), *MINIT* (ascending), and *LNAME* (ascending), we write the following command:

```
CREATE INDEX NAMES_INDEX
ON EMPLOYEE (LNAME ASC, FNAME DESC, MINT);
```

There are two additional options on indexes in SQL:



1. To specify the key constraint on the indexing attribute, or combination of attributes.

```
CREATE UNIQUE INDEX SSN_INDEX
ON EMPLOYEE (SSN);
```

An attempt to create a unique index on an existing base table will fail if the current tuples in the table do not obey the uniqueness constraint on the indexing attribute. Also, notice creating an index on primary key is preferable. Two **NULL** values are considered equal for unique indexing purposes.

2. To specify a **CLUSTER** index, the key word **CLUSTER** is added at the end of the **CREATE INDEX** command. Join and selection condition are even more efficient when specified on attribute with cluster index. A base table can have at most one clustering index but any number of non-clustering indexes. The following example create an index on the *DNO* attribute of the *EMPLOYEE* table:

```
CREATE INDEX DNO_INDEX
ON EMPLOYEE (DNO)
CLUSTER;
```

In order to drop the *DNO_INDEX* we write the following command:

```
DROP INDEX DNO_INDEX;
```

5.6 Specifying Constraints in SQL

Because **SQL** allows **NULLs** as attribute values, a constraint **NOT NULL** may be specified if **NULL** is not permitted for a particular attribute. This is always implicitly specified for the attributes that are part of the primary key of each relation, but it can be specified for any other attributes whose values are required not to be **NULL**.

It is also possible to define a default value for an attribute by appending the clause **DEFAULT <value>** to an attribute definition. The default value is included in any new tuple if an explicit value is not provided for that attribute.

```
CREATE TABLE DEPARTMENT
(
    Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555',
    ... ,
    CONSTRAINT DEPTPK
    PRIMARY KEY(Dnumber),
    CONSTRAINT DEPTSK
    UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
    FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
    ON DELETE SET DEFAULT ON UPDATE CASCADE );
```



Another type of constraint can restrict attribute or domain values using the **CHECK** clause following an attribute.

For example, suppose that department numbers are restricted to integer numbers between 1 and 20; then, we can change the attribute declaration of Dnumber in the DEPARTMENT table

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
```

5.7 Specifying Constraints as Assertions

In **SQL**, users can specify general constraints—those that do not fall into any of the categories described in SQL Data Definition and SQL Constraints —via **declarative assertions**, using the **CREATE ASSERTION** statement of the DDL.

Each assertion is given a constraint name and is specified via a condition similar to the WHERE clause of an SQL query. For example, to specify the constraint that *the salary of an employee must not be greater than the salary of the manager of the department that the employee works for* in SQL, we can write the following assertion:

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT *
                     FROM EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
                     WHERE E.Salary>M.Salary AND E.Dno=D.Dnumber
                        AND D.Mgr_ssn=M.Ssn )
);
```

The constraint name SALARY_CONSTRAINT is followed by the keyword **CHECK**, which is followed by a condition in parentheses that must hold true on every database state for the assertion to be satisfied.

The **DBMS** is responsible for ensuring that the condition is not violated. Any **WHERE** clause condition can be used, but many constraints can be specified using the **EXISTS** and **NOT EXISTS** style of **SQL** conditions.

Whenever some tuples in the database cause the condition of an **ASSERTION** statement to evaluate to *FALSE*, the constraint is violated. The constraint is satisfied by a database state if no combination of tuples in that database state violates the constraint.

The basic technique for writing such assertions is to specify a query that selects any tuples *that violate the desired condition*. By including this query inside a **NOT EXISTS** clause, the assertion will specify that the result of this query must be empty so that the condition will always be *TRUE*.

Thus, the assertion is violated if the result of the query is not empty. In the preceding example, the query selects all employees whose salaries are greater than the salary of the manager of their department. If the result of the query is not empty, the assertion is violated.



Note that the **CHECK** clause and constraint condition can also be used to specify constraints on *individual* attributes and domains and on *individual* tuples.

A major difference between **CREATE ASSERTION** and the individual domain constraints and tuple constraints is that the **CHECK** clauses on individual attributes, domains, and tuples are checked in **SQL only when tuples are inserted or updated**.

Hence, constraint checking can be implemented more efficiently by the **DBMS** in these cases. The schema designer should use **CHECK** on attributes, domains, and tuples only when he or she is sure that the constraint can *only be violated by insertion or updating of tuples*.

On the other hand, the schema designer should use **CREATE ASSERTION** only in cases where it is not possible to use **CHECK** on attributes, domains, or tuples, so that simple checks are implemented more efficiently by the **DBMS**.

5.8 Stored Procedures

Stored Procedures are program modules that are stored by the **DBMS** at the database server. It is sometimes useful to create database program modules—procedures or functions—that are stored and executed by the **DBMS** at the database server.

These are historically known as database **stored procedures**, although they can be functions or procedures.

The term used in the **SQL** standard for stored procedures is **persistent stored modules** because these programs are stored persistently by the **DBMS**, similarly to the persistent data stored by the **DBMS**.

Stored procedures are useful in the following circumstances:

- If a database program is needed by several applications, it can be stored at the server and invoked by any of the application programs. This reduces duplication of effort and improves software modularity.
- Executing a program at the server can reduce data transfer and communication cost between the client and server in certain situations.
- These procedures can enhance the modeling power provided by views by allowing more complex types of derived data to be made available to the database users via the stored procedures.
- Additionally, they can be used to check for complex constraints that are beyond the specification power of assertions and triggers.

In general, many commercial **DBMSs** allow stored procedures and functions to be written in a general-purpose programming language or simple **SQL** commands such as retrievals and updates. The general form of declaring stored procedures is as follows:

```
CREATE PROCEDURE <procedure name> (<parameters>)
<local declarations>
<procedure body>;
```



The parameters and local declarations are optional and are specified only if needed. For declaring a function, a return type is necessary, so the declaration form is:

```
CREATE FUNCTION <function name> (<parameters>)
RETURNS <return type>
<local declarations>
<function body> ;
```

In general, each parameter should have a **parameter type** that is one of the SQL data types. Each parameter should also have a **parameter mode**, which is one of IN, OUT, or INOUT. These correspond to parameters whose values are input only, output (returned) only, or both input and output, respectively.

Because the procedures and functions are stored persistently by the DBMS, it should be possible to call them from the various SQL interfaces and programming techniques.

The **EXEC** or **EXECUTE statement** can be used to invoke a stored procedure. The format of the statement is as follows:

```
EXECUTE <procedure or function name> (<argument list>) ;
```

Example 1

The following SQL statement creates a stored procedure that selects Customers from a particular City from the "Customers" table:

```
CREATE PROCEDURE SelectAllCustomers @City nvarchar(30)
AS
    SELECT * FROM Customers WHERE City = @City
GO
```

Execute the stored procedure above as follows:

```
EXEC SelectAllCustomers City = "London";
```

Example 2

The following SQL statement creates a stored procedure that selects Customers from a particular City with a particular PostalCode from the "Customers" table:

```
CREATE PROCEDURE SelectAllCustomers @City nvarchar(30), @PostalCode nvarchar(10)
AS
    SELECT * FROM Customers
    WHERE City = @City AND PostalCode = @PostalCode
GO
```

Execute the stored procedure above as follows:

```
EXEC SelectAllCustomers City = "London", PostalCode = "WA1 1DP";
```



Example 3

The following procedure returns two result sets.

```
CREATE PROCEDURE MultipleResults
AS
    SELECT TOP(10) BusinessEntityID, Lastname, FirstName FROM Person;
    SELECT TOP(10) CustomerID, AccountNumber FROM Customer;
GO
```

5.9 SQL/PSM: Extending SQL

SQL/PSM is the part of the SQL standard that specifies how to write persistent stored modules. It includes the statements to create functions and procedures. It also includes additional programming constructs to enhance the power of SQL for the purpose of writing the code (or body) of stored procedures and functions. The conditional branching statement in SQL/PSM has the following form:

```
IF <condition> THEN <statement list>
ELSEIF <condition> THEN <statement list>
...
ELSEIF <condition> THEN <statement list>
ELSE <statement list>
END IF ;
```



Consider the following example, which illustrates how the conditional branch structure can be used in an SQL/PSM function.

```
CREATE FUNCTION Dept_size(IN deptno INTEGER)
RETURNS VARCHAR [7]
DECLARE No_of_ems INTEGER ;
    SELECT COUNT(*) INTO No_of_ems
    FROM EMPLOYEE WHERE Dno = deptno ;

    IF No_of_ems > 100 THEN RETURN "HUGE"
    ELSEIF No_of_ems > 25 THEN RETURN "LARGE"
    ELSEIF No_of_ems > 10 THEN RETURN "MEDIUM"
    ELSE RETURN "SMALL"
END IF;
```

SQL/PSM has several constructs for looping. There are standard while and repeat looping structures, which have the following forms:

```
WHILE <condition> DO
<statement list>
END WHILE;

REPEAT
<statement list>
UNTIL <condition>
END REPEAT ;
```

There also:

- **BREAK**
Causes an exit from the innermost **WHILE** loop. Any statements that appear after the **END** keyword, marking the end of the loop, are executed.
- **CONTINUE**
Causes the **WHILE** loop to restart, ignoring any statements after the **CONTINUE** keyword.



T-SQL Example

In the following example, if the average list price of a product is less than \$300, the WHILE loop doubles the prices and then selects the maximum price. If the maximum price is less than or equal to \$500, the WHILE loop restarts and doubles the prices again. This loop continues doubling the prices until the maximum price is greater than \$500, and then exits the WHILE loop and prints a message.

```
WHILE (SELECT AVG(ListPrice) FROM Product) < $300
    BEGIN
        UPDATE Product SET ListPrice = ListPrice * 2
        SELECT MAX(ListPrice) FROM Product
        IF (SELECT MAX(ListPrice) FROM Product) > $500 THEN
            BREAK
        ELSE
            CONTINUE
        ...
    END
PRINT 'Too much for the market to bear';
```

5.10 Triggers in SQL

In many cases it is convenient to specify the type of action to be taken when certain events occur and when certain conditions are satisfied.

For example, it may be useful to specify a condition that, if violated, causes some user to be informed of the violation.

- A manager may want to be informed if an employee's travel expenses exceed a certain limit by receiving a message whenever this occurs.

The action that the DBMS must take in this case is to send an appropriate message to that user. The condition is thus used to **monitor** the database. Other actions may be specified, such as executing a specific *stored procedure* or triggering other updates.

The **CREATE TRIGGER** statement is used to implement such actions in **SQL**.

A database trigger is a stored procedure that is invoked automatically when a predefined event occurs. Database triggers enable **DBAs** (Database Administrators) to create additional relationships between separate databases.

In other words, a trigger can be defined to execute before or after or instead of an **INSERT**, **UPDATE**, or **DELETE** operation, either once per modified row, or once per SQL statement. If a trigger event occurs, the trigger's function is called at the appropriate time to handle the event.



Notes:

1. Triggers can be assigned to tables or views.
2. It is not possible to create a trigger to fire when a data modification occurs in two or more tables. A trigger can be associated only with a single table.

5.10.1 Why Use Triggers?

To improve data integrity, trigger can be used. When an action is performed on data, it is possible to check if the manipulation of the data concurs with the underlying business rules, and thus avoids erroneous entries in a table. For example:

- We might want to ship a free item to a client with the order, if it totals more than \$1000. A trigger will be built to check the order total upon completion of the order, to see if an extra order line needs to be inserted.
- In a banking scenario, when a request is made to withdraw cash from a cash point, the stored procedure will create a record on the client's statement table for the withdrawal, and the trigger will automatically reduce the balance as required. The trigger may also be the point at which a check is made on the client's balance to verify that there is enough balance to allow the withdrawal. By having a trigger on the statement table, we are secure in the knowledge that any statement entry made, whether withdrawal or deposit, will be validated and processed in one central place.

Another use of triggers can be to carry out an action when a specific criterion has been met. One example of this is a case where an e-mail requesting more items to be delivered is sent, or an order for processing could be placed, when stock levels reach a preset level.

We must be careful that the table we insert into does not have a trigger that will cause this first trigger to fire.

It is possible to code triggers that result in an endless loop, as we can define a trigger on *TableA*, which inserts into *TableB*, and a trigger for *TableB*, which updates *TableA*. This scenario will ultimately end in an error being generated by the **SQL Server**. Figure 5.1 demonstrate this scenario. The sequences of events as follows:

1. A stored procedure, A, updates TableA.
2. This fires a trigger from TableA.
3. The defined trigger on TableA updates TableB.
4. TableB has a trigger which fires.
5. This trigger from TableB updates TableA.



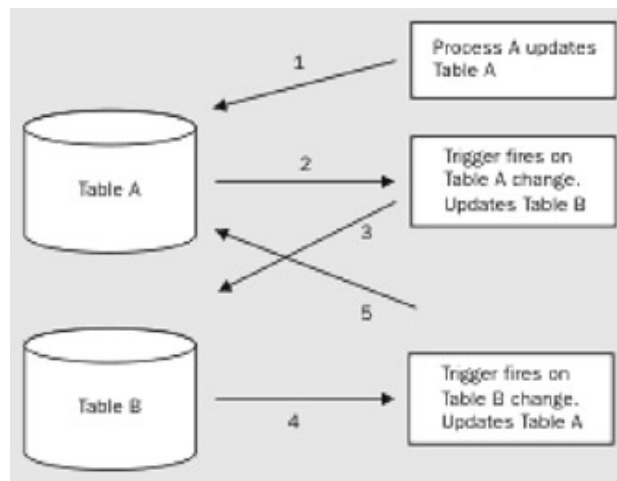


Figure 5.1

5.10.2 Trigger Syntax

```
CREATE TRIGGER name ON table
[BEFORE/AFTER/INSTEAD OF]
[INSERT, UPDATE, DELETE] AS
BEGIN
    -- SQL statements
END
```

Note: Microsoft SQL Server does not have **BEFORE** but has **INSTEAD OF** which can do the same function.

5.10.3 Trigger Examples

Assume that we need to track the **DML** actions that are performed on a specific table and write these logs in a history table, where the *ID* of the inserted, updated or deleted record and the action that is performed will be written to the history table. The **CREATE TABLE T-SQL** statements below can be used to create both the source and history tables:

```
CREATE TABLE TriggerDemo_Parent
( ID          INT IDENTITY (1,1)      PRIMARY KEY,
  Emp_First_name  VARCHAR (50),
  Emp_Last_name   VARCHAR (50),
  Emp_Salary      INT )
GO
CREATE TABLE TriggerDemo_History
( HID          INT IDENTITY (1,1)      PRIMARY KEY,
  ParentID      INT,
  PerformedAction VARCHAR (50) )
GO
```



T-SQL Trigger Example 1

Insert Trigger

```
CREATE TRIGGER AfterInsertTrigger
ON TriggerDemo_Parent
AFTER INSERT
AS
    INSERT INTO TriggerDemo_History
    VALUES ((SELECT TOP 1 inserted.ID FROM inserted), 'Insert')
GO
```

Delete Trigger

```
CREATE TRIGGER AfterDeleteTrigger
ON TriggerDemo_Parent
AFTER DELETE
AS
    INSERT INTO TriggerDemo_History
    VALUES ((SELECT TOP 1 deleted.ID FROM deleted), 'Delete')
GO
```

Update Trigger

```
CREATE TRIGGER AfterUPDATETrigger
ON TriggerDemo_Parent
AFTER UPDATE
AS
    INSERT INTO TriggerDemo_History
    VALUES ((SELECT TOP 1 inserted.ID FROM inserted), 'UPDATE')
GO
```

T-SQL Trigger Example 2

The following DML trigger prints a message to the client when anyone tries to add or change data in the Customer table.

```
CREATE TRIGGER reminder1
ON Customer
AFTER INSERT, UPDATE
AS
    RAISERROR ('Notify Customer Relations', 16, 10);
GO
```

Note: 16 is the Severity Level & 10 is the user state



Remarks**1. severity**

Is the user-defined severity level associated with this message.

Severity levels from 0 through 18 can be specified by any user. Severity levels from 19 through 25 can only be specified by members of the sysadmin. For severity levels from 19 through 25, the WITH LOG option is required.

Severity levels from 20 through 25 are considered fatal. If a fatal severity level is encountered, the client connection is terminated after receiving the message, and the error is logged in the error and application logs.

2. State

Is an integer from 0 through 255. Negative values default to 1. Values larger than 255 should not be used.

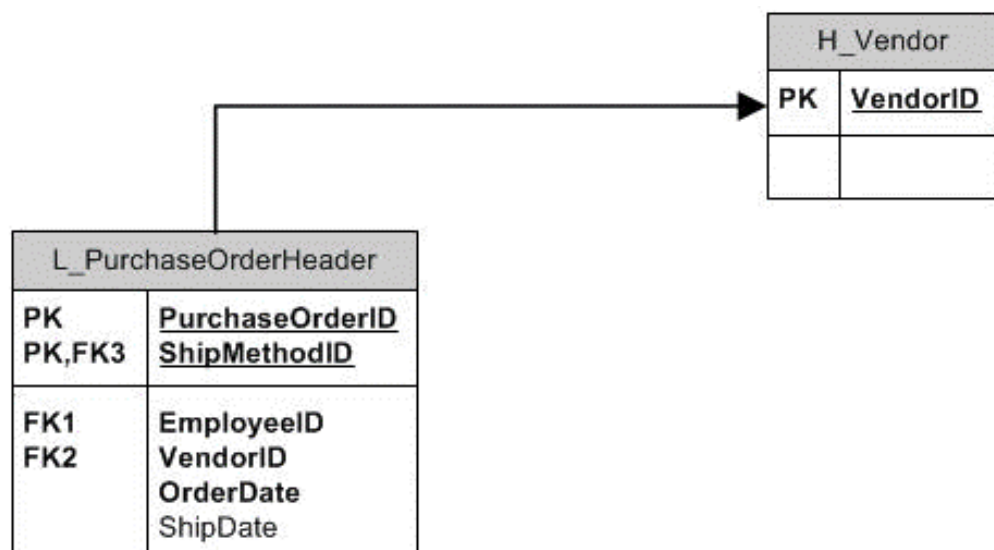
If the same user-defined error is raised at multiple locations, using a unique state number for each location can help find which section of code is raising the errors.

Because **CHECK** constraints can reference only the columns on which the column-level or table-level constraint is defined, any cross-table constraints (in this case, business rules) must be defined as triggers.

The *Example 3* creates a **DML** trigger in the *AdventureWorks2012* database. This trigger checks to make sure the credit rating for the vendor is good (not 5) when an attempt is made to insert a new purchase order into the *PurchaseOrderHeader* table.

T-SQL Trigger Example 3

To obtain the credit rating of the vendor, the Vendor table must be referenced. If the credit rating is too low, a message is displayed, and the insertion does not execute.



```

CREATE TRIGGER LowCredit ON PurchaseOrderHeader
AFTER INSERT
AS
IF (@@ROWCOUNT_BIG = 0)
RETURN;
IF EXISTS (SELECT *
           FROM inserted AS i
           JOIN Vendor AS v   ON v.BusinessEntityID = i.VendorID
           WHERE v.CreditRating = 5 )
BEGIN
    RAISERROR ('A vendor's credit rating is too low to accept new purchase orders.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN
END;
GO

```

Remarks:

1. This Trigger can be written using **BEFORE OR INSTEAD_OF**
2. @@ROWCOUNT_BIG Returns the number of rows affected by the last query

The second type of DML triggers is the INSTEAD OF DML trigger. As mentioned previously, the INSTEAD OF trigger will override the statement of the action that fires the trigger with the statement provided in the trigger.

Assume that we need to log the DML actions that users are trying to perform on a specific table, without allowing them to perform that action.

The CREATE TABLE T-SQL statements below can be used to create both the source and alternative tables:

```

CREATE TABLE TriggerDemo_NewParent
(
    ID                INT                IDENTITY (1,1)    PRIMARY KEY,
    Emp_First_name    VARCHAR (50),
    Emp_Last_name     VARCHAR (50),
    Emp_Salary        INT
)
GO
CREATE TABLE TriggerDemo_InsteadParent
(
    ID                INT                IDENTITY (1,1)    PRIMARY KEY,
    ParentID          INT,
    PerformedAction    VARCHAR (50),
)
GO

```



T-SQL Trigger Example 4

Insert Trigger

```
CREATE TRIGGER InsteadOfInsertTrigger
ON TriggerDemo_NewParent
INSTEAD OF INSERT
AS
INSERT INTO TriggerDemo_InsteadParent VALUES ((SELECT TOP 1 inserted.ID FROM
inserted), 'Trying to Insert new ID')
GO
```

Update Trigger

```
CREATE TRIGGER InsteadOfUpdateTrigger
ON TriggerDemo_NewParent
INSTEAD OF UPDATE
AS
INSERT INTO TriggerDemo_InsteadParent VALUES ((SELECT TOP 1 inserted.ID FROM
inserted), 'Trying to Update an existing ID')
GO
```

T-SQL Trigger Example 5

Insert data in view and verify functionality of **INSTEAD OF** trigger

```
USE AdventureWorks
GO
CREATE TRIGGER INSTEADOF_TR_I_EmpQualification ON vw_EmpQualification
INSTEAD OF INSERT AS
BEGIN
    DECLARE @Code TINYINT
    SELECT @Code = qualificationCode
    FROM lib_Qualification L INNER JOIN INSERTED I
        ON L.qualification = I.qualification
    IF (@code is NULL )
    BEGIN
        RAISERROR (N'The provided qualification does not exist in qualification library', 16, 1)
        RETURN
    END
    INSERT INTO employees (empcode, name, designation, qualificationCode, deleted)
    SELECT empcode, name, designation, @code, 0
    FROM inserted
END
GO
```



T-SQL Trigger Example 6

This Example Returns a Boolean value that indicates whether an **INSERT** or **UPDATE** attempt was made on a specified column of a table or view. **UPDATE()** is used anywhere inside the body of a **T-SQL INSERT** or **UPDATE** trigger to test whether the trigger should execute certain actions. Update data in view and verify functionality of **INSTEAD OF** trigger

```
USE AdventureWorks
GO
```

```
CREATE TRIGGER INSTEADOF_TR_U_EmpQualification ON vw_EmpQualification
INSTEAD OF UPDATE AS
BEGIN
IF (UPDATE(qualification)) -- If qualification is updated
BEGIN
    DECLARE @code TINYINT
    UPDATE employees
    SET @code = L.qualificationcode
    FROM lib_qualification L INNER JOIN inserted I ON L.qualification = I.qualification
    IF (@code is NULL )
    BEGIN
        RAISERROR (N'The provided qualification does not exist in qualification library', 16, 1)
        RETURN
    END
    UPDATE employees
    SET qualificationCode = @code
    FROM inserted I INNER JOIN employees E ON I.empcode = E.empcode
END

IF (UPDATE(EmpCode)) -- If employee code is updated
BEGIN
    RAISERROR (N'You cannot edit employee code, Transaction has been failed', 16, 1)
    RETURN
END

IF (UPDATE(name)) -- If name is updated
BEGIN
    UPDATE employees
    SET name = I.name
    FROM inserted I INNER JOIN employees E ON I.empcode = E.empcode
    WHERE E.empcode = I.empcode
END
```



```

IF (UPDATE(designation)) -- If designation is updated
BEGIN
    UPDATE employees
    SET designation = I.designation
    FROM inserted I INNER JOIN employees E ON I.empcode = E.empcode
    WHERE E.empcode = I.empcode
END
END
GO

```

Construction Database Trigger T-SQL Example

Consider the **Construction Enterprise Database** (Chapter 1, Exercise 1). In Example prevent “Equipment to be assigned to more than one project at a time”. This trigger can be written using **AFTER** and **INSTEAD OF** as follows:

```

CREATE TRIGGER T_OVERLAP_EQUIPMENT_ASSIGNMENT ON tbl_Assignment
AFTER INSERT AS
BEGIN
    SET NOCOUNT ON; -- Hide number of rows affected
    IF (( SELECT COUNT(*)
          FROM Inserted AS I INNER JOIN tbl_Assignment AS a
            ON i.EID = a.EID
          WHERE (i.StartDate >= a.StartDate AND i.EndDate <= a.EndDate) OR
                (i.StartDate < a.StartDate AND i.EndDate <= a.EndDate) OR
                (i.StartDate >= a.StartDate AND i.EndDate > a.EndDate) OR
                (i.StartDate < a.StartDate AND i.EndDate > a.EndDate)) >= 2)
    BEGIN
        RAISERROR (N'Equipment already assigned to another project', 16, 1)
        ROLLBACK TRANSACTION
    END
END

```



Same functionality could be done using the **INSTEAD OF** Trigger:

```
CREATE TRIGGER T_OVERLAP_EQUIPMENT_ASSIGNMENT ON tbl_Assignment
INSTEAD OF INSERT AS
BEGIN
SET NOCOUNT ON; -- Hide number of rows affected
IF ((    SELECT COUNT(*)
        FROM Inserted AS I INNER JOIN tbl_Assignment AS a
            ON i.EID = a.EID
        WHERE (i.StartDate >= a.StartDate AND i.EndDate <= a.EndDate) OR
            (i.StartDate < a.StartDate AND i.EndDate <= a.EndDate) OR
            (i.StartDate >= a.StartDate AND i.EndDate > a.EndDate) OR
            (i.StartDate < a.StartDate AND i.EndDate > a.EndDate)) >= 1)
BEGIN
    RAISERROR (N'Equipment already assigned to another project', 16, 1)
ELSE BEGIN
    INSERT INTO tbl_Assignment SELECT PID, EID, StartDate, EndDate
    FROM Inserted
END
END
```



Exercises

1. Describe the six clauses in the syntax of an SQL retrieval query. Show what type of constructs can be specified in each of the six clauses. Which of the six clauses are required and which are optional?
2. Describe conceptually how an SQL retrieval query will be executed by specifying the conceptual order of executing each of the six clauses.
3. Specify the following views in SQL on the COMPANY database schema.
 - a. A view that has the department name, manager name, and manager salary for every department
 - b. A view that has the employee name, supervisor name, and employee salary for each employee who works in the 'Research' department
 - c. A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project
 - d. A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project *with more than one employee working on it*



Chapter 6 Database Security

6.1 Types of Security

Database security is a broad area that addresses many issues, including the following:

- **Various legal and ethical issues** regarding the right to access certain information
For example, some information may be deemed to be private and cannot be accessed legally by unauthorized organizations or persons.
- **Policy issues** at the governmental, institutional, or corporate level regarding what kinds of information should not be made publicly available.
For example, credit ratings and personal medical records.
- **System-related issues** such as the system levels at which various security functions should be enforced.
For example, whether a security function should be handled at the physical hardware level, the operating system level, or the DBMS level.
- **The need in some organizations to identify multiple security levels and to categorize the data and users based on these classifications.**
For example, top secret, secret, confidential, and unclassified. The security policy of the organization with respect to permitting access to various classifications of data must be enforced.

6.1.1 Threats to Databases

- **Loss of integrity.** Database integrity refers to the requirement that information be protected from improper modification.
 - Modification of data includes creating, inserting, and updating data; changing the status of data; and deleting data.
 - Integrity is lost if unauthorized changes are made to the data by either intentional or accidental acts. If the loss of system or data integrity is not corrected, continued use of the contaminated system or corrupted data could result in inaccuracy, fraud, or erroneous decisions.
- **Loss of availability.** Database availability refers to making objects available to a human user or a program who/which has a legitimate right to those data objects.
 - Loss of availability occurs when the user or program cannot access these objects.
- **Loss of confidentiality.** Database confidentiality refers to the protection of data from unauthorized disclosure.
 - The impact of unauthorized disclosure of confidential information can range from violation of the Data Privacy Act to the jeopardization of national security.



- Unauthorized, unanticipated, or unintentional disclosure could result in loss of public confidence, embarrassment, or legal action against the organization.

In a multiuser database system, the DBMS must provide techniques to enable certain users or user groups to access selected portions of a database without gaining access to the rest of the database.

For example, sensitive information such as employee salaries or performance reviews should be kept confidential from most of the database system's users. A DBMS typically includes a **database security and authorization subsystem** that is responsible for ensuring the security of portions of a database against unauthorized access.

It is now ordinary to refer to two types of database security mechanisms:

- **Discretionary security mechanisms.** These are used to grant privileges to users, including the capability to access specific data files, records, or fields in a specified mode (such as read, insert, delete, or update).
- **Mandatory security mechanisms.** These are used to enforce multilevel security by classifying the data and users into various security classes (or levels) and then implementing the appropriate security policy of the organization.
 - For example, a typical security policy is to permit users at a certain classification (or clearance) level to see only the data items classified at the user's own (or lower) classification level.
 - An extension of this is *role-based security*, which enforces policies and privileges based on the concept of organizational roles.

6.1.2 Control Measures

Four main control measures are used to provide security of data in databases:

- **Access control**
 - The security mechanism of a DBMS must include provisions for restricting access to the database system as a whole.
 - is handled by creating user accounts and passwords to control the login process by the DBMS.
- **Inference control**
 - Statistical database users such as government statisticians or market research firms are allowed to access the database to retrieve statistical information about a population but not to access the detailed confidential information about specific individuals.
- **Flow control**
 - prevents information from flowing in such a way that it reaches unauthorized users.
- **Data encryption**



- used to protect sensitive data (such as credit card numbers) that is transmitted via some type of communications network.
- The data is **encoded** using some coding algorithm. An unauthorized user who accesses encoded data will have difficulty deciphering it, but authorized users are given decoding or decrypting algorithms (or keys) to decipher the data.

6.1.3 Database Security and the DBA

The database administrator (DBA) is the central authority for managing a database system. The DBA's responsibilities include granting privileges to users who need to use the system and classifying users and data in accordance with the policy of the organization.

The DBA has a **DBA account** in the DBMS, sometimes called a **system** or **superuser account**, which provides powerful capabilities that are not made available to regular database accounts and users.

DBA-privileged commands include commands for granting and revoking privileges to individual accounts, users, or user groups and for performing the following types of actions:

1. **Account creation.** This action creates a new account and password for a user or a group of users to enable access to the DBMS.
2. **Privilege granting.** This action permits the DBA to grant certain privileges to certain accounts.
3. **Privilege revocation.** This action permits the DBA to revoke (cancel) certain privileges that were previously given to certain accounts.
4. **Security level assignment.** This action consists of assigning user accounts to the appropriate security clearance level.

6.1.4 Access Control, User Accounts, and Database Audits

The DBA will then create a new **account number** and **password** for the user if there is a legitimate need to access the database. The user must **log in** to the DBMS by entering the account number and password whenever database access is needed.

The database system must also keep track of all operations on the database that are applied by a certain user throughout each **login session**, which consists of the sequence of database interactions that a user performs from the time of logging in to the time of logging off.

When a user logs in, the **DBMS** can record the user's account number and associate it with the computer or device from which the user logged in. To keep a record of all updates applied to the database and of particular users who applied each update, we can modify the *system log*. (Recall Recovery Chapter) that the **system log** includes an entry for each operation applied to the database that may be required for recovery from a transaction failure or system crash. We can expand the log entries so that they also include



the account number of the user and the online computer or device ID that applied each operation recorded in the log.

If any tampering with the database is suspected, a **database audit** is performed, which consists of reviewing the log to examine all accesses and operations applied to the database during a certain time period. When an illegal or unauthorized operation is found, the DBA can determine the account number used to perform the operation. A database log that is used mainly for security purposes serves as an **audit trail**.

6.2 Discretionary Access Control

The typical method of enforcing **discretionary access control** in a database system is based on the granting and revoking of **privileges**.

Types of Discretionary Privileges

Informally, there are two levels for assigning privileges to use the database system:

1. **The account levels.** At this level, the DBA specifies the particular privileges that each account holds independently of the relations in the database.
2. **The relation (or table) level.** At this level, the DBA can control the privilege to access each individual relation (or table) or view in the database.

6.2.1 Privileges, Roles, Schema and Database Users:

Privileges:

Privileges are the right to execute **SQL** statement. The database administrator is a high-level user with the ability to grant user access to the database and its objects. The users require system privileges to gain access to the database and object privileges to manipulate the contents of the objects in the database.

Users also can be given the privileges to grant additional privileges to other users or to roles.

1. Roles

Roles are named groups of related privileges.

2. Schema

is a collection of objects such as tables, Views, and indexes. The schema is owned by a Database user and has the same name of that user.

3. Creating User

The Database Administrator (**DBA**) creates new user by executing the **CREATE USER** command with the following syntax:

```
CREATE USER < username> IDENTIFIED BY < password >;
```



When a user is created, he doesn't have any privileges, so DBA allocates a number of system privileges to that user. System privileges in turn determine what the user can do the database level.

Example

```
CREATE USER Hassan IDENTIFIED BY Has;
```

By this command the **DBA** has created the user and can assign privileges to that user.

Typical User Privileges

System Privilege	Operations Authorized
Create Session	Connect to the database
Create Table	Create tables in the user's Schema
Create View	Create Views in the user's Schema

6.2.2 Granting and Revoking of Privileges

Suppose that the DBA creates four accounts—A1, A2, A3, and A4 and wants only A1 to be able to create base relations. To do this, the DBA must issue the following GRANT command in SQL:

```
GRANT CREATETAB TO A1;
```

The CREATETAB (create table) privilege gives account A1 the capability to create new database tables (base relations) and is hence an account privilege.

In **SQL92**, the same effect can be accomplished by having the DBA issue a CREATE SCHEMA command, as follows:

```
CREATE SCHEMA EXAMPLE AUTHORIZATION A1;
```

User account *A1* can now create tables under the schema called *EXAMPLE*. To continue our example, suppose that *A1* creates the two base relations *EMPLOYEE* and *DEPARTMENT*; *A1* is then the **owner** of these two relations and hence has *all the relation privileges* on each of them.

Next, suppose that account A1 wants to grant to account A2 the privilege to insert and delete tuples in both of these relations.

However, *A1* does not want *A2* to be able to propagate these privileges to additional accounts. *A1* can issue the following command:

```
GRANT INSERT, DELETE ON EMPLOYEE, DEPARTMENT TO A2;
```



Next, suppose that *A1* wants to allow account *A3* to retrieve information from either of the two tables and to be able to propagate the **SELECT** privilege to other accounts. *A1* can issue the following command:

```
GRANT SELECT ON EMPLOYEE, DEPARTMENT TO A3 WITH GRANT OPTION;
```

The clause **WITH GRANT OPTION** means that *A3* can now propagate the privilege to other accounts by using **GRANT**.

For example, *A3* can grant the **SELECT** privilege on the *EMPLOYEE* relation to *A4* by issuing the following command:

```
GRANT SELECT ON EMPLOYEE TO A4;
```

Notice that *A4* cannot propagate the **SELECT** privilege to other accounts because the **GRANT OPTION** was not given to *A4*.

Now suppose that *A1* decides to revoke the *SELECT* privilege on the *EMPLOYEE* relation from *A3*; *A1* then can issue this command:

```
REVOKE SELECT ON EMPLOYEE FROM A3;
```

The **DBMS** must now revoke the **SELECT** privilege on *EMPLOYEE* from *A3*, and it must also *automatically revoke* the **SELECT** privilege on *EMPLOYEE* from *A4*.

This is because *A3* granted that privilege to *A4*, but *A3* does not have the privilege any more.

Next, suppose that *A1* wants to give back to *A3* a limited capability to **SELECT** from the *EMPLOYEE* relation and wants to allow *A3* to be able to propagate the privilege.

The limitation is to retrieve only the Name, Bdate, and Address attributes and only for the tuples with Dno = 5. *A1* then can create the following view:

```
CREATE VIEW A3EMPLOYEE AS
SELECT Name, Bdate, Address
FROM EMPLOYEE
WHERE Dno = 5;
```

After the view is created, *A1* can grant **SELECT** on the view *A3EMPLOYEE* to *A3* as follows:

```
GRANT SELECT ON A3EMPLOYEE TO A3 WITH GRANT OPTION;
```

Finally, suppose that *A1* wants to allow *A4* to update only the Salary attribute of *EMPLOYEE*; *A1* can then issue the following command:

```
GRANT UPDATE ON EMPLOYEE (Salary) TO A4;
```



6.2.3 Granting Object Privileges

The DBA can allow users to perform a particular action on a specific table, view by granting the object privileges. These privileges are shown in the following table:

Object Privilege	Table	View
ALTER	yes	no
DELETE	yes	yes
INDEX	yes	no
INSERT	yes	yes
SELECT	yes	yes
UPDATE	yes	yes



Exercises

- 1 - What is the difference between discretionary and mandatory access control?
- 2 - How can privacy of data be preserved in a database?
- 3 - What are some of the current outstanding challenges for database security?



Chapter 7 Database Recovery & Concurrency

7.1 Transaction Processing Systems

They are systems with large databases and hundreds of concurrent users that are executing database transactions.

Transaction:

It is an executing program that forms a logical unit of database processing. A Transaction includes one or more database access operations (e.g. insertion, deletion, and modification or retrieval operations). Transaction can also be defined as an atomic unit of work that is either completed entirely or not at all, if it fails for any reason.

Examples of Transaction Processing Systems:

- Reservation Systems
- Banking Systems
- Supermarket Checkout Systems

In these systems a Concurrency Control Problem arises which occurs when multiple transactions submitted by various users interfere with one another in a way that produces incorrect results.

7.1.1 Single-User versus Multi-User Systems

In a computer system, with single central processing unit, executing multiple programs (processes or transactions) is allowed (i.e. multiple user environments), using Interleaving Concept. Interleaving keeps the CPU busy when a process requires CPU is switched to execute another process if the current process has an I/O operation or it has finished the currently allocated CPU slot of time to it. Figure 7.1 shows **Interleaving Processing** versus **Parallel Processing** of concurrent transactions.

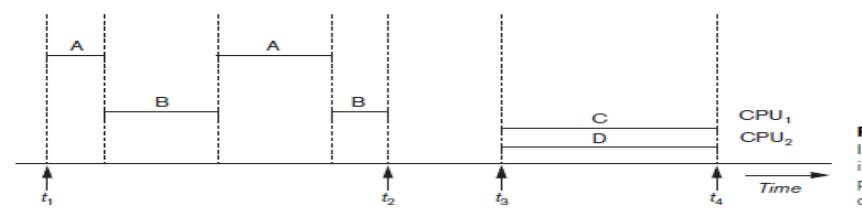


Figure 7.1



7.1.2 Basic Database Access Operations

The basic database access operations that a transaction can include are as follows:

1. **Read-Item(X)**: Reads a database item named X into a program variable (also named X for simplicity).
2. **Write-Item (X)**: Writes value of program variable X into the database item named X.

Executing Read-Item (X) command includes the following steps:

1. Find the address of the disk block that contains item X.
2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
3. Copy item X from the buffer to the program variable named X.

Executing Write-Item (X) command includes the following steps:

1. Find the address of the disk block that contains item X.
2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
3. Copy item X from the program variable named X into its location in the buffer.
4. Store the updated block from the buffer back to disk (either immediately or at some later point of time).

7.1.3 Why Concurrency Control is Needed?

In a Multi-User environment, transactions submitted by the various users may execute concurrently and may access and update the same database items. If this concurrent execution is uncontrolled, it leads to the following main problems:

1. **The Lost Update Problem:**
This Problem occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database items incorrect (see Figure 7.2a).
2. **The Temporary Update (or Dirty Read) Problem:**
This problem occurs when one transaction updates a database item and then the transaction fails for some reasons. The updated item is accessed by another transaction before it is changed back to its original value (see Figure 7.2b)
3. **The Incorrect Summary Problem:**
If one transaction is calculating an aggregate function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and other after they are updated (Figure 7.2c).



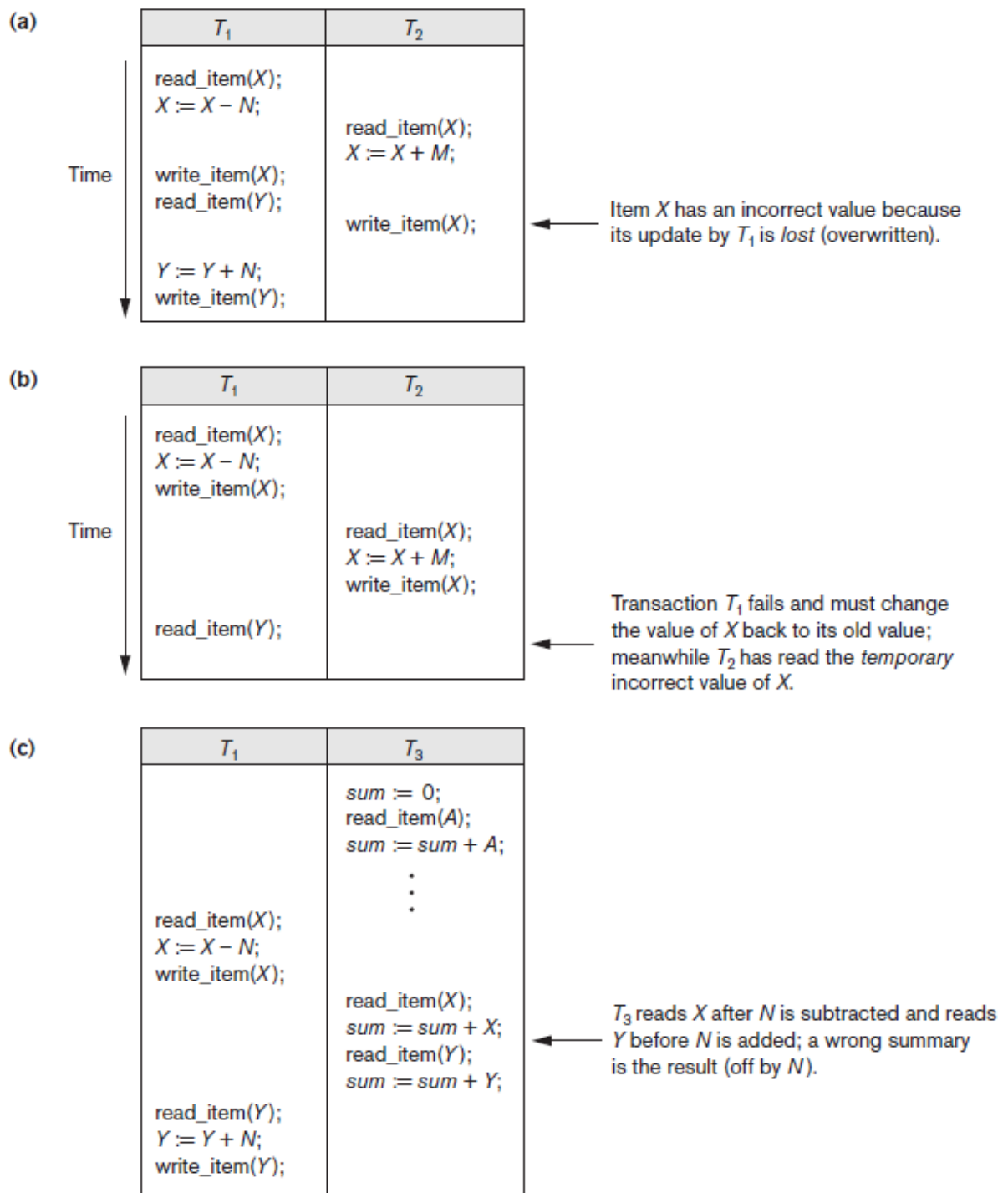


Figure 7.2

7.1.4 Why Recovery Is Needed?

Recovery is required to ensure the following when transaction is submitted to **DBMS** for execution:

1. All the operations in the transaction are completed successfully and their effect is recorded permanently in the database.
2. It is not allowed to some of transaction operations to update database, if transaction fails for any reasons.



7.1.5 Types of Failures:

Failures are generally classified as Transaction, System and Media failures. These are several possible reasons for a transaction to fail in the middle of execution:

1. **Computer failure (system crash):**
A hardware, software, or network error occurs in the computer system during transaction execution. Disk failure is one of the main media failures since it contains the system log file.
2. **A transaction or system error:**
Some operations in a transaction may cause it fails (e.g. divide by zero or logical programming errors).
3. **Local errors or exception conditions detected by the transaction:**
During transaction execution, certain conditions may occur that necessitate transaction cancellation (e.g. data for transaction not found or insufficient account balance).
4. **Concurrency control enforcement:**
The concurrency control method may decide to abort the transaction to be restarted later.
5. **Physical problems and catastrophes**

7.1.6 Transaction States & Operations

As mentioned earlier a transaction is an atomic unit of work that is either completed entirely or not done at all. For recovery purposes, the system needs to keep track of the following operations by **DBMS Recovery Manager**:

1. **Begin Transaction**
This marks the beginning of transaction execution.
2. **Read or Write Operations of Database Items in a Transaction**
Some operations in a transaction may cause it fails (e.g. divide by zero or logical programming errors).
3. **End-transaction**
This marks the end of transaction execution. At this point it is necessary to check whether the changes introduced by the transaction can be permanently applied to the database (committed) or whether the transaction must be aborted.
4. **Commit-transaction**
This marks a successful end of a transaction, so that any changes executed by the transaction can be safely committed.
5. **Rollback (Abort)**
This marks an unsuccessful end of a transaction so that any changes performed by transaction to the database must be undone. Figure 7.3 shows the state transition diagram that describes how the transaction moves through its execution states.



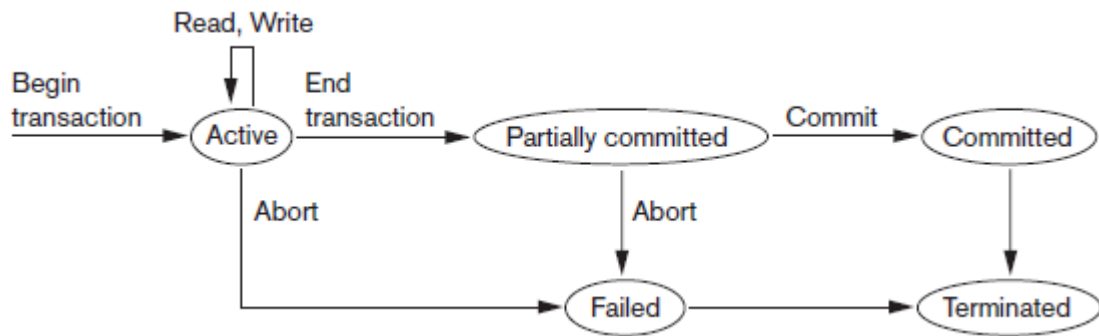


Figure 7.3

Remark:

When transaction ends, it moves to the **Partially Committed State**. At this point, some recovery protocols need to ensure that a system failure will not result, so an inability to record the changes of transaction permanently (this done by recording changes in the **System Log**). Also, at this point the DBMS concurrency control system can force the transaction to a failed state or it may fail and aborted during its active state. Failed or aborted transaction may be restarted later either automatically or after being resubmitted by the user. Aborted or failed transaction changes must be rollback.

7.1.7 The System Log

To be able to recover from failures that affect transactions, the system maintains a **Log** to keep track of all transaction operations that affect the values of database items. The records (entries) that are written to the **Log** have a unique **Transaction-ID (T)**. This key is generated automatically by the system to identify each transaction.

The Log entries have the following forms:

1. [Start-Transaction, T]
2. [Write-Item, T, X, Old-value, [new-value]]
3. [Read- Item, T, X]
4. [Commit, T]

Indicates that transaction T has completed successfully, and its effect can be committed (recorded permanently to the database).

5. [Abort, T]



Commit Point of a Transaction

A transaction T reaches its commit point when all its operations that access the database have been executed successfully and the effect of all the transaction operations on the database have been recorded in the **Log**. Beyond the Commit point, the transaction is said to be committed, and its effect is assumed to be permanently recorded in the database. The transaction then writes a commit record [commit, T] into the Log. If a system failure occurs, we search back in the Log for all transactions T that have written a [Start-Transaction, T] record in the Log but have not written their [Commit, T] record yet; these transactions **may** have to be rolled back to Undo their effect on the database during the recovery process.

Transactions that have written their commit record in the Log must also have recorded all their write operations in the Log, so their effect on the database can be redone from the Log records.

Desirable Properties of Transactions

Transactions should possess several properties. These are often called the **ACID** properties. The following are the **ACID** properties:

1. Atomicity:

A transaction is an atomic unit of processing to be performed entirely or not performed at all. It is the responsibility of the transaction recovery subsystem.

2. Consistency preservation:

A transaction is consistency preserving, execution takes the database from a consistent state to another consistent state. It is the responsibility of the programmers who write the database program to enforce integrity constraints. A consistent state of the database satisfies the constraints specified in the schema.

3. Isolation:

A transaction execution should not be interfered with any other transactions executing concurrently. Isolation may be enforced by hiding (invisible) its updates to other transactions until it is committed (this will avoid dirty read problem). It is responsibility of the concurrency control subsystem.

4. Durability:

The changes applied to the database by a committed transaction must persist in the database. These changes must not be lost because of any failure. It is responsibility of the recovery subsystem of the DBMS.



Schedules (Histories) of transactions

Schedule:

It is order of execution of operations from various transactions that are concurrently executed in an interleaved fashion.

A Schedule (History) S of n transactions ($T_1, T_2, \dots, T_n$) is an ordering of the operations of the transactions subject to the constraint that, for each transaction T_i in S , the operations of T_i must appear in the same order in which they occur in T_i .

Schedule Examples:

1. Schedule of Figure (3 (a))

$S_a: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$

2. Schedule of Figure (3 (b))

$S_b: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); a_1;$

Conflict Operations in a Schedule

Two operations in a schedule are said to be conflict if they satisfy all the following three conditions:

1. They belong to different transactions
2. They access the same database item.
3. At least one of the operations is write- item.

Examples of the conflicting operations:

$r_1(X)$ and $w_2(X)$ in schedule S_a .
 $r_2(X)$ and $w_1(X)$ in schedule S_a .
 $w_1(X)$ and $w_2(X)$ in schedule S_a .

Examples of the non-conflicting operations:

$r_1(X)$ and $w_1(X)$ in schedule S_a .
 $w_2(X)$ and $w_1(Y)$ in schedule S_a .



Complete Schedule

A schedule S of n transactions $(T_1, T_2, \dots, T_n)$ is said to be complete if the following conditions hold:

1. The operations in S are exactly those operations in $(T_1, T_2, \dots, T_n)$ including a commit or abort operation as the last operation for each transaction in the schedule.
2. For any pair of operations from the same transaction T_i , their order of appearance in S is the same as their order of appearance in T_i .
3. For any two conflicting operations, one of the two must occur before the other in the schedule (i.e., ordering of non-conflicting operations in two different transactions is not important).

Committed Projection of a schedule

The committed projection $c(S)$ of a schedule S includes only the operation in S that belong to committed transactions (i.e. T_i that has C_i operation)

7.2 Characterizing Schedules Based on Recoverability

Recoverable Schedules: They are schedules which ensure that any committed Transaction. Should never be a necessity to rollback. A schedule S is recoverable if no transaction T in S commits until all transaction T' have written on item that T reads (after the write item of T')

Examples:

$S_c: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); c_2; a_1;$

Not Recoverable (T_2 must abort $\rightarrow T_1$ abort)

$S_d: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2;$

Recoverable Postponed until T_1 commits

$S_e: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$

Recoverable Cascading Rollback (time consuming)

Check the schedule S : $r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); c_1;$

Remark:

A Schedule is said to be cascade less, or to avoid cascading rollback, if every transaction in the schedule reads only items that were written by committed transactions. In this case, all items read will not be discarded, so no cascading rollback will occur. To satisfy this criterion, the $r_2(X)$ command in schedules S_d and S_e must be postponed until after T_1 has committed (or aborted), thus delaying T_2 ensuring no cascading rollback if T_1 aborts.



Another Example:

Determine if the following Schedule is recoverable or unrecoverable:

S: $r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); c_1;$
 S: $r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); a_1;$

Strict Schedules: transactions in these schedules can neither read nor write item x until the last transaction that wrote x has committed (or aborted). Strict Schedules simplify the recovery process. In a strict Schedule, the process of undoing a write item (x) operation of an aborted transaction is simply to restore the before image (old value or BFIM) of data item x (because there is no interference with other transactions). This simply procedure always works correctly for strict schedules, but it may not work for recoverable or cascade less schedules. For example, consider schedule S: $w_1(x, s); w_2(x, 8); a_1$ (it is not strict Schedule).

Suppose that the value of x was originally 9, which is the BFIM stored in the system log along with $w_1(x, s)$ operation. If T_1 aborts, as in S, the recovery procedure will restore the value of x to 9, even though it has already been changed to 8 by transaction t_2 , thus leading to in correct result. Although schedule S is cascade less, it is not a strict schedule since it permits T_2 to write item x even though the transaction T_1 that last wrote x had not yet committed (or aborted). A strict schedule does not have this problem.

We have now characterized schedules according to the following terms:

1. Recoverability
2. Avoidance of cascading rollback
3. strictness

We have thus seen that those properties of schedules are successively more stringent condition. Thus condition (2) implies condition (1) and condition (3) implies both (2) and (1). Thus, all strict schedules are cascade less and all cascade less are recoverable.

7.3 Characterizing Schedules Based ON Serializability

The concept of Serializability of schedules is used to identify which schedules are correct when transaction executions have interleaving of their operations in the schedules. If no interleaving is considered in the transaction execution, it is called serial schedules otherwise it is called non serial. If a schedule has n transaction, then there are $n!$ serial schedules.

Suppose that two users submit to the DBMS transactions T_1 & T_2 at approximately the same time. So, there will be $2!$ serial scheduler and many possible orders in which the system can execute the individual operations of the of the transactions T_1 & T_2 if interleaving execution is considered. Some of these alternatives are shown in figure 7.4.



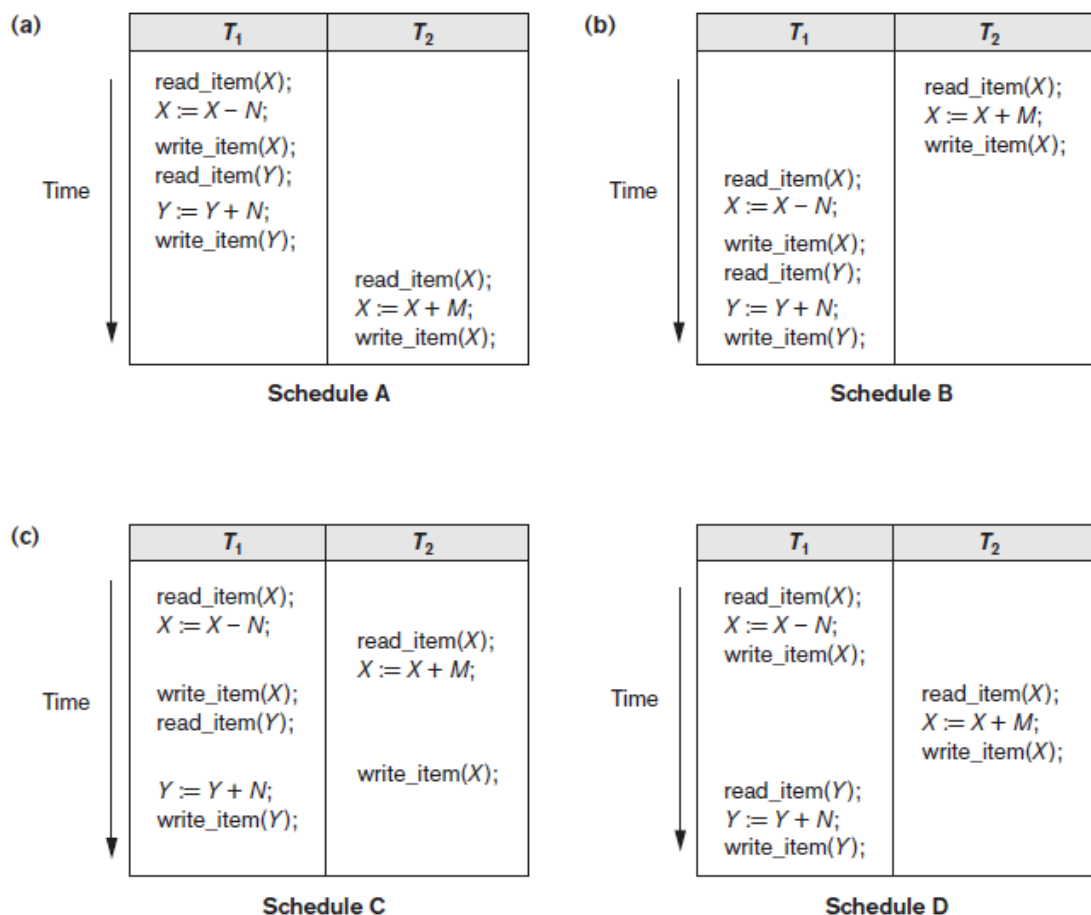


Figure 7.4

7.3.1 Conflict – Serializable Schedules

Formally, a schedule **S** is serial if, for every transaction **T** participating in the schedule, all the operations of **T** are executed consecutively (no concurrency) in the schedule, otherwise, the schedule is called non serial.

One reasonable assumption we can make, if we consider the transactions to be independent (no conflicting operations), is that every serial schedule is considered correct. Note that if there is a dependency, serial schedules of the same transaction may have different effect on the database. The main problem with serial schedules is that they prevent interleaving. So, we do not utilize CPU time efficiently and they are unacceptable in practice.

Serializable schedule (conflict serializable schedule)

A schedule **S** of **N** interleaved transactions is Serializable if it is conflict equivalent to some serial schedules of the same **N** transactions.

Conflict equivalence of schedules:

Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules. For example, if a read and write operation occur in the order $r_1(X)$, $w_2(X)$ in schedule **S1** and in the reverse order $w_2(X)$, $r_1(X)$ in schedule **S2**, the value read by $r_1(X)$ can be different in the two schedules.



Remarks:

1. If the schedule S is Serializable, in such a case we can reorder the non-conflicting operations of S until we form the equivalent serial schedule S'. Schedule D (shown in Figure 7.4) is equivalent to the serial schedule A.
2. Result equivalent of schedule cannot be used to define equivalence of schedules (see example Figure 7.5).

S_1	S_2
<code>read_item(X);</code> <code>$X := X + 10$;</code> <code>write_item(X);</code>	<code>read_item(X);</code> <code>$X := X * 1.1$;</code> <code>write_item(X);</code>

Figure 7.5 Two serial schedules may have different effect on the Database if they are dependent (generate different results).

7.3.2 Algorithm for testing conflict Serializability of schedules:**Algorithm: Testing Conflict Serializability of a Schedule S**

1. For each transaction T_i participating in schedule S, create a node labelled T_i in the precedence graph.
2. For each case in S where T_j executes a `read_item(X)` after T_i executes a `write_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.
3. For each case in S where T_j executes a `write_item(X)` after T_i executes a `read_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.
4. For each case in S where T_j executes a `write_item(X)` after T_i executes a `write_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles.

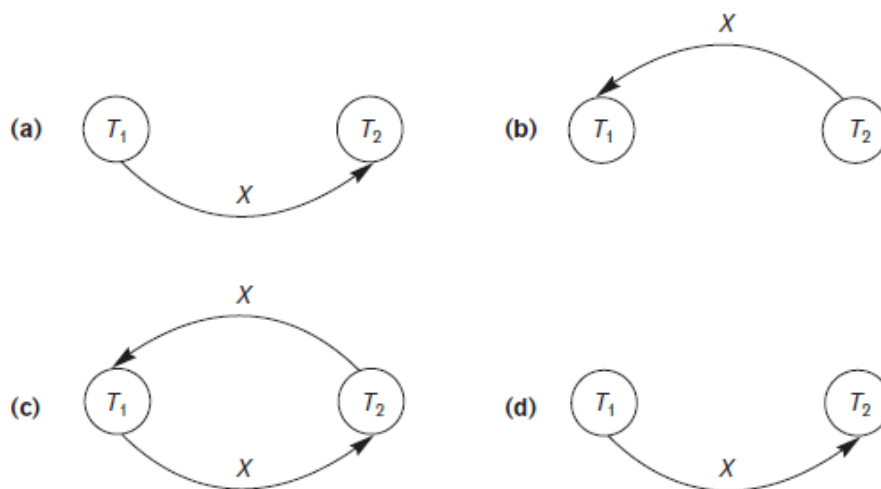


Figure 7.6



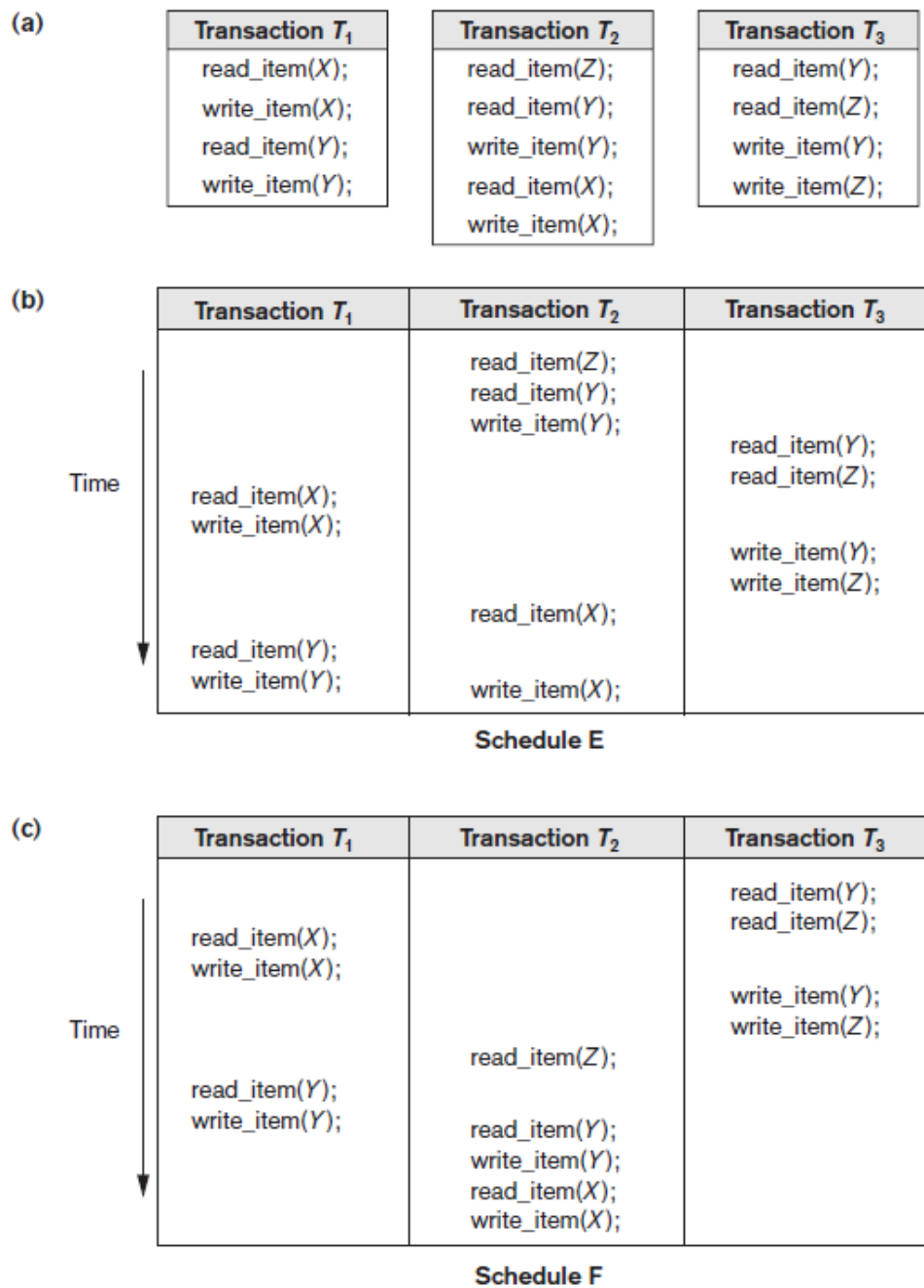


Figure 7.7



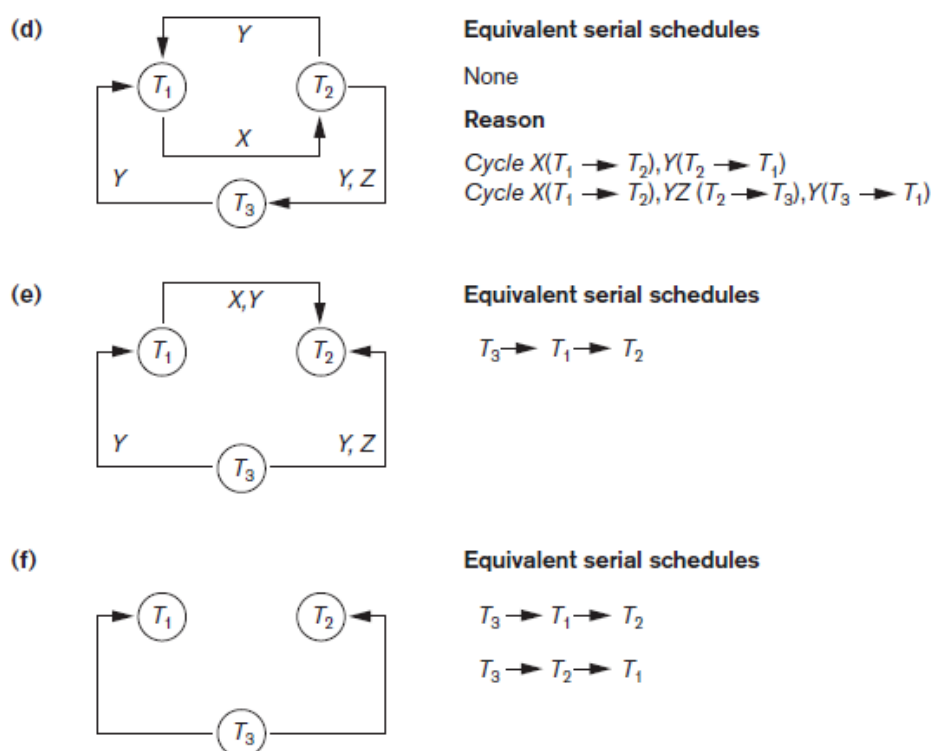


Figure 7.8

7.4 Recovery Concepts

7.4.1 Categorization of recovery algorithms:

Recovery from transactions failures means that the database is restored to the most recent consistent state just before the time of failure using the transaction information stored in the system log.

Conceptually, two techniques for recovery from non-catastrophic transaction failures:

1. Deferred update
2. Immediate update

Deferred update:

This technique does not physically update the database on disk until after a transaction reaches its commit point. Before reaching commit, all transaction updates are recorded in the local transaction workspace (or buffers). During commit, the updates are recorded first in the log and then written to the database.

If a transaction fails before reaching its commit point, it will not have changed the database in any way, so UNDO is not needed.

The technique is also called NO_REDO /REDO algorithm, hence some committed transaction may be required to be redone.



Immediate update:

In this technique the database may be un-updated by some operations of a transaction before the transaction reaches its commit point. However, these operations are recorded in the log on disk by force writing before they are applied to the database, making recovery still possible. So, this algorithm is called UNDO|REDO algorithm. (to give a chance for a transaction to fail and recovery from failure is possible).

7.4.2 Caching (Buffering) of disk blocks

1. **Caching** (buffering) of disk pages is an operating system function but because of its importance to the efficiency of recovery procedures, it's handled by the DBMS through calling low-level operating system routines.
2. **DBMS cash:** Collection of in-memory buffers to hold database disk pages (blocks).
3. **Cache Directory:** It's a database table that is used to keep track of which database items are in the **DBMS** cash. This table may contain *<disk page address, buffer location, dirty bit, pin_unpin bit>*.
4. **Handling cache directory:** When the **DBMS** requests action on some item, it first checks the cache directory to determine whether the disk page containing the item is in the cache. If it's not, then the item must be loaded on disk and the appropriate disk pages are copied into the cache.

It may be necessary to replace (or flush) some of the cache buffers to space available for the new item. The replacement strategies may be:

- **LRU** (Least recently used) page.
 - **FIFO** page.
5. **Dirty Bit:** It's bit associated with each buffer in the DBMS cache. When a page is first read from the database disk into a cache buffers, the cache directory is updated with the new disk page address, and the dirty bit is set to zero. As soon as the buffer is modified the dirty bit for the corresponding directory entry is set to 1 (one). When the buffer contents are replaced (flushed) from the cache, the contents must be written back to the corresponding disk page only if its dirty bit is 1.

So, the dirty bit is used to indicate whether the cached page (buffer) is modified or not.

6. **Pin_Unpin Bit:** A page in **DBMS** cache is pinned (bit=1) if it cannot be written back to disk yet.



Two main strategies can be used when flushing a modified buffer back to the disk:

1. In-place updating
2. Shadowing updating

In-place updating

Writes the buffer back the same original disk function (overwriting old values).

Shadowing

Writes an update buffer at a different disk location, so multiple versions of data items can be maintained.

The BFIM, the old value of the data item before updating is the before image.

The AFIM, the new value of data item after updating (After image).

In shadowing both **BFIM** & **AFIM** can be kept on disk which facilitates recovery process.

7.4.3 Transaction rollback

If a transaction fails for whatever reason after updating the database, it may be necessary to roll back the transaction. If any data item values have been changed by the transaction and written to the database, they must be restored to their previous values (**BFIMs**). The undo type log entries are used to restore the old values of data items that must be rolled back.

Definition

Cascading Rollback: If a transaction (T) is rolled back, any transaction (S) that has read the value of some item (X) written by (T), then (S) also must be rolled back.

The transaction (S) rolling back may affect other transactions to be rolled back as well and so on. This phenomenon is called cascading rollback and can occur when the recovery protocol ensures recoverable schedules but does not ensure strict or cascade less schedules. See example Figure 7.9.



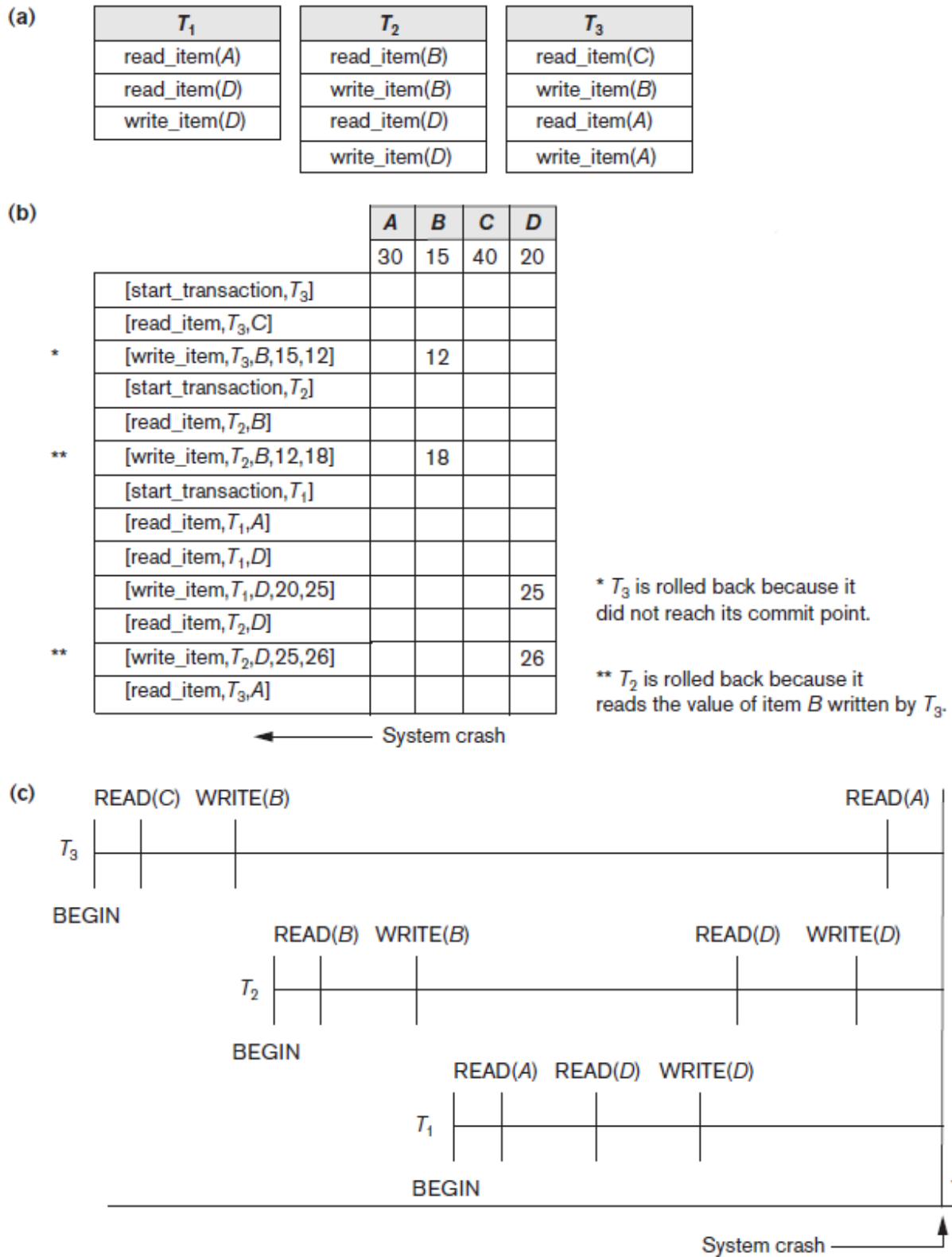


Figure 7.9



7.4.4 Checkpoint

It's a type of entry in the log file. A [checkpoint] record is written into the log periodically at that point when the system writes out to the database on disk all DBMS buffers that have been modified.

As consequence of this, all transactions that have their [commit, T] entries in the log before a [checkpoint] entry don't need to have their write operations redone in case of a system crash, since all their updates will be recorded in the database on disk during checkpointing.

The recovery manager of a DBMS must decide at what intervals to take a checkpoint. The interval may be measured in time (e.g. every m minutes) or the number (t) of committed transactions since the last checkpoint, where the values (m) or (t) are system parameters.

Taking a checkpoint consists of the following actions:

1. Suspend execution of transactions temporarily.
2. Force-write all main memory buffers that have been modified to disk.
3. Write a [checkpoint] record to the log, and force-write the log to disk.
4. Resume executing transactions.

Fuzzy Checkpointing Techniques:

In this technique, the system can resume transaction processing after a [begin_checkpoint] record is written to the log without having to wait for **step 2** to finish. When **step 2** is completed, an [end_checkpoint, ...] record is written in the log with the relevant information collected during checkpointing. However, until **step 2** is completed, the previous checkpoint record should remain valid. To accomplish this, the system maintains a file on disk that contains a pointer to the valid checkpoint, which continues to point to the previous checkpoint record in the log. Once **step 2** is concluded, that pointer is changed to point to the new checkpoint in the log.

7.5 The ARIES Recovery Algorithm

We now describe the **ARIES (Algorithm for Recovery and Isolation Exploiting Semantics)** algorithm an example of a recovery algorithm used in database systems. ARIES uses a **Steal / No_Force** approach to specify when a page from the DBMS cache can be written to disk.

- **Steal:** When protocol allows writing an updated page buffer before transaction commits
- **No Force:** When transaction commits, it's not necessary to write immediately all pages updated by this transaction)

So, this algorithm doesn't require large DBMS cache (steal) and it eliminates the I/O cost to read pages again from disk (No_Force).



This algorithm has three phases:

1. Analysis phase
2. Redo phase
3. Undo phase

1. Analysis phase:

At crash:

1. Read from log appended file the LSN of last checkpoint (Form special file created during checkpointing). This checkpoint is a fuzzy checkpoint.
2. Read from the log actions starting from LSN of the last checkpoint.
3. Read the End checkpoint and recall both (These tables are created during checkpointing)
 - Transaction table
 - Dirty page table
4. Modify the above tables to determine both the redo & undo sets of operations.

2. Redo phase:

1. Determine **M** (the smallest **LSN** of **LSNs** exists in the dirty page table after the analysis phase)
2. Scan log from **LSN** equal to **M** until end of log.
3. For each entry in the log:
 - i. Let **N**= the **LSN** of the entry.
 - ii. Let **P**= the **page ID** of entry.
 - iii. Let **D**=the **LSN** of **P** in the dirty page table
 - iv. If **P** is not exists in the Dirty page table:
 1. Then this change is already written to disk before the checkpoint and need not to be reapplied.
 - v. Else If **N**< **D**.
 1. Then this change has been applied to the page.
 - vi. Else
 - vii. this change (update) must be re-applied.

3. Undo phase:

Starts by scanning backward from the end of the log in the undo transaction Set (active [not committed] transactions). Undo set is determined from transaction table.

1. Starts scanning backward from the end of the log in the undo transaction set (active [Not Committed] transactions)
2. Undo the transaction (the appropriate actions)
3. For each undo operation, compensating these changes in the log.

The recovery completes at the end of undo phase.



(a)

Lsn	Last_lsn	Tran_id	Type	Page_id	Other_information
1	0	T_1	update	C	...
2	0	T_2	update	B	...
3	1	T_1	commit		...
4	begin checkpoint				
5	end checkpoint				
6	0	T_3	update	A	...
7	2	T_2	update	C	...
8	7	T_2	commit		...

(b)

Transaction_id	Last_lsn	Status
T_1	3	commit
T_2	2	in progress

Page_id	Lsn
C	1
B	2

(c)

Transaction_id	Last_lsn	Status
T_1	3	commit
T_2	8	commit
T_3	6	in progress

Page_id	Lsn
C	7
B	2
A	6

Figure 7.10

Figure 7.10 shows an example of recovery in ARIES. (a) The log at point of crash. (b) The Transaction and Dirty Page Tables at time of checkpoint. (c) The Transaction and Dirty Page Tables after the analysis phase.

Important Remarks:

Checkpointing in **ARIES** creates besides the log, two tables:

1. Transaction table.
 2. Dirty page table.
- The transaction table contains an entry for each active transaction, with information such as the **transaction ID**, **status** and **LSN** of the most recent log record for the transaction.
 - The dirty page table contains page (Not performed yet).an entry for each dirty page in the buffer which includes the **page ID** and **LSN** corresponding to the earliest update to that.



Exercises

- 1 - What is the difference between the UNDO/REDO and the UNDO/NO-REDO algorithms for recovery with immediate update? Develop the outline for an UNDO/NO-REDO algorithm.
- 2 - Describe the shadow paging recovery technique. Under what circumstances does it not require a log?
- 3 - Describe the three phases of the ARIES recovery method.
- 4 - What do the terms steal/no-steal and force/no-force mean with regard to buffer management for transaction processing?



Chapter 8 Distributed Databases

Beside centralized database, that resides on a single hardware machine, with associated secondary storage devices such as disks for on-line database storage and tapes for backup. In recent years, there has been a rapid trend toward the distribution of computer systems over multiple sites that are connected via communication network.

8.1 Reasons for Distribution and DDBMS functions:

A distributed DB is a collection of data that belong logically to some systems, but it is physically spread over the sites of a computer network. Several factors have led to the development of DDBSs.

8.1.1 The advantage of DDBSs as follows:

1. **Distributed nature of some DB applications:** Many database applications are naturally distributed over different locations.
2. **Increased reliability and availability:** Reliability is broadly defined as the probability that the system is up at a particular moment in time, whereas the availability is the probability that the system is continuously available during a time interval. When the data and DBMS software are distributed over several sites, one site may fail while other sites continue in operation. Only the data and software that exist at the failed site cannot be accessed, other data and software can still be used.
3. Allowing data sharing while maintaining some measure of local control. In DDBSs, it is possible to control the data and the software locally at each site. However, certain data can be accessed by other remote sites through the DDBMS software.
4. Improve performance by distributing a large database over multiple sites, smaller database will exist at each site. Each site will have a smaller number of transactions executing than if all transactions were submitted to a single centralized database. Distribution leads to increase complexity in the system design and implementation.

To satisfactorily achieve the advantages listed above, the DDBMS software must be able to provide additional functions. **Some of these are:**

1. To access remote sites and transmit queries and data among the various sites via a communication network.
2. To keep track of the data distribution and replication in the DDBMS catalog.
3. To devise execution strategies for queries and transactions that access data from more than one site.
4. To decide on which copy of a replicated data item to access.
5. To maintain the consistency of copies of a replicated data item.
6. To recover from individual site crashes and from new types of failures such as failure of a communication link.



8.1.2 Architecture of a DDES

At the physical hardware level, the main factors that distinguish a DDBS from a centralized system are the following:

- There are multiple computers, called sites or nodes.
- These sites must be connected by some type of communication network to transmit data and commands among sites, as shown in the following Figure 8.1:

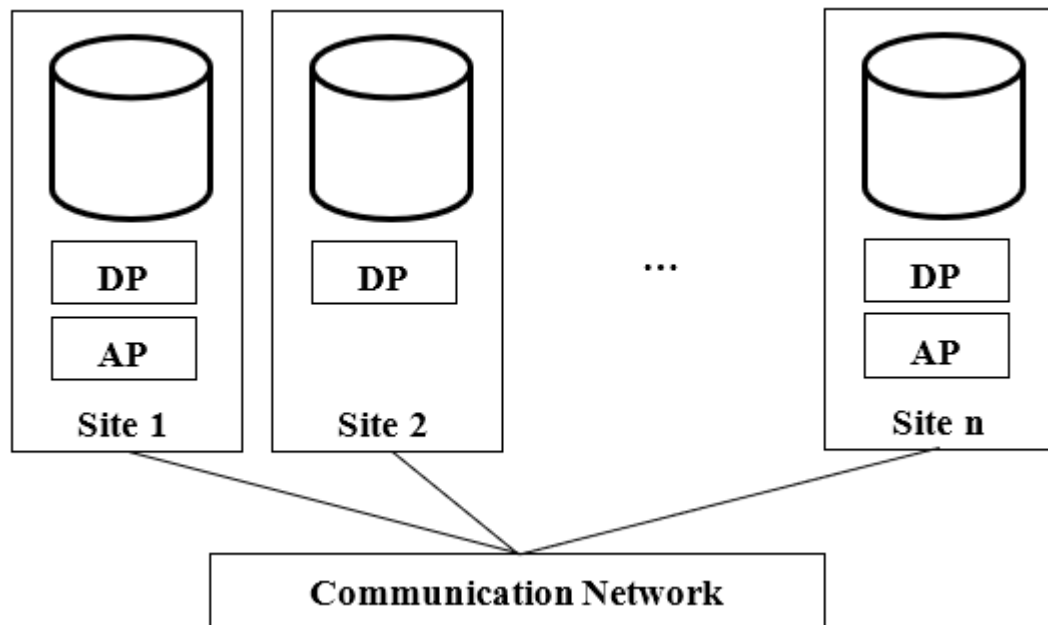


Figure 8.1

The sites may all be in the same building or typically within a mile radius and connected via a local area network, or they may be geographically distributed over large distances and connected via a long-level network. Local area networks typically use cable, whereas long-level networks use telephone lines or satellites.

8.1.3 Application Processors and Data Processors

To manage the complexity of a DDBMS, it is customary to divide software into three main modules:

- The data processor (DP) software is responsible for local data management at a site, much like centralized DBMS software.
- The Application processor (AP) software is responsible for most of the distribution functions; it accesses data distribution information from the DDBMS catalog and is responsible for processing all requests access to more than one site.
- The communication software provides the communications primitives that are used by the AP to transmit commands and data among the various sites as needed.

In a DDBS it is possible that some sites contain both AP and DP software, whereas other sites contain only one or the other as illustrated in the previous figure.

A site used mainly for DB function is called a **back-machine**, and a site that is used primarily for the AP function is called a **front-end machine**.



Distribution Transparency

An important function of the AP is to hide the details of data distribution from the user; that is, the user should write global queries and transactions as through the database were centralized, without having to specify the sites at which the data referenced in global queries and transactions reside. This property is called Distribution Transparency.

8.2 Data Fragmentation, Replication, and Allocation Techniques

This section discusses the techniques that are used to break up the database into logical units, called fragments that may be assigned for storage at various sites. In addition, it discusses the use of data Replication, so certain data may be stored in more than one site, as well as the process of allocating fragments or replicates of fragments.

The information concerning data fragmentation, allocation, and replication is stored in a global system catalog that is accessed by AP as needed.

8.2.1 Data Fragmentation

Before distributing the data, the logical units of the database must be determined. The simplest logical units are the relations themselves; that is, each whole relation will be stored at a particular site. However, in many cases the relation can be divided into smaller logical units for distribution.

We have three different types of fragmentation:

1. Horizontal Fragmentation

A horizontal fragmentation of a relation is a subset of the tuples in that relation. The tuples that belong to the horizontal fragment are specified by a condition on one or more attributes of the relation.

2. Vertical Fragmentation

Another type of fragmentation is called vertical fragmentation. A vertical fragment of a relation keeps only certain attributes in the relation that are related together in some way.

3. Mixed fragmentation

Both vertical and horizontal fragments can be intermixed yielding mixed fragments.

Remark: The original relation can be reconstructed by applying union and outer join.



8.2.2 Data Replication and Allocation

Replication is useful in improving the availability of data. The most extreme case is replication of the whole database at every site in the distributed system, thus creating a fully replicated distributed database.

This can improve availability remarkably because the system can continue to operate if at least one site is up, it also improves performance of retrieval for global queries because the result of such a query can be obtained locally from one site.

The disadvantage of full replication is that it can slow down update operations drastically because a single logical update must be performed on every copy of the database to keep the copies consistent.

Also, full replication makes recovery techniques more expensive than if there was no replication.

The other extreme from full replication is to have no replication; that each fragment is stored at exactly one site. In this case, all fragments must be disjoint, except for the repetition of primary keys among vertical (or mixed) fragments. This technique is called **non-redundant allocation**.

Between these two extremes, we have a wide spectrum of partial replication of the data; that is some fragments of the database may be replicated whereas others are not.

The number of copies of each fragment can range from one to the number of sites in the distributed system.

A description of the replication of fragments is sometimes called a replication schema. Each copy of a fragment and the fragment itself must be assigned to a particular site in the distributed system.

This process is called data distribution. The degree of replication depends on some factors:

1. Performance and availability goals of the system.
2. Types and frequencies of transactions submitted at each site.

Example of Fragmentation, Allocation and Replication:

We now consider an example of fragmenting and distributing the company database previously mentioned. Suppose that the company has three computer sites -one for each current department. Sites 2 & 3 for departments 5 & 4 respectively. At each of these sites, we expect frequent access to the *EMPLOYEE* and *PROJECT* information for the employees who work in that department and the projects controlled by that department.



Further, we assume that these sites mainly access the *NAME*, *SSN*, *SALARY*, and *SUPERSSN* attributes of *EMPLOYEE*. Site 1 is used by company headquarter and accesses all employee and project information regularly, in addition to keeping track of Dependent information for insurance purposes. The following figures explains this example:

(a) Employees in Department 5

G1

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0

C1 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 5))

G2

<u>Essn</u>	<u>Pno</u>	Hours
333445555	10	10.0

C2 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 4))

G3

<u>Essn</u>	<u>Pno</u>	Hours
333445555	20	10.0

C3 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 1))

(b) Employees in Department 4

G4

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

C4 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 5))

G5

<u>Essn</u>	<u>Pno</u>	Hours
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0

C5 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 4))

G6

<u>Essn</u>	<u>Pno</u>	Hours
987654321	20	15.0

C6 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 1))

(c) Employees in Department 1

G7

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

C7 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 5))

G8

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

C8 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 4))

G9

<u>Essn</u>	<u>Pno</u>	Hours
888665555	20	Null

C9 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 1))

Figure 8.2



(a) EMPD_5

Fname	Minit	Lname	Ssn	Salary	Super_ssn	Dno
John	B	Smith	123456789	30000	333445555	5
Franklin	T	Wong	333445555	40000	888665555	5
Ramesh	K	Narayan	666884444	38000	333445555	5
Joyce	A	English	453453453	25000	333445555	5

DEP_5

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22

DEP_5_LOCS

Dnumber	Location
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON_5

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0

PROJS_5

Pname	Pnumber	Plocation	Dnum
Product X	1	Bellaire	5
Product Y	2	Sugarland	5
Product Z	3	Houston	5

Data at site 2

(b) EMPD_4

Fname	Minit	Lname	Ssn	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	25000	987654321	4
Jennifer	S	Wallace	987654321	43000	888665555	4
Ahmad	V	Jabbar	987987987	25000	987654321	4

DEP_4

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Administration	4	987654321	1995-01-01

DEP_4_LOCS

Dnumber	Location
4	Stafford

WORKS_ON_4

Essn	Pno	Hours
333445555	10	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0

PROJS_4

Pname	Pnumber	Plocation	Dnum
Computerization	10	Stafford	4
New_benefits	30	Stafford	4

Data at site 3

Figure 8.3



8.3 Types of Distributed Database System

The term distributed data base management system can be applied to describe a variety of systems that differ from one another in many respects

The main factors that differentiate distributed systems are:

- Degree of homogeneity of the DDBMS software.
If all DPS use identical software and all APS use the same software, the DDBMS is called homogenous; otherwise is called heterogeneous.
- Degree of local autonomy:
If all access to DDBMS must go through an AP, then the system has no local autonomy. On the other hand, if direct access by local transaction to a DP is permitted, the system has some degree of local autonomy.
- Degree of distribution, transparency or alternatively the degree of schema integration:
If the user sees a single integrated schema without any information concerning fragmentation, replication, or distribution, the DDBMS is said to have a high degree of distribution transparency (or schema integration). However, in different case if the user sees all fragmentation, allocation, and replication, the DDBMS has no distribution transparency and no schema integration.

8.3.1 Concurrency Control and Recovery in Distributed Database

For concurrency control and recovery purposes, numerous problems arise in distributed DBMS environment. **Some of these problems are the following:**

- **Dealing with multiply copies of data items**
The concurrency control method is responsible for maintaining consistency among these copies. The recovery method is responsible for: making a copy consistent with other copies if the site on which the copy is stored fails and recovers later.
- **Failure of individual site**
The DDBMS should continue to operate with its running sites if possible when one or more individual sites fails. When a site recovers, its local database must be brought up to date with the rest of the sites before it rejoins the system.
- **Failure of communication links**
The system must deal with failure of one or more of the communication links that connect the sites.
- **Distributed commit**



Problems arise with committing a transaction that is accessing database stored on multiple sites if some sites fail during the commit process, the two-phase commit protocol is often used to deal with this problem.

- **Distributed deadlock**

Deadlock may occur among several sites, so techniques for dealing with deadlock must be extended to take this into account.

8.4 Distributed Recovery

The recovery process in distributed database is quite involved. We will give only a very brief idea of some of the issues here. In some cases, it is quite difficult even to determine when a site is down without exchanging numerous messages with other sites.

For example, suppose that site X sends a message to site Y and expects a response from X but does not receive it; there are several possible explanations

- The message was not delivered to Y because of failure.
- Site Y is down and could not respond.
- Site Y is running and sent a response, but the response was not delivered.

Without additional information or sending of additional messages, it is difficult to determine what happened.

Another problem with Distributed recovery is distributed commit. When a transaction is updating data at several sites, it cannot commit until it is sure that the effect of transaction on every site cannot be lost. This means that every site must have recorded the local effects of the transactions permanently in the local site log on disk.



Exercises

- 1 - What are the main reasons for and potential advantages of distributed databases?
- 2 - What additional functions does a DDBMS have over a centralized DBMS?
- 3 - How is a horizontal partitioning of a relation specified? How can a relation be put back together from a complete horizontal partitioning?
- 4 - How is a vertical partitioning of a relation specified? How can a relation be put back together from a complete vertical partitioning?



Chapter 9 Data warehouse & Data Mining

9.1 What is a data warehouse?

Data warehousing provides architectures and tools for business executives to systematically organize, understand, and use their data to make strategic decisions. Many organizations have found that data warehouse systems are valuable tools in today's competitive, fast evolving world. In the last several years, many firms have spent millions of dollars in building enterprise-wide data warehouses. Many people feel that with competition mounting in every industry, data warehousing is the latest must-have marketing weapon to keep customers by learning more about their needs.

9.1.1 Data Warehouse

A Data Warehouse refers to a database that is maintained separately from an organization's operational databases. Data warehouse systems allow for the integration of a variety of data sources. They support information processing by providing a solid platform of consolidated, historical data for analysis.

Alternatively, a Data Warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision-making process.

Subject-oriented: A data warehouse is organized around major subjects, such as customer, vendor, product, and sales. rather than concentrating on the day-to-day operations and transaction processing of an organization, a data warehouse focuses on the modeling and analysis of data for decision makers.

Integrated: A data warehouse is usually constructed by integrating multiple heterogeneous sources, such as relational databases, flat files, and on-line transaction records. Data cleaning and data integration techniques are applied to ensure consistency in naming conventions, encoding structures, attribute measures, and so on.

Time-variant: Data are stored to provide information from a historical perspective (e.g., the past 5-10 years). Every key structure in the data warehouse contains, either implicitly or explicitly, an element of time.



Nonvolatile: A data warehouse is always a physically separate store of data transformed from the application data found in the operational environment. Due to this separation, a data warehouse does not require transaction processing and recovery mechanisms. It usually requires only two operations in data accessing: initial loading of data and access of data.

9.2 OLTP & OLAP

OLTP Systems: They are on-line operational database systems which perform on-line transaction and query processing.

OLAP Systems: They are Data warehouse systems, on the other hand, serve users or “knowledge workers” (managers, analysts, executives) in the role of data analysis and decision support. Such systems facilitate summarization, aggregation, present data in various formats in order to accommodate the diverse needs of the different users.

9.2.1 The major distinguishing features for OLTP and OLAP

1. **Users and system orientation:** an OLTP system is customer-oriented and is used for transaction and query processing by clerks, clients, and information technology professionals. An OLAP system is market-oriented and is used for data analysis by knowledge workers, including managers, executives, and analysts.
2. **Data contents:** An OLTP system manages current data that, typically, are too detailed to be easily used for decision making. An OLAP system manages large amounts of historical data. It provides facilities for summarization and aggregation of data to discover knowledge at different levels of granularity. These features make the data easier for use in decision support.
3. **Database design:** An OLTP system usually adopts an entity-relationship (ER) data model and an application-oriented database design. An OLAP system typically adopts either a star or snowflake model (to be discussed in next sections). These models are subject-oriented database designs to facilitate data analysis tasks.



4. **View (Scope):** An OLTP system focuses mainly on the current data within an enterprise or department (operational database), without referring to historical data or data in different organizations. In contrast, OLAP systems deal with information that originates from different organizations (by the analysis of large volumes of integrated historical data).

9.2.2 Why separate data warehouse from operational databases is recommended?

Since operational databases store huge amounts of data", you observe, “why not perform on-line analytical processing directly on such databases instead of spending additional time and resources to construct a separate data warehouse?”

Reasons

1. A major reason for such a separation is **to promote the high performance of both systems. Processing OLAP queries in operational databases would substantially degrade the performance of operational tasks.**
2. On the other hand, data warehouse queries are often complex. They involve the computation of large groups of data at summarized levels, and require implementation methods based on multidimensional views (i.e. DW is an appropriate platform for data analysis).

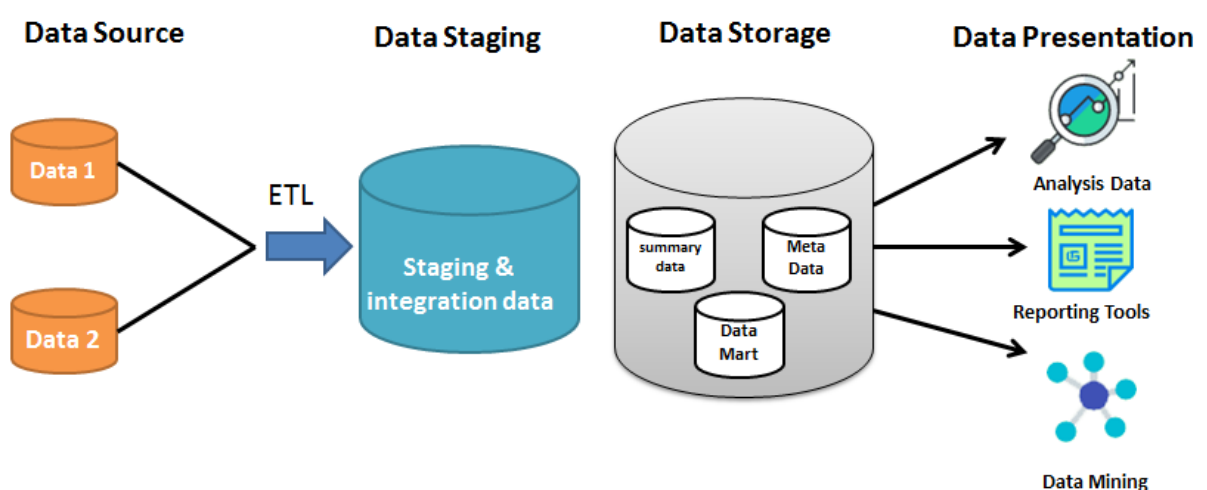


Figure 1-1 Architecture of typical data warehouse

9.2.3 Data Sources for the Mining Process

Data Repositories for the mining process may be :

- **Transactional Database**

In general, a transactional database consists of a file where each record represents a transaction.

A transaction typically includes a unique transaction identity number (trans ID), and a list of the items making up the transaction (such as items purchased in a store).

- **Relational Database**

A relational database is a collection of tables, built using RDBMS, each of which is assigned a unique name. Each table consists of a set of attributes (columns or fields) and usually stores many tuples (records or rows).

9.3 Data Warehouse

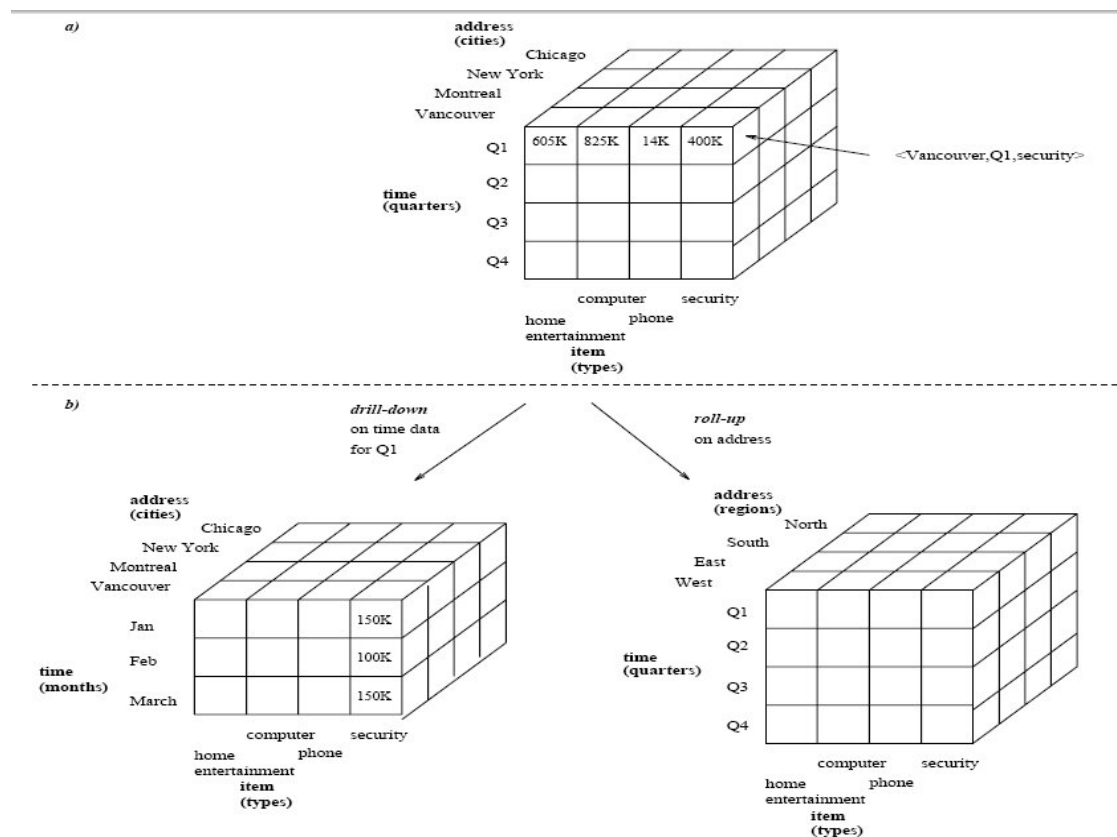
A **Data Warehouse** is a repository of information Collected from multiple sources, stored under a unified schema, and which usually resides at a single site. Data warehouses are constructed via a process of data cleaning, data transformation, data integration, data loading, and periodic data refreshing.

In order to facilitate decision making, the data in a data warehouse are organized around major subjects, such as customer, item, supplier, and activity. The data are stored to provide information from a historical perspective (such as from the past 5-10 years) and are typically summarized. For example, rather than storing the details of each sales transaction, the data warehouse may store a summary of the transactions per item type for each store, or summarized to a higher level, for each sales region.

A data warehouse is usually modeled by a multidimensional database structure, where each dimension corresponds to an attribute or a set of attributes in the schema, and each cell stores the value of some aggregate measure such as count or sales amount. It provides



a multidimensional view of data and allows the pre-computation and fast accessing of summarized data.



- **Base Cuboid:** it is the data cube structure that stores the primitive or lowest level of information.
- **(Non-base) Cuboids:** they correspond to a higher level multidimensional (cube) structure.
- **Apex Cuboid:** the 0_D cuboid which holds the highest level of summarization i.e., the last possible roll up operation.
- **Data Cube:** It is a base cuboid together with (Non-base) Cuboids.
- **Drill-Down and Roll Up:** which allow the user to view the data at differing degrees of summarization.

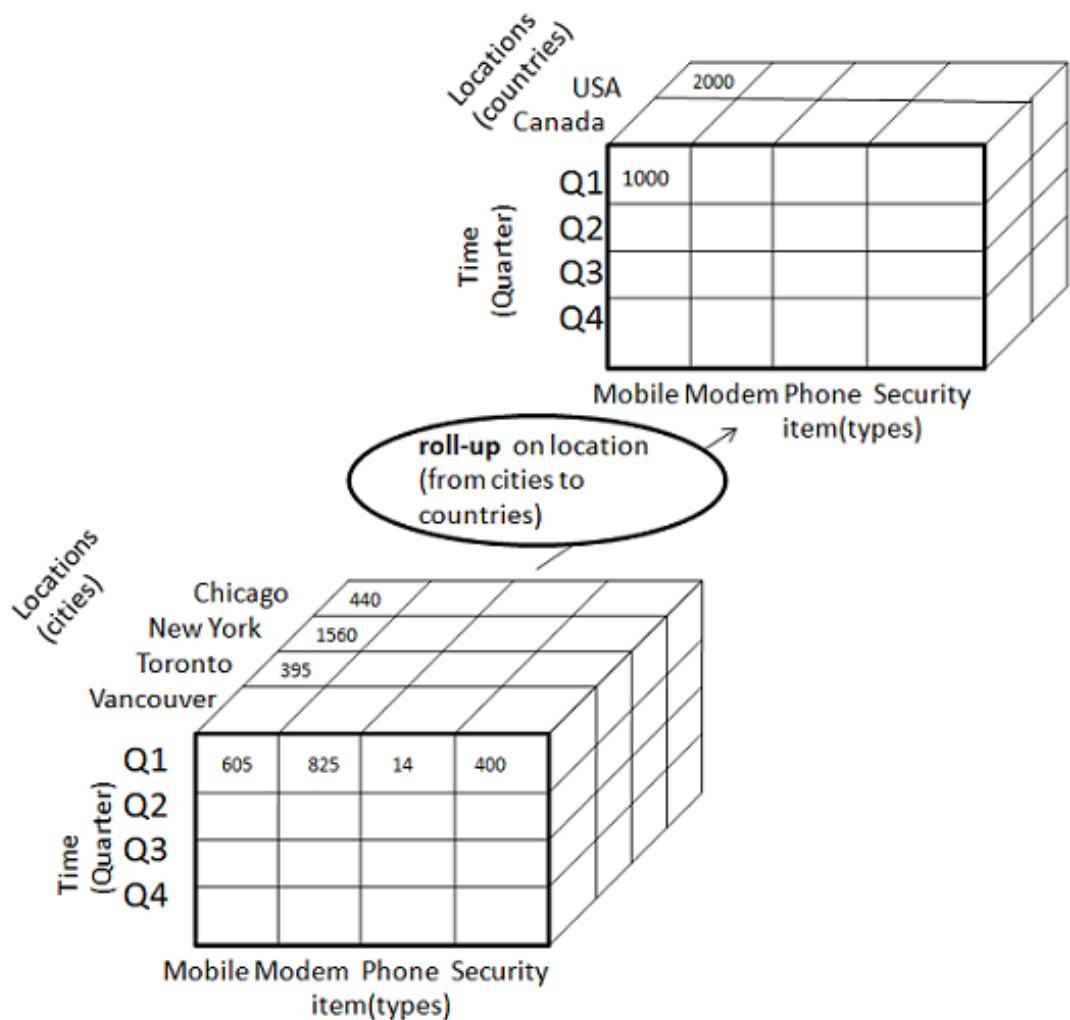
9.3.1 OLAP Operations

- Roll-up
- Drop-down
- Slice
- Dice

Roll-Up

Roll-up performs aggregation on a data cube in any of the following ways:

- By climbing up a concept hierarchy for a dimension.
- By dimension reduction.

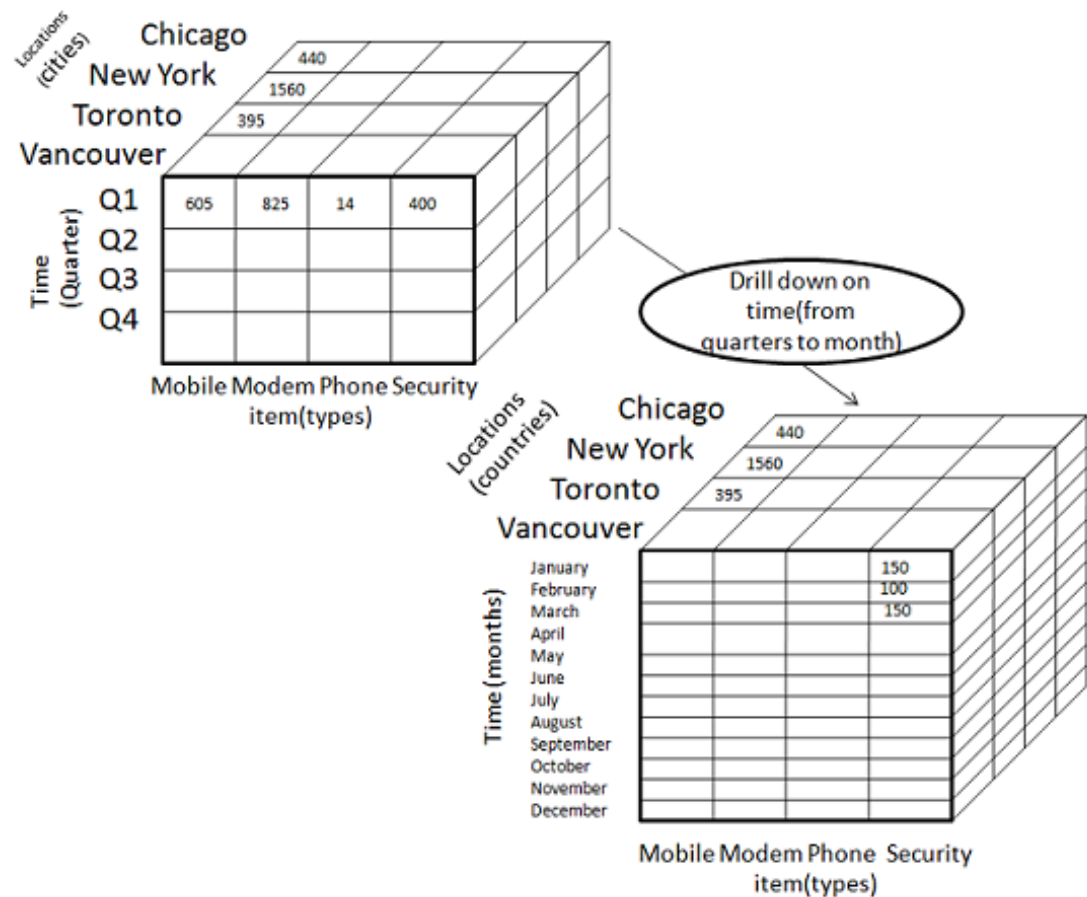


Drill-Down



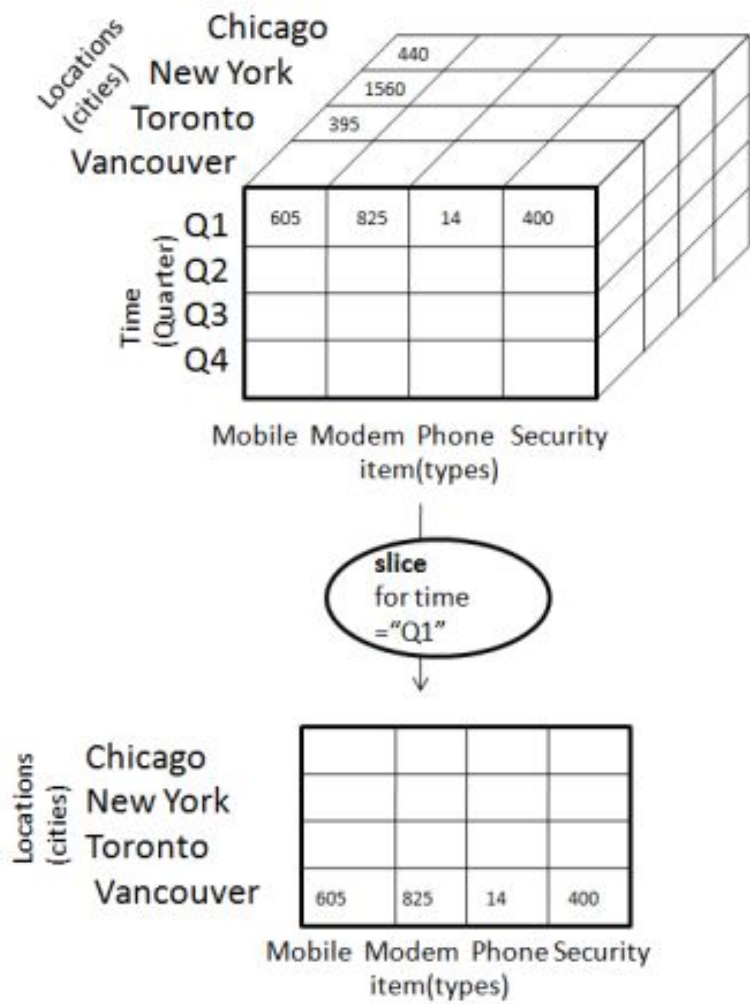
Drill-down is the reverse operation of roll-up. It is performed by either of the following ways:

- By stepping down a concept hierarchy for a dimension.
- By introducing a new dimension.



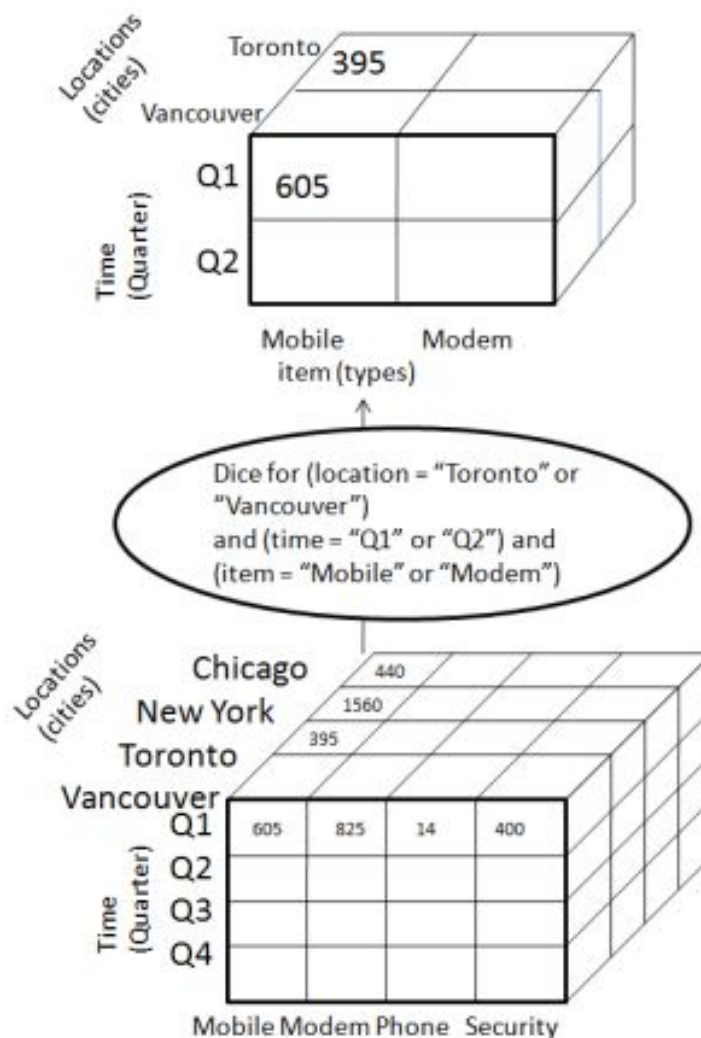
Slice

The slice operation selects one item from one dimension from a given cube and provides a new sub-cube. Consider the following diagram that shows how slice works.



Dice

Dice selects two or more items from one or more dimension(s) from a given cube and provides a new sub-cube. Consider the following diagram that shows the dice operation.



Remark

By providing multidimensional data views and the pre-computation of summarized data, data warehouse systems are well suited for On-Line Analytical Processing, or OLAP (simple and basic analysis tools).

9.4 Multiple Dimension Data Model

- **Data Cube**

Allows data to be modeled and viewed in multiple dimensions.



- **Dimensions**

Are the perspectives or entities (Subject-oriented) with respect to which an organization wants to keep records for knowledge discovery.

For example, “All Electronics” company may create a sales data warehouse in order to keep records of the store's sales with respect to the dimensions time, item, branch, and location.

- **Dimension Table**

It is the table which describes the dimension.

For example, a dimension table for item may contain the attributes item name, brand, and type.

- **Facts**

Numerical measures by which we want to analyze relationships between dimensions.

Examples of facts for a sales data warehouse include dollars sold (sales amount in dollars), units sold (number of units sold)

- **Fact Table:**

Represents the theme for which a multidimensional data model is organized. it contains the names of facts or measures as well as the keys to each of the related dimension table



Examples of data Cubes

A 2-D view of sales data for “All Electronics” according to the dimensions time and item, where the sales are from branches located in the city of Vancouver. The measure displayed is dollars sold.

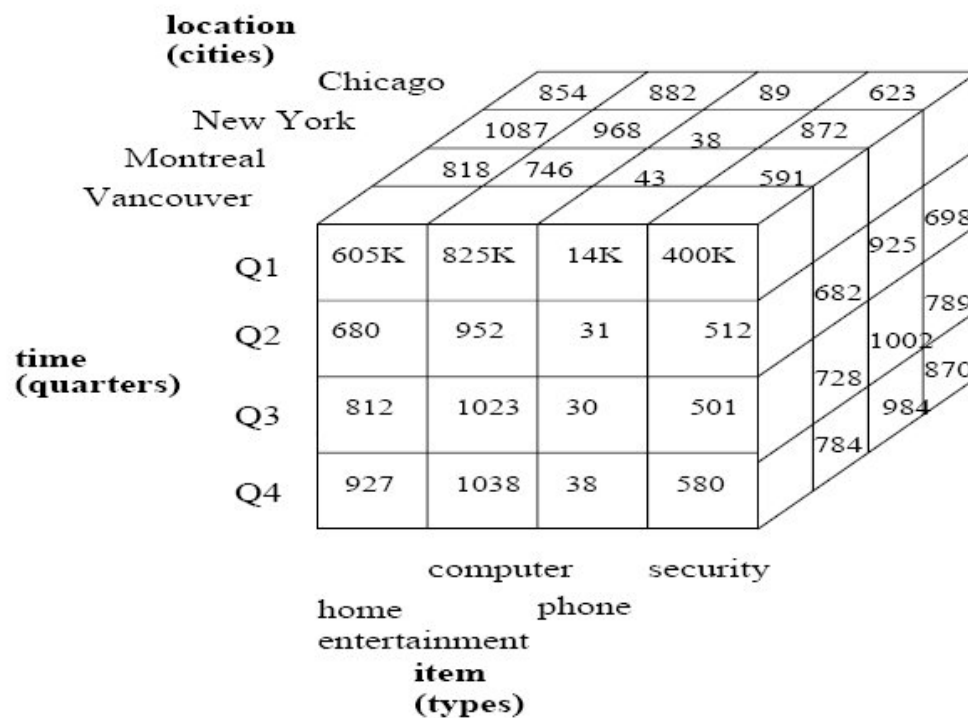
time (quarter)	Sales for all locations in Vancouver			
	item (type)			
	home entertainment	computer	phone	security
Q1	605K	825K	14K	400K
Q2	680K	952K	31K	512K
Q3	812K	1023K	30K	501K
Q4	927K	1038K	38K	580K

A 3-D view of sales data for “All Electronics”, according to the dimensions time, item, and location. The measure displayed is dollars sold, as a series of 2D

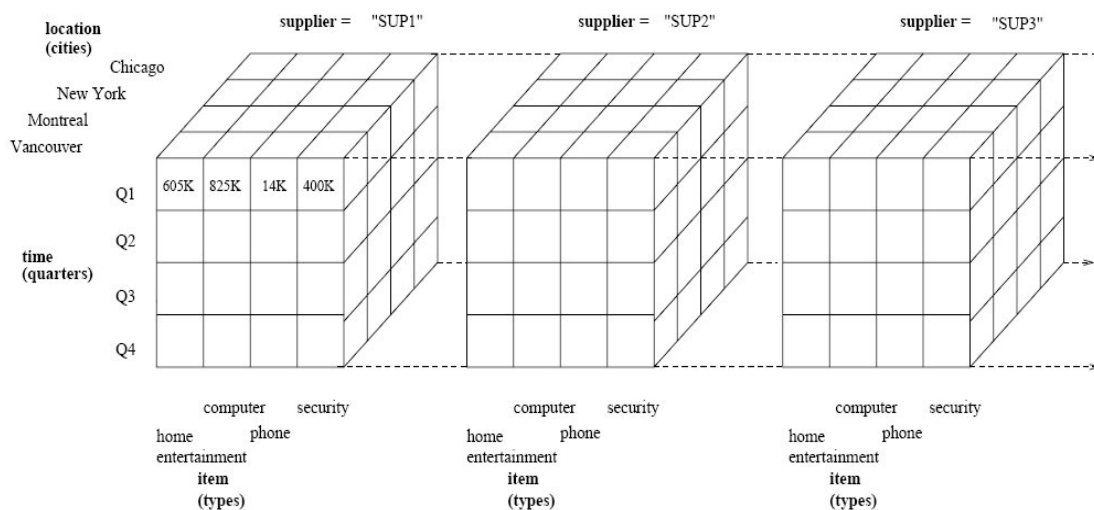
t i m e	location = “Vancouver”				location = “Montreal”			
	item				item			
	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.
Q1	605K	825K	14K	400K	818K	746K	43K	591K
Q2	680K	952K	31K	512K	894K	769K	52K	682K
Q3	812K	1023K	30K	501K	940K	795K	58K	728K
Q4	927K	1038K	38K	580K	978K	864K	59K	784K

For supplier (SUP1) a 3-D data cube representation of the data according to the dimensions time, item, and location. The measure displayed is dollars sold.

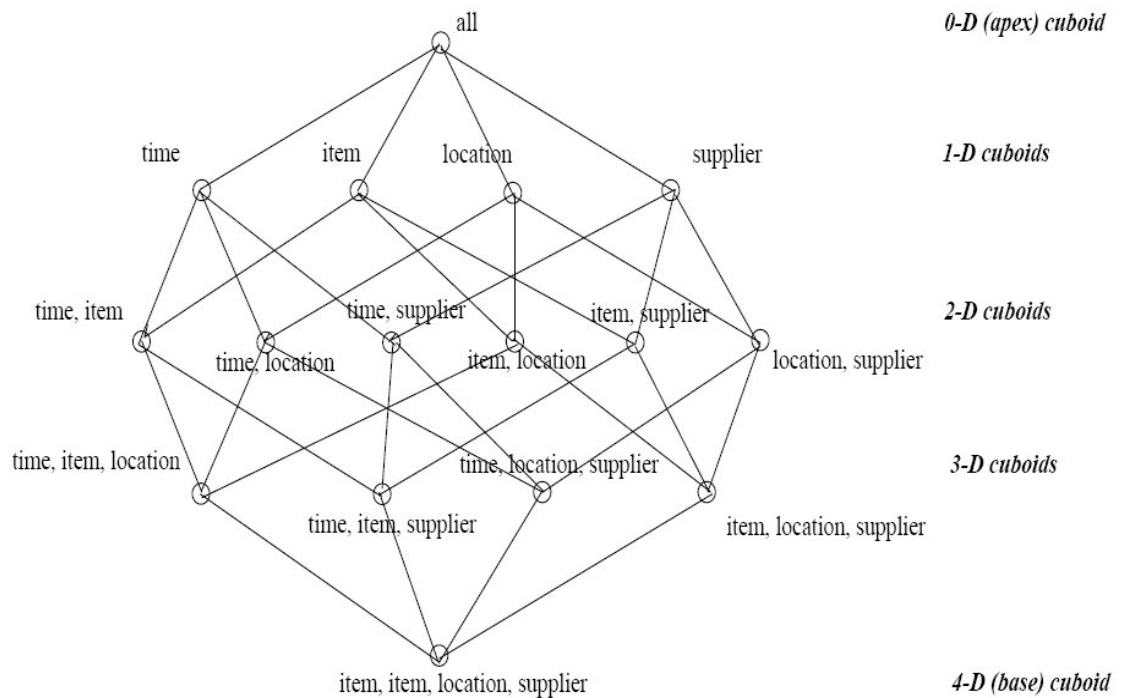




A 4-D data cube representation of sales data, according to the dimensions time, item, location, and supplier. The measure displayed is dollars sold as a series of 3D.



- **Base cuboid:** The cuboid which holds the lowest level of summarization for every dimension
- **Apex cuboid:** The 0-D cuboid which holds the highest level of summarization (zero dimension).

**Note:**

Lattice of cuboids, making up a 4-D data cube for the dimensions time, item, location, and supplier. each cuboid represents a different degree of summarization.

9.4.1 Schemas for multidimensional Databases.

1. Stars schema.
2. Snowflake schema.
3. Fact Constellations schema.

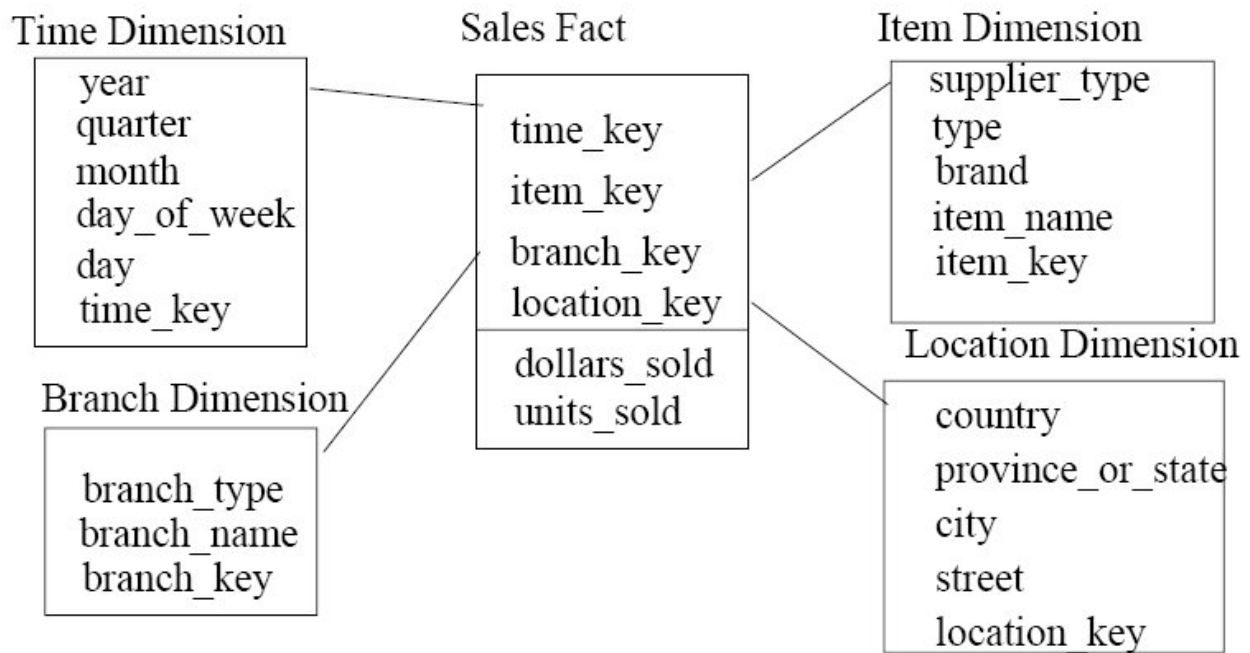
Star schema

The star schema is a modeling in which the data warehouse contains:

1. A large central table (fact table)
2. A set of smaller attendant tables (dimension tables), one for each dimension

The schema graph resembles a starburst





An example of a star schema. “All Electronics” Sales are considered along four dimensions, namely time, item, branch, and location. The schema contains a central fact table for sales which contains keys to each of the four dimensions, along with two measures: dollars sold, and units sold.

Notice that in the star schema, each dimension is represented by only one table, and each table contains a set of attributes.

For example, the location dimension table contains the attribute set location key, street, city, state, country. This constraint may introduce some redundancy.

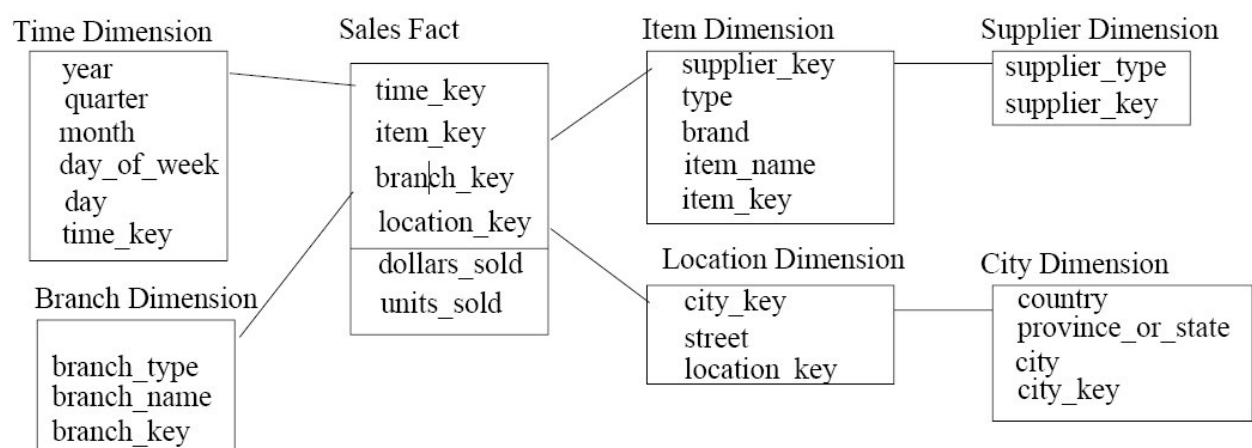
For example, “Vancouver” and “Victoria” are both cities in the Canadian state British Columbia. Entries for such cities in the location dimension table will create redundancy among the attributes state and country, i.e..., Vancouver, British Columbia, Canada) and (.., Victoria, British Columbia, Canada).



Snowflake schema

The snowflake schema is a variant of the star schema model, where some dimension tables are normalized, (further splitting the dimension tables into additional tables). The resulting schema graph forms a shape like a snowflake.

The major **difference** between the snowflake and star schema models is that the dimension tables of the snowflake model are in normalized form. Such a table is easy to maintain and saves storage space. Since much of this space is redundant data, creating a normalized structure will reduce the overall space requirement. However, the snowflake structure can reduce the effectiveness of browsing since more joins will be needed to execute a query. Consequently, the system performance may be adversely impacted. Performance benchmarking can be used to determine what is best for your design.



An example of a snowflake schema for “All Electronics” sales is given in the previous figure. Here, the sales fact table is identical to that of the star schema.

The main difference between the two schemas is in the definition of dimension tables. The single dimension table for item in the star schema is normalized in the snowflake schema, resulting in new item and supplier tables.

For example, the item dimension table now contains the attributes supplier key, type, brand, item name, and item key, the latter of which is linked to the supplier dimension table, containing supplier type and supplier key information.



Similarly, the single dimension table for location in the star schema can be normalized into two tables: new location and city. The location key of the new location table now links to the city dimension.

Notice that further normalization can be performed on province or state and country in the snowflake schema when desirable.

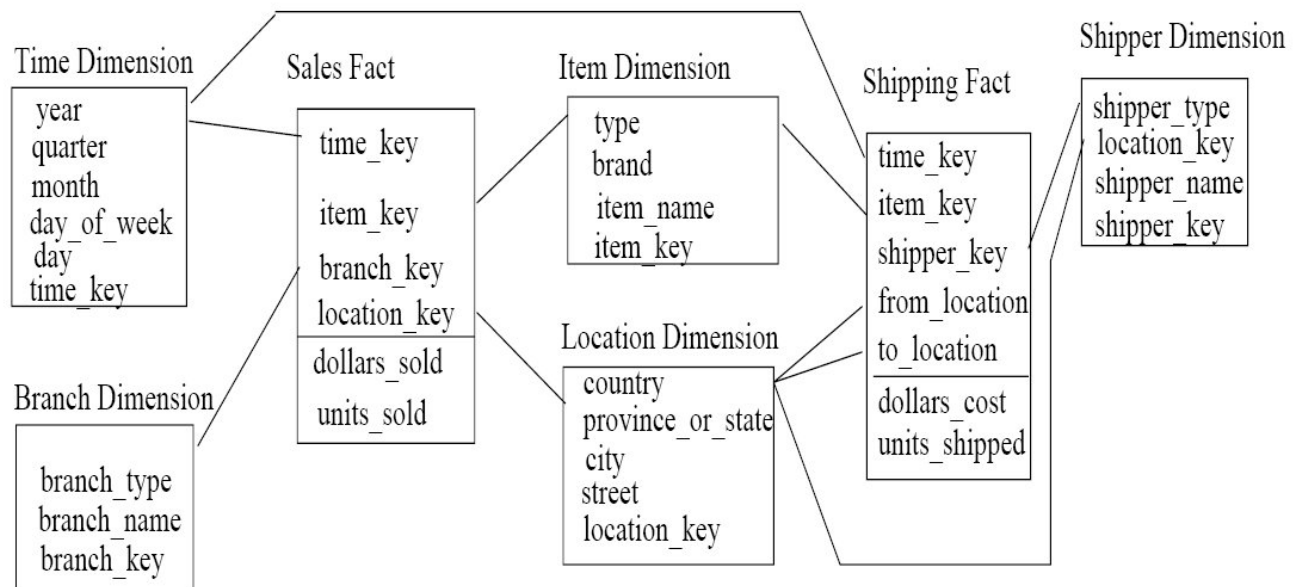
Remark

A compromise between the star schema and the snowflake schema is to adopt a mixed schema where only the very large dimension tables are normalized. Normalizing large dimension tables saves storage space, while keeping small dimension tables unnormalized may reduce the cost and performance degradation due to joins on multiple dimension tables. Doing both may lead to an overall performance gain. However, careful performance tuning could be required to determine which dimension tables should be normalized and split into multiple tables.



Fact Constellation Schema

Sophisticated applications may require multiple fact tables to share dimension tables. This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation.



An example of a fact constellation schema, this schema specifies two fact tables, sales, and shipping. The sales table definition is identical to that of the star schema. The shipping table has five dimensions, or keys: time key, item key, shipper key, from location, and to location, and two measures: dollars cost, and units shipped.

A fact constellation schema allows dimension tables to be shared between fact tables. For example, the dimensions tables for time, item, and location, are shared between both the sales and shipping fact tables.

Remarks

- A **Data Warehouse**: collects information about subjects that span the entire organization, such as customers, items, sales, assets, and personnel, and thus its scope is enterprise wide. For data warehouses, the fact constellation schema is commonly used since it can model multiple, interrelated subjects.
- A **Data Mart**: is a department subset of the data warehouse that focuses on selected subjects, and thus its scope is department wide. For data marts, the star or snowflake schema are popular since each are geared towards modeling single subjects.



Exercises

- 1 - What are the different phases of the knowledge discovery from databases?
- 2 - Describe a complete application scenario in which new knowledge may be mined from an existing database of transactions.
- 3 - What are the goals or tasks that data mining attempts to facilitate?
- 4 - Describe the characteristics of a data warehouse. Divide them into the functionality of a warehouse and the advantages users derive from the warehouse.
- 5 - What is the multidimensional data model? How is it used in data warehousing?



References

1. Fundamentals of Database Systems, 7th Edition, Elmasri & Navathe.
2. Modern Database Management, 10th Edition, Jeffrey Hoffer, Ramesh & Heikki Topi.

